# SMART CAMPUS AI

## INTELLIGENT COMPLAINT MANAGEMENT SYSTEM

Complete Technical Source Code & Multi-Agent Architecture Specification

| | |
|---|---|
| **Project Name** | Smart Campus AI - Intelligent Complaint Management System |
| **Primary Stacks** | Python 3.12 \| Streamlit \| LangGraph \| ChromaDB \| SQLite |
| **Architecture** | 5 Cooperative AI Agents + Dynamic Agentic RAG Vector Retrieval |
| **Portals Included** | Student Portal \| 7 Department Workspaces \| Central Admin Console |
| **Target Version** | v2.4.0 (GitHub Ready Production Codebase) |
| **Database** | SQLite with PBKDF2 Password Hashing & Full Audit Trail |
| **Total Codebase** | 30 Files \| 4,544 Total Lines of Clean Python & Markdown Code |
| **Repository Status** | Initialized on 'main' branch, clean .gitignore, zero emojis |

**SYSTEM OVERVIEW:** Smart Campus AI is an enterprise-grade academic grievance redressal platform. It features five cooperative LangGraph agents: *Agent 1 (Understanding)* decomposes natural-language text; *Agent 2 (Classification)* categorizes grievances into 7 campus domains; *Agent 3 (Priority)* computes multi-factor urgency scores (Low, Medium, High, Critical) with clear rationales; *Agent 4 (Department Routing)* assigns responsible departments and detects ambiguity; and *Agent 5 (Ticket & Tracking)* generates sequential ticket identifiers (CAMP-XXXX), writes transactional records, and dispatches in-app alerts to department queues.

Agentic RAG integrates ChromaDB with campus policy documents to ground priority escalations and department procedures. Operational field stations for all 7 departments allow technician dispatch, hardware replacement tracking, and immediate work completion triggers that synchronize across the university governance cockpit.

# TABLE OF CODEBASE MODULES & FILES

| # | File Path | Type | Lines | Module / Purpose |
|---|---|---|---|---|
| 1 | app.py | Python | 184 lines | Streamlit Multi-Page Entry Point, Session State & Navigati... |
| 2 | config.py | Python | 182 lines | System Constants, Department Directories, SLA Matrix & See... |
| 3 | database.py | Python | 708 lines | SQLite Database Schema, PBKDF2 Password Hashing, CRUD & An... |
| 4 | test_system.py | Python | 157 lines | Automated 9-Stage Multi-Agent System Verification Test Sui... |
| 5 | requirements.txt | Config | 11 lines | Python Runtime Dependencies & Package Versions |
| 6 | .env.example | Config | 6 lines | Environment Variables Configuration Template |
| 7 | .gitignore | Config | 36 lines | Git Exclusion Rules for Repository Cleanliness |
| 8 | agents/state.py | Python | 45 lines | TypedDict ComplaintState Schema for LangGraph Workflow |
| 9 | agents/understanding_agent.py | Python | 87 lines | Agent 1: Natural Language Complaint Understanding & Intent... |
| 10 | agents/classification_agent.py | Python | 75 lines | Agent 2: Academic & Facility Taxonomy Classification |
| 11 | agents/priority_agent.py | Python | 84 lines | Agent 3: Multi-Factor Priority Detection (Low, Med, High, ... |
| 12 | agents/routing_agent.py | Python | 105 lines | Agent 4: Autonomous Department Routing & Ambiguity Detecti... |
| 13 | agents/ticket_agent.py | Python | 40 lines | Agent 5: Ticket Creation, Sequential ID Generator & Persis... |
| 14 | agents/workflow.py | Python | 126 lines | LangGraph StateGraph Coordinator with Agentic RAG Dynamic ... |
| 15 | rag/vector_store.py | Python | 205 lines | ChromaDB Vector Store with Fast Local Vectorizer & Resilie... |
| 16 | rag/rag_engine.py | Python | 74 lines | Agentic RAG Dynamic Router, Heuristic Trigger & Context Re... |
| 17 | knowledge_base/college_policies.md | Markdown | 23 lines | Campus Code of Conduct, Student Responsibilities & Exam Co... |
| 18 | knowledge_base/laboratory_guidelines.md | Markdown | 23 lines | Computing Laboratories, Workstation Protocols & Hardware G... |
| 19 | knowledge_base/hostel_rules.md | Markdown | 21 lines | Residential Living, Maintenance Protocols & Curfew Regulat... |
| 20 | knowledge_base/department_contacts.md | Markdown | 22 lines | Official Campus Department Directory, Room Numbers & Helpd... |
| 21 | knowledge_base/campus_maintenance_procedures.md | Markdown | 19 lines | Campus Maintenance SLA Turnaround Matrix & Escalation Leve... |
| 22 | knowledge_base/complaint_management_policy.md | Markdown | 20 lines | Student Grievance Redressal Charter, SLA Guarantees & Appe... |
| 23 | ui/styles.py | Python | 440 lines | Global SaaS CSS Styling, Metric Card Themes, Badges & Colo... |
| 24 | ui/landing.py | Python | 237 lines | Enterprise SaaS Landing Page, Feature Highlights, Departme... |
| 25 | ui/login_pages.py | Python | 213 lines | Dedicated Portal Sign-In Screens for Student, Department S... |
| 26 | ui/student_portal.py | Python | 374 lines | Student Portal: Complaint Submission, AI Execution Trace, ... |
| 27 | ui/department_page.py | Python | 392 lines | Department Operations Field Station: Alerts, Roster Dispat... |
| 28 | ui/admin_dashboard.py | Python | 439 lines | Central Executive Governance Console: KPIs, Department Ana... |
| 29 | ui/kb_browser.py | Python | 105 lines | Campus Knowledge Base Browser & Live ChromaDB Semantic Sea... |
| 30 | ui/workflow_view.py | Python | 91 lines | Interactive Multi-Agent Architecture Viewer & LangGraph St... |

# MODULE 1: CORE SYSTEM RUNTIME & DATABASE ENGINE

**Overview:** Main application router, central configuration, SQLite persistence, and test harness.

## FILE: app.py

**Type:** Python | **Lines:** 184

*Streamlit Multi-Page Entry Point, Session State & Navigation Router*

Path: smart_campus_ai/app.py

```
0001 | import streamlit as st
0002 | import database as db
0003 | from config import DEPARTMENT_DETAILS
0004 | from ui.styles import get_custom_css
0005 | from ui.landing import render_landing_page
0006 | from ui.student_portal import render_student_portal
0007 | from ui.department_page import render_department_page
0008 | from ui.admin_dashboard import render_admin_dashboard
0009 | from ui.kb_browser import render_kb_browser
0010 | from ui.workflow_view import render_workflow_view
0011 | from ui.login_pages import (
0012 |     render_login_hub,
0013 |     render_student_login_page,
0014 |     render_dept_login_page,
0015 |     render_admin_login_page
0016 | )
0017 |
0018 | # Streamlit Page Configuration (No emojis)
0019 | st.set_page_config(
0020 |     page_title="Smart Campus AI - Complaint Management System",
0021 |     page_icon=None,
0022 |     layout="wide",
0023 |     initial_sidebar_state="expanded"
0024 | )
0025 |
0026 | # Initialize Database Schema & Seed Data
0027 | db.init_db()
0028 |
0029 | # Apply SaaS Custom Styling
0030 | st.markdown(get_custom_css(), unsafe_allow_html=True)
0031 |
0032 | # Navigation helper
0033 | def navigate_to(page_name):
0034 |     st.session_state["current_page"] = page_name
0035 |     st.rerun()
0036 |
0037 | # Initialize Session State
0038 | if "current_page" not in st.session_state:
0039 |     st.session_state["current_page"] = "landing"
0040 |
```

```
0041 | if "student_user" not in st.session_state:
0042 |     st.session_state["student_user"] = None
0043 |
0044 | if "dept_user" not in st.session_state:
0045 |     st.session_state["dept_user"] = None
0046 |
0047 | if "admin_user" not in st.session_state:
0048 |     st.session_state["admin_user"] = None
0049 |
0050 | # Sidebar Navigation (No emojis)
0051 | with st.sidebar:
0052 |     st.markdown("""
0053 |     <div style="padding: 10px 0; border-bottom: 1px solid #E2E8F0; margin-bottom: 15px;">
0054 |         <strong style="color: #0F2744; font-size: 1.15rem; display: block; line-height: 1.2;">Smart Campus AI</strong>
0055 |         <span style="font-size: 0.78rem; color: #64748B; font-weight: 500;">Complaint Management System</span>
0056 |     </div>
0057 |     """, unsafe_allow_html=True)
0058 |
0059 |     current = st.session_state["current_page"]
0060 |
0061 |     # Core Navigation Sections
0062 |     st.markdown("<p style='font-size: 0.75rem; font-weight: 700; color: #94A3B8; text-transform: uppercase; margin-bottom: 6px;'>Navigati
0063 |
0064 |     nav_main = {
0065 |         "landing": "Home & Overview",
0066 |         "student": "Student Portal",
0067 |         "dept_page": "Department Operations",
0068 |         "admin": "Central Admin Console",
0069 |         "kb": "Knowledge Base & RAG",
0070 |         "workflow": "System Architecture"
0071 |     }
0072 |
0073 |     for page_key, label in nav_main.items():
0074 |         is_active = (current == page_key)
0075 |         button_type = "primary" if is_active else "secondary"
0076 |         if st.button(label, key=f"nav_{page_key}", use_container_width=True, type=button_type):
0077 |             navigate_to(page_key)
0078 |
0079 |     st.markdown("<hr style='margin: 1rem 0; border: none; border-top: 1px solid #E2E8F0;'>", unsafe_allow_html=True)
0080 |
```

```
0081 |     # Dedicated Login Pages Access
0082 |     st.markdown("<p style='font-size: 0.75rem; font-weight: 700; color: #94A3B8; text-transform: uppercase; margin-bottom: 6px;'>Sign In
0083 |
0084 |     nav_logins = {
0085 |         "login_student": "Student Sign In",
0086 |         "login_dept": "Department Staff Sign In",
0087 |         "login_admin": "Central Admin Sign In",
0088 |         "login_hub": "All Sign In Options"
0089 |     }
0090 |
0091 |     for log_key, log_label in nav_logins.items():
0092 |         is_active = (current == log_key)
0093 |         button_type = "primary" if is_active else "secondary"
0094 |         if st.button(log_label, key=f"nav_{log_key}", use_container_width=True, type=button_type):
0095 |             navigate_to(log_key)
0096 |
0097 |     st.markdown("<hr style='margin: 1rem 0; border: none; border-top: 1px solid #E2E8F0;'>", unsafe_allow_html=True)
0098 |
0099 |     # Department Quick Switcher
0100 |     st.markdown("<p style='font-size: 0.75rem; font-weight: 700; color: #94A3B8; text-transform: uppercase; margin-bottom: 6px;'>Departme
0101 |     dept_names = list(DEPARTMENT_DETAILS.keys())
0102 |     active_dept = st.session_state.get("active_dept_name", "Lab Maintenance")
0103 |     selected_dept_side = st.selectbox(
0104 |         "Select Department Workspace",
0105 |         dept_names,
0106 |         index=dept_names.index(active_dept) if active_dept in dept_names else 0,
0107 |         label_visibility="collapsed"
0108 |     )
0109 |     if selected_dept_side != active_dept:
0110 |         st.session_state["active_dept_name"] = selected_dept_side
0111 |         if current != "dept_page":
0112 |             navigate_to("dept_page")
0113 |         else:
0114 |             st.rerun()
0115 |
0116 |     st.markdown("<hr style='margin: 1rem 0; border: none; border-top: 1px solid #E2E8F0;'>", unsafe_allow_html=True)
0117 |
0118 |     # Demonstration Trigger Button
0119 |     st.markdown("<p style='font-size: 0.75rem; font-weight: 700; color: #94A3B8; text-transform: uppercase; margin-bottom: 6px;'>Demonstr
0120 |     if st.button("Test Exam Scenario", use_container_width=True, help="Load CSE lab computers down test case"):
```

```
        st.session_state["demo_prefill"] = True
        navigate_to("student")

    st.markdown("<hr style='margin: 1rem 0; border: none; border-top: 1px solid #E2E8F0;'>", unsafe_allow_html=True)

    # Active Sessions Status & Logout
    st.markdown("<p style='font-size: 0.75rem; font-weight: 700; color: #94A3B8; text-transform: uppercase; margin-bottom: 6px;'>Active S

    student_user = st.session_state.get("student_user")
    dept_user = st.session_state.get("dept_user")
    admin_user = st.session_state.get("admin_user")

    has_session = False
    if student_user:
        has_session = True
        st.markdown(f"**Student:** `{student_user['username']}`")
        if st.button("Log Out Student", key="side_logout_student"):
            st.session_state["student_user"] = None
            st.rerun()

    if dept_user:
        has_session = True
        st.markdown(f"**Dept:** `{dept_user['username']}` ({dept_user.get('department', '')})")
        if st.button("Log Out Dept Staff", key="side_logout_dept"):
            st.session_state["dept_user"] = None
            st.rerun()

    if admin_user:
        has_session = True
        st.markdown(f"**Admin:** `{admin_user['username']}`")
        if st.button("Log Out Admin", key="side_logout_admin"):
            st.session_state["admin_user"] = None
            st.rerun()

    if not has_session:
        st.caption("No active user session. Select a portal above to sign in.")

    st.markdown("<div style='margin-top: 1.8rem; font-size: 0.75rem; color: #94A3B8;'>Powered by LangGraph, ChromaDB & SQLite</div>", uns

# Main Application Routing
```

```
current_page = st.session_state.get("current_page", "landing")

if current_page == "landing":
    render_landing_page(navigate_to)
elif current_page == "student":
    render_student_portal()
elif current_page == "dept_page":
    render_department_page()
elif current_page == "admin":
    render_admin_dashboard()
elif current_page == "login_hub":
    render_login_hub(navigate_to)
elif current_page == "login_student":
    render_student_login_page(navigate_to)
elif current_page == "login_dept":
    render_dept_login_page(navigate_to)
elif current_page == "login_admin":
    render_admin_login_page(navigate_to)
elif current_page == "kb":
    render_kb_browser()
elif current_page == "workflow":
    render_workflow_view()
else:
    render_landing_page(navigate_to)
```

## FILE: config.py

**Type:** Python | **Lines:** 182

*System Constants, Department Directories, SLA Matrix & Seed Users*

Path: smart_campus_ai/config.py

```
0001 | import os
0002 | from pathlib import Path
0003 | from dotenv import load_dotenv
0004 |
0005 | # Load environment variables
0006 | load_dotenv()
0007 |
0008 | BASE_DIR = Path(__file__).resolve().parent
0009 | DB_PATH = BASE_DIR / "smart_campus.db"
0010 | UPLOAD_DIR = BASE_DIR / "uploads"
0011 | KNOWLEDGE_BASE_DIR = BASE_DIR / "knowledge_base"
0012 | CHROMA_PERSIST_DIR = BASE_DIR / "chroma_db"
0013 |
0014 | # Create directories if they do not exist
0015 | UPLOAD_DIR.mkdir(parents=True, exist_ok=True)
0016 | KNOWLEDGE_BASE_DIR.mkdir(parents=True, exist_ok=True)
0017 | CHROMA_PERSIST_DIR.mkdir(parents=True, exist_ok=True)
0018 |
0019 | # AI Configurations
0020 | OPENAI_API_KEY = os.getenv("OPENAI_API_KEY", "")
0021 | OPENAI_MODEL = os.getenv("OPENAI_MODEL", "gpt-4o-mini")
0022 | EMBEDDING_MODEL = os.getenv("EMBEDDING_MODEL", "text-embedding-3-small")
0023 |
0024 | # Complaint Categories
0025 | CATEGORIES = [
0026 |     "Laboratory",
0027 |     "Hostel",
0028 |     "Classroom",
0029 |     "Library",
0030 |     "Transport",
0031 |     "Infrastructure",
0032 |     "Other",
0033 | ]
0034 |
0035 | # Priority Levels
0036 | PRIORITY_LEVELS = ["Low", "Medium", "High", "Critical"]
0037 |
0038 | # Status States
0039 | STATUS_LEVELS = ["Open", "In Progress", "Resolved"]
0040 |
```

```
# Canonical Departments
DEPARTMENTS = [
    "Lab Maintenance",
    "Hostel Administration",
    "Estate & Facilities",
    "IT Support",
    "Library Services",
    "Transport Office",
    "Academic Affairs",
    "Unassigned / Review Needed",
]

# Department Detailed Metadata: Themes, Staff Rosters & Contacts
DEPARTMENT_DETAILS = {
    "Lab Maintenance": {
        "slug": "lab_maintenance",
        "head": "Eng. Robert Vance",
        "email": "lab-maintenance@campus.edu",
        "location": "Technology Tower, Ground Floor, Room TT-012",
        "color": "#0284C7",
        "bg_color": "#E0F2FE",
        "border_color": "#7DD3FC",
        "technicians": [
            "Robert Vance (Senior Systems Engineer)",
            "Anita Sharma (Hardware Specialist)",
            "David Miller (Network Technician)",
            "Sunil Rao (Lab Assistant)"
        ],
        "default_username": "dept_lab",
        "description": "Supervises computer laboratories, hardware maintenance, software environments, and exam workstation readiness."
    },
    "Hostel Administration": {
        "slug": "hostel_admin",
        "head": "Dr. Sarah Jenkins (Chief Warden)",
        "email": "hostel-office@campus.edu",
        "location": "Central Student Center, 1st Floor",
        "color": "#4F46E5",
        "bg_color": "#EEF2FF",
        "border_color": "#A5B4FC",
        "technicians": [
            "Dr. Sarah Jenkins (Chief Warden)",
            "Michael Ross (Resident Inspector)",
            "Priya Nair (Sanitation Supervisor)",
            "James Wilson (Hostel Caretaker)"
        ],
        "default_username": "dept_hostel",
        "description": "Governs dormitory allocations, residential plumbing, dining hall hygiene, room maintenance, and hostel curfew."
    },
    "Estate & Facilities": {
        "slug": "estate_facilities",
        "head": "Mr. David Kumar (Director)",
        "email": "estate-facilities@campus.edu",
        "location": "Maintenance Yard, Operations Building",
        "color": "#059669",
        "bg_color": "#ECFDF5",
        "border_color": "#6EE7B7",
        "technicians": [
            "David Kumar (Facilities Director)",
            "Carlos Santana (Senior Electrician)",
            "Sunita Verma (Civil & Water Works)",
            "Alan Baker (HVAC Technician)"
        ],
        "default_username": "dept_estate",
        "description": "Oversees physical campus buildings, classroom air conditioning, electrical grid, plumbing mains, and civil infras
    },
    "IT Support": {
        "slug": "it_support",
        "head": "Ms. Priya Patel (Lead Admin)",
        "email": "it-support@campus.edu",
        "location": "Tech Building, 2nd Floor, Room TB-205",
        "color": "#7C3AED",
        "bg_color": "#F5F3FF",
        "border_color": "#C4B5FD",
        "technicians": [
            "Priya Patel (Lead Network Admin)",
            "Alex Chen (Systems Administrator)",
            "Kevin Wright (Student Helpdesk)",
            "Fatima Zahra (Cybersecurity Analyst)"
        ],
        "default_username": "dept_it",
```

```
        "description": "Manages campus Wi-Fi credentials, student portals, email accounts, institutional servers, firewalls, and digital
    },
    "Library Services": {
        "slug": "library_services",
        "head": "Prof. Michael Chang",
        "email": "library@campus.edu",
        "location": "Central Library, Main Circulation Desk",
        "color": "#D97706",
        "bg_color": "#FFFBEB",
        "border_color": "#FCD34D",
        "technicians": [
            "Prof. Michael Chang (Chief Librarian)",
            "Sarah Lin (Digital Catalog Admin)",
            "Rajiv Verma (Circulation Officer)",
            "Emily Watson (Reading Room Supervisor)"
        ],
        "default_username": "dept_library",
        "description": "Manages book lending kiosks, RFID turnstiles, quiet reading halls, research databases, and library air conditioni
    },
    "Transport Office": {
        "slug": "transport_office",
        "head": "Capt. Arvind Singh",
        "email": "transport@campus.edu",
        "location": "Gate 2 Administration Complex",
        "color": "#EA580C",
        "bg_color": "#FFF7ED",
        "border_color": "#FDBA74",
        "technicians": [
            "Capt. Arvind Singh (Transit Director)",
            "Harpreet Singh (Fleet Supervisor)",
            "Suresh Rao (Route Coordinator)",
            "Vikram Seth (Mechanic In-Charge)"
        ],
        "default_username": "dept_transport",
        "description": "Administers campus shuttle buses, route timings, driver rosters, transit passes, and shuttle breakdown response."
    },
    "Academic Affairs": {
        "slug": "academic_affairs",
        "head": "Dean Dr. Eleanor Ross",
        "email": "academic-affairs@campus.edu",
        "location": "Academic Block A, Deanery Suite",
        "color": "#E11D48",
        "bg_color": "#FFF1F2",
        "border_color": "#FDA4AF",
        "technicians": [
            "Dean Dr. Eleanor Ross (Academic Dean)",
            "Rachel Green (Examination Coordinator)",
            "Marcus Bell (Registrar Assistant)",
            "Clara Oswald (Lecture Hall Allocator)"
        ],
        "default_username": "dept_academic",
        "description": "Manages lecture schedules, mid-term & practical examination schedules, faculty disputes, and academic appeals."
    }
}

# SLA Target Timelines
SLA_HOURS_MAP = {
    "Critical": 4,
    "High": 12,
    "Medium": 48,
    "Low": 96
}
```

## FILE: database.py

**Type:** Python | **Lines:** 708

*SQLite Database Schema, PBKDF2 Password Hashing, CRUD & Analytics*

Path: smart_campus_ai/database.py

```
0001 | import sqlite3
0002 | import hashlib
0003 | import os
0004 | import json
0005 | from datetime import datetime
0006 | from config import DB_PATH, DEPARTMENT_DETAILS, SLA_HOURS_MAP
0007 |
0008 | def get_connection():
0009 |     conn = sqlite3.connect(DB_PATH)
0010 |     conn.row_factory = sqlite3.Row
0011 |     return conn
0012 |
0013 | def hash_password(password: str, salt: str = None) -> str:
0014 |     if not salt:
0015 |         salt = os.urandom(16).hex()
0016 |     hashed = hashlib.pbkdf2_hmac(
0017 |         "sha256", password.encode("utf-8"), salt.encode("utf-8"), 100000
0018 |     ).hex()
0019 |     return f"{salt}${hashed}"
0020 |
0021 | def verify_password(password: str, stored_hash: str) -> bool:
0022 |     try:
0023 |         salt, hashed = stored_hash.split("$")
0024 |         check = hashlib.pbkdf2_hmac(
0025 |             "sha256", password.encode("utf-8"), salt.encode("utf-8"), 100000
0026 |         ).hex()
0027 |         return check == hashed
0028 |     except Exception:
0029 |         return False
0030 |
0031 | def init_db():
0032 |     conn = get_connection()
0033 |     cursor = conn.cursor()
0034 |
0035 |     # Users table
0036 |     cursor.execute("""
0037 |         CREATE TABLE IF NOT EXISTS users (
0038 |             id INTEGER PRIMARY KEY AUTOINCREMENT,
0039 |             username TEXT UNIQUE NOT NULL,
0040 |             password_hash TEXT NOT NULL,
```

```
            email TEXT UNIQUE NOT NULL,
            full_name TEXT NOT NULL,
            role TEXT NOT NULL,
            department TEXT DEFAULT 'General',
            phone TEXT DEFAULT '',
            created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
        )
    """)

    # Complaints table
    cursor.execute("""
        CREATE TABLE IF NOT EXISTS complaints (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            ticket_id TEXT UNIQUE NOT NULL,
            student_id INTEGER NOT NULL,
            student_name TEXT NOT NULL,
            student_email TEXT NOT NULL,
            title TEXT NOT NULL,
            description TEXT NOT NULL,
            category TEXT NOT NULL,
            priority TEXT NOT NULL,
            priority_reason TEXT DEFAULT '',
            department TEXT NOT NULL,
            status TEXT NOT NULL DEFAULT 'Open',
            attachment_path TEXT DEFAULT '',
            rag_context TEXT DEFAULT '',
            needs_review INTEGER DEFAULT 0,
            assigned_technician TEXT DEFAULT '',
            sla_hours INTEGER DEFAULT 24,
            student_rating INTEGER DEFAULT 0,
            student_feedback TEXT DEFAULT '',
            created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
            updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
            FOREIGN KEY(student_id) REFERENCES users(id)
        )
    """)

    # Check and add any missing columns in complaints table if it already existed
    cursor.execute("PRAGMA table_info(complaints)")
    existing_cols = [row["name"] for row in cursor.fetchall()]
```

```
    if "assigned_technician" not in existing_cols:
        cursor.execute("ALTER TABLE complaints ADD COLUMN assigned_technician TEXT DEFAULT ''")
    if "sla_hours" not in existing_cols:
        cursor.execute("ALTER TABLE complaints ADD COLUMN sla_hours INTEGER DEFAULT 24")
    if "student_rating" not in existing_cols:
        cursor.execute("ALTER TABLE complaints ADD COLUMN student_rating INTEGER DEFAULT 0")
    if "student_feedback" not in existing_cols:
        cursor.execute("ALTER TABLE complaints ADD COLUMN student_feedback TEXT DEFAULT ''")
    if "completed_by_dept" not in existing_cols:
        cursor.execute("ALTER TABLE complaints ADD COLUMN completed_by_dept INTEGER DEFAULT 0")
    if "replaced_parts" not in existing_cols:
        cursor.execute("ALTER TABLE complaints ADD COLUMN replaced_parts TEXT DEFAULT ''")

    # Complaint History / Audit Trail
    cursor.execute("""
        CREATE TABLE IF NOT EXISTS complaint_history (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            complaint_id INTEGER NOT NULL,
            ticket_id TEXT NOT NULL,
            previous_status TEXT NOT NULL,
            new_status TEXT NOT NULL,
            changed_by TEXT NOT NULL,
            notes TEXT DEFAULT '',
            timestamp TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
            FOREIGN KEY(complaint_id) REFERENCES complaints(id)
        )
    """)

    # Resolution Notes
    cursor.execute("""
        CREATE TABLE IF NOT EXISTS resolution_notes (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            complaint_id INTEGER NOT NULL,
            ticket_id TEXT NOT NULL,
            author_name TEXT NOT NULL,
            author_role TEXT NOT NULL,
            note_text TEXT NOT NULL,
            is_internal INTEGER DEFAULT 0,
            created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
            FOREIGN KEY(complaint_id) REFERENCES complaints(id)
```

```
        )
    """)

    # Notifications Table
    cursor.execute("""
        CREATE TABLE IF NOT EXISTS notifications (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            ticket_id TEXT NOT NULL,
            recipient_role TEXT NOT NULL,
            department TEXT NOT NULL,
            title TEXT NOT NULL,
            message TEXT NOT NULL,
            urgency TEXT NOT NULL DEFAULT 'Medium',
            is_read INTEGER DEFAULT 0,
            created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
        )
    """)

    conn.commit()

    # Seed Accounts: Ensure Central Admin, Default Student, and All 7 Department accounts exist
    admin_pass = hash_password("Admin@123")
    student_pass = hash_password("Student@123")

    # Central Administrator
    cursor.execute("SELECT id FROM users WHERE username = 'admin'")
    if not cursor.fetchone():
        cursor.execute("""
            INSERT INTO users (username, password_hash, email, full_name, role, department, phone)
            VALUES (?, ?, ?, ?, ?, ?, ?)
        """, ("admin", admin_pass, "admin@campus.edu", "Campus Administrator", "admin", "Administration", "+1-555-0100"))

    # Default Student
    cursor.execute("SELECT id FROM users WHERE username = 'student1'")
    if not cursor.fetchone():
        cursor.execute("""
            INSERT INTO users (username, password_hash, email, full_name, role, department, phone)
            VALUES (?, ?, ?, ?, ?, ?, ?)
        """, ("student1", student_pass, "alex.johnson@student.campus.edu", "Alex Johnson", "student", "Computer Science", "+1-555-0199"))

    # Seed Department Officers for each of the 7 departments
    dept_seeds = [
        ("dept_lab", "Lab@123", "lab-maintenance@campus.edu", "Robert Vance (Lab Maintenance)", "admin", "Lab Maintenance"),
        ("dept_hostel", "Hostel@123", "hostel-office@campus.edu", "Dr. Sarah Jenkins (Hostel Admin)", "admin", "Hostel Administration"),
        ("dept_estate", "Estate@123", "estate-facilities@campus.edu", "David Kumar (Estate & Facilities)", "admin", "Estate & Facilities"
        ("dept_it", "IT@123", "it-support@campus.edu", "Priya Patel (IT Support)", "admin", "IT Support"),
        ("dept_library", "Library@123", "library@campus.edu", "Prof. Michael Chang (Library Services)", "admin", "Library Services"),
        ("dept_transport", "Transport@123", "transport@campus.edu", "Capt. Arvind Singh (Transport Office)", "admin", "Transport Office")
        ("dept_academic", "Academic@123", "academic-affairs@campus.edu", "Dean Eleanor Ross (Academic Affairs)", "admin", "Academic Affai
    ]

    for u_name, u_pwd, u_email, u_full, u_role, u_dept in dept_seeds:
        cursor.execute("SELECT id FROM users WHERE username = ?", (u_name,))
        if not cursor.fetchone():
            cursor.execute("""
                INSERT INTO users (username, password_hash, email, full_name, role, department, phone)
                VALUES (?, ?, ?, ?, ?, ?, ?)
            """, (u_name, hash_password(u_pwd), u_email, u_full, u_role, u_dept, "+1-555-0150"))

    conn.commit()
    conn.close()

def create_user(username, password, email, full_name, role="student", department="General", phone=""):
    conn = get_connection()
    cursor = conn.cursor()
    pwd_hash = hash_password(password)
    try:
        cursor.execute("""
            INSERT INTO users (username, password_hash, email, full_name, role, department, phone)
            VALUES (?, ?, ?, ?, ?, ?, ?)
        """, (username, pwd_hash, email, full_name, role, department, phone))
        conn.commit()
        user_id = cursor.lastrowid
        return True, user_id, "User registered successfully."
    except sqlite3.IntegrityError as e:
        if "username" in str(e):
            return False, None, "Username already exists."
        elif "email" in str(e):
            return False, None, "Email already exists."
        return False, None, "Registration failed: user already exists."
```

```
    finally:
        conn.close()

def authenticate_user(username, password):
    conn = get_connection()
    cursor = conn.cursor()
    cursor.execute("SELECT * FROM users WHERE username = ? OR email = ?", (username, username))
    user = cursor.fetchone()
    conn.close()
    if user and verify_password(password, user["password_hash"]):
        return dict(user)
    return None

def get_user_by_id(user_id):
    conn = get_connection()
    cursor = conn.cursor()
    cursor.execute("SELECT id, username, email, full_name, role, department, phone, created_at FROM users WHERE id = ?", (user_id,))
    row = cursor.fetchone()
    conn.close()
    return dict(row) if row else None

def get_next_ticket_id():
    conn = get_connection()
    cursor = conn.cursor()
    cursor.execute("SELECT id FROM complaints ORDER BY id DESC LIMIT 1")
    row = cursor.fetchone()
    conn.close()
    next_num = 1 if not row else row["id"] + 1
    return f"CAMP-{next_num:04d}"

def insert_complaint(student_id, student_name, student_email, title, description,
                     category, priority, priority_reason, department,
                     attachment_path="", rag_context="", needs_review=0,
                     ticket_id=None, sla_hours=None):
    conn = get_connection()
    cursor = conn.cursor()
    if not ticket_id:
        cursor.execute("SELECT id FROM complaints ORDER BY id DESC LIMIT 1")
        row = cursor.fetchone()
        next_num = 1 if not row else row["id"] + 1
```

```
        ticket_id = f"CAMP-{next_num:04d}"

    if not sla_hours:
        sla_hours = SLA_HOURS_MAP.get(priority, 24)

    cursor.execute("""
        INSERT INTO complaints (
            ticket_id, student_id, student_name, student_email, title,
            description, category, priority, priority_reason, department,
            status, attachment_path, rag_context, needs_review, sla_hours,
            created_at, updated_at
        ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, 'Open', ?, ?, ?, ?, CURRENT_TIMESTAMP, CURRENT_TIMESTAMP)
    """, (
        ticket_id, student_id, student_name, student_email, title,
        description, category, priority, priority_reason, department,
        attachment_path, rag_context, needs_review, sla_hours
    ))
    complaint_id = cursor.lastrowid

    # Record initial creation in audit history
    cursor.execute("""
        INSERT INTO complaint_history (complaint_id, ticket_id, previous_status, new_status, changed_by, notes)
        VALUES (?, ?, 'None', 'Open', ?, 'Complaint submitted and ticket generated by AI System')
    """, (complaint_id, ticket_id, student_name))

    # AUTOMATIC IN-APP NOTIFICATIONS
    # 1. Notify the assigned department administrator
    cursor.execute("""
        INSERT INTO notifications (ticket_id, recipient_role, department, title, message, urgency)
        VALUES (?, 'dept_admin', ?, ?, ?, ?)
    """, (
        ticket_id,
        department,
        f"New {priority} Priority Complaint: {ticket_id}",
        f"Complaint '{title}' logged by {student_name} and routed to {department}. Target SLA: {sla_hours}h.",
        priority
    ))

    # 2. Notify Central Administration
    cursor.execute("""
```

```
        INSERT INTO notifications (ticket_id, recipient_role, department, title, message, urgency)
        VALUES (?, 'admin', 'Central', ?, ?, ?)
    """, (
        ticket_id,
        f"New Ticket Ingested: {ticket_id} ({department})",
        f"AI System assigned ticket {ticket_id} ({priority}) to {department}.",
        priority
    ))

    conn.commit()
    conn.close()
    return complaint_id, ticket_id

def get_complaint_by_ticket_id(ticket_id):
    conn = get_connection()
    cursor = conn.cursor()
    cursor.execute("SELECT * FROM complaints WHERE ticket_id = ?", (ticket_id,))
    row = cursor.fetchone()
    conn.close()
    return dict(row) if row else None

def get_complaints_by_student(student_id):
    conn = get_connection()
    cursor = conn.cursor()
    cursor.execute("SELECT * FROM complaints WHERE student_id = ? ORDER BY id DESC", (student_id,))
    rows = cursor.fetchall()
    conn.close()
    return [dict(r) for r in rows]

def get_complaints_by_department(dept_name, search=None, category=None, priority=None, status=None):
    conn = get_connection()
    cursor = conn.cursor()
    query = "SELECT * FROM complaints WHERE department = ?"
    params = [dept_name]

    if search:
        query += " AND (ticket_id LIKE ? OR title LIKE ? OR description LIKE ? OR student_name LIKE ?)"
        term = f"%{search}%"
        params.extend([term, term, term, term])
    if category and category != "All":
```

```
        query += " AND category = ?"
        params.append(category)
    if priority and priority != "All":
        query += " AND priority = ?"
        params.append(priority)
    if status and status != "All":
        query += " AND status = ?"
        params.append(status)

    query += " ORDER BY id DESC"
    cursor.execute(query, params)
    rows = cursor.fetchall()
    conn.close()
    return [dict(r) for r in rows]

def get_all_complaints(search=None, category=None, priority=None, status=None, department=None):
    conn = get_connection()
    cursor = conn.cursor()
    query = "SELECT * FROM complaints WHERE 1=1"
    params = []

    if search:
        query += " AND (ticket_id LIKE ? OR title LIKE ? OR description LIKE ? OR student_name LIKE ?)"
        term = f"%{search}%"
        params.extend([term, term, term, term])
    if category and category != "All":
        query += " AND category = ?"
        params.append(category)
    if priority and priority != "All":
        query += " AND priority = ?"
        params.append(priority)
    if status and status != "All":
        query += " AND status = ?"
        params.append(status)
    if department and department != "All":
        query += " AND department = ?"
        params.append(department)

    query += " ORDER BY id DESC"
    cursor.execute(query, params)
```

```
    rows = cursor.fetchall()
    conn.close()
    return [dict(r) for r in rows]

def update_complaint_status(ticket_id, new_status, changed_by, notes=""):
    conn = get_connection()
    cursor = conn.cursor()
    cursor.execute("SELECT id, status, department, student_name FROM complaints WHERE ticket_id = ?", (ticket_id,))
    complaint = cursor.fetchone()
    if not complaint:
        conn.close()
        return False, "Complaint ticket not found."

    old_status = complaint["status"]
    cursor.execute("""
        UPDATE complaints
        SET status = ?, updated_at = CURRENT_TIMESTAMP
        WHERE ticket_id = ?
    """, (new_status, ticket_id))

    cursor.execute("""
        INSERT INTO complaint_history (complaint_id, ticket_id, previous_status, new_status, changed_by, notes)
        VALUES (?, ?, ?, ?, ?, ?)
    """, (complaint["id"], ticket_id, old_status, new_status, changed_by, notes))

    if notes:
        cursor.execute("""
            INSERT INTO resolution_notes (complaint_id, ticket_id, author_name, author_role, note_text, is_internal)
            VALUES (?, ?, ?, 'Staff', ?, 0)
        """, (complaint["id"], ticket_id, changed_by, notes))

    # Add notification for status change
    cursor.execute("""
        INSERT INTO notifications (ticket_id, recipient_role, department, title, message, urgency)
        VALUES (?, 'student', ?, ?, ?, 'Medium')
    """, (
        ticket_id,
        complaint["department"],
        f"Ticket {ticket_id} Status: {new_status}",
        f"Status changed from {old_status} to {new_status} by {changed_by}."
```

```
    ))

    conn.commit()
    conn.close()
    return True, f"Status updated to '{new_status}' successfully."

def mark_complaint_completed(ticket_id, completed_by, notes="", replaced_parts=""):
    """
    Called when Department Portal marks a complaint as COMPLETED.
    Directly updates the ticket status to 'Resolved' so the Central Admin Portal
    and Student Portal immediately reflect it as RESOLVED.
    Dispatches immediate alert to Central Admin.
    """
    conn = get_connection()
    cursor = conn.cursor()
    cursor.execute("SELECT id, status, department, title, student_name FROM complaints WHERE ticket_id = ?", (ticket_id,))
    complaint = cursor.fetchone()
    if not complaint:
        conn.close()
        return False, "Complaint ticket not found."

    old_status = complaint["status"]
    dept = complaint["department"]

    # Mark status as Resolved and record completed_by_dept flag
    cursor.execute("""
        UPDATE complaints
        SET status = 'Resolved',
            completed_by_dept = 1,
            replaced_parts = ?,
            updated_at = CURRENT_TIMESTAMP
        WHERE ticket_id = ?
    """, (replaced_parts.strip(), ticket_id))

    audit_note = f"Work marked COMPLETED by {dept} ({completed_by}). Ticket officially marked as RESOLVED in Central Admin. Diagnostic No
    if replaced_parts:
        audit_note += f" Hardware / Parts Replaced: {replaced_parts}."

    cursor.execute("""
        INSERT INTO complaint_history (complaint_id, ticket_id, previous_status, new_status, changed_by, notes)
```

```
        VALUES (?, ?, ?, 'Resolved', ?, ?)
    """, (complaint["id"], ticket_id, old_status, completed_by, audit_note))

    # Add official technician resolution report
    note_content = f"Work marked COMPLETED by {completed_by} ({dept}). {notes}"
    if replaced_parts:
        note_content += f" | Parts / Equipment Swapped: {replaced_parts}"

    cursor.execute("""
        INSERT INTO resolution_notes (complaint_id, ticket_id, author_name, author_role, note_text, is_internal)
        VALUES (?, ?, ?, 'Department Specialist', ?, 0)
    """, (complaint["id"], ticket_id, completed_by, note_content))

    # Dispatch alert to Central Administration
    cursor.execute("""
        INSERT INTO notifications (ticket_id, recipient_role, department, title, message, urgency)
        VALUES (?, 'admin', 'Central', ?, ?, 'Low')
    """, (
        ticket_id,
        f"Ticket {ticket_id} Completed by {dept}",
        f"Department '{dept}' marked work COMPLETED for '{complaint['title']}'. The ticket is now marked as RESOLVED across Central Admin
    ))

    # Dispatch alert to Student
    cursor.execute("""
        INSERT INTO notifications (ticket_id, recipient_role, department, title, message, urgency)
        VALUES (?, 'student', ?, ?, ?, 'Low')
    """, (
        ticket_id,
        dept,
        f"Ticket {ticket_id} Resolved",
        f"{dept} has completed work on your complaint '{complaint['title']}'. Status is RESOLVED. Please submit your resolution satisfact
    ))

    conn.commit()
    conn.close()
    return True, f"Ticket {ticket_id} successfully marked as COMPLETED by {dept}. Central Admin and Student portals updated to RESOLVED."

def assign_technician_to_ticket(ticket_id, technician_name, assigned_by, notes=""):
    conn = get_connection()
```

```
    cursor = conn.cursor()
    cursor.execute("SELECT id, department FROM complaints WHERE ticket_id = ?", (ticket_id,))
    complaint = cursor.fetchone()
    if not complaint:
        conn.close()
        return False, "Complaint ticket not found."

    cursor.execute("""
        UPDATE complaints
        SET assigned_technician = ?, updated_at = CURRENT_TIMESTAMP
        WHERE ticket_id = ?
    """, (technician_name, ticket_id))

    change_desc = f"Assigned Technician: {technician_name}. {notes}".strip()
    cursor.execute("""
        INSERT INTO complaint_history (complaint_id, ticket_id, previous_status, new_status, changed_by, notes)
        VALUES (?, ?, 'Assigned', 'Assigned', ?, ?)
    """, (complaint["id"], ticket_id, assigned_by, change_desc))

    conn.commit()
    conn.close()
    return True, f"Assigned technician '{technician_name}' to ticket {ticket_id}."

def submit_student_feedback(ticket_id, rating, feedback_text=""):
    conn = get_connection()
    cursor = conn.cursor()
    cursor.execute("""
        UPDATE complaints
        SET student_rating = ?, student_feedback = ?, updated_at = CURRENT_TIMESTAMP
        WHERE ticket_id = ?
    """, (rating, feedback_text.strip(), ticket_id))
    conn.commit()
    conn.close()
    return True, "Feedback submitted successfully. Thank you!"

def update_complaint_assignment(ticket_id, new_department, new_priority, changed_by, notes=""):
    conn = get_connection()
    cursor = conn.cursor()
    cursor.execute("SELECT id, department, priority FROM complaints WHERE ticket_id = ?", (ticket_id,))
    complaint = cursor.fetchone()
```

```
    if not complaint:
        conn.close()
        return False, "Complaint ticket not found."

    sla = SLA_HOURS_MAP.get(new_priority, 24)
    cursor.execute("""
        UPDATE complaints
        SET department = ?, priority = ?, sla_hours = ?, needs_review = 0, updated_at = CURRENT_TIMESTAMP
        WHERE ticket_id = ?
    """, (new_department, new_priority, sla, ticket_id))

    change_desc = f"Updated assignment: Department -> {new_department}, Priority -> {new_priority}. {notes}".strip()
    cursor.execute("""
        INSERT INTO complaint_history (complaint_id, ticket_id, previous_status, new_status, changed_by, notes)
        VALUES (?, ?, 'Modified', 'Modified', ?, ?)
    """, (complaint["id"], ticket_id, changed_by, change_desc))

    # Trigger notification to the newly assigned department
    cursor.execute("""
        INSERT INTO notifications (ticket_id, recipient_role, department, title, message, urgency)
        VALUES (?, 'dept_admin', ?, ?, ?, ?)
    """, (
        ticket_id,
        new_department,
        f"Reassigned Ticket: {ticket_id} ({new_priority})",
        f"Ticket reassigned to {new_department} by {changed_by}.",
        new_priority
    ))

    conn.commit()
    conn.close()
    return True, "Complaint reassigned successfully."

def add_resolution_note(ticket_id, author_name, author_role, note_text, is_internal=0):
    conn = get_connection()
    cursor = conn.cursor()
    cursor.execute("SELECT id FROM complaints WHERE ticket_id = ?", (ticket_id,))
    complaint = cursor.fetchone()
    if not complaint:
        conn.close()
```

```
        return False, "Complaint ticket not found."

    cursor.execute("""
        INSERT INTO resolution_notes (complaint_id, ticket_id, author_name, author_role, note_text, is_internal)
        VALUES (?, ?, ?, ?, ?, ?)
    """, (complaint["id"], ticket_id, author_name, author_role, note_text, is_internal))

    conn.commit()
    conn.close()
    return True, "Resolution note added."

def get_complaint_history(ticket_id):
    conn = get_connection()
    cursor = conn.cursor()
    cursor.execute("SELECT * FROM complaint_history WHERE ticket_id = ? ORDER BY id ASC", (ticket_id,))
    rows = cursor.fetchall()
    conn.close()
    return [dict(r) for r in rows]

def get_resolution_notes(ticket_id, include_internal=False):
    conn = get_connection()
    cursor = conn.cursor()
    if include_internal:
        cursor.execute("SELECT * FROM resolution_notes WHERE ticket_id = ? ORDER BY id ASC", (ticket_id,))
    else:
        cursor.execute("SELECT * FROM resolution_notes WHERE ticket_id = ? AND is_internal = 0 ORDER BY id ASC", (ticket_id,))
    rows = cursor.fetchall()
    conn.close()
    return [dict(r) for r in rows]

# NOTIFICATION SYSTEM QUERIES
def get_notifications(department=None, role=None, unread_only=False):
    conn = get_connection()
    cursor = conn.cursor()
    query = "SELECT * FROM notifications WHERE 1=1"
    params = []

    if department:
        query += " AND (department = ? OR department = 'All')"
        params.append(department)
```

```
    if role:
        query += " AND (recipient_role = ? OR recipient_role = 'all')"
        params.append(role)
    if unread_only:
        query += " AND is_read = 0"

    query += " ORDER BY id DESC LIMIT 50"
    cursor.execute(query, params)
    rows = cursor.fetchall()
    conn.close()
    return [dict(r) for r in rows]

def mark_notification_as_read(notif_id):
    conn = get_connection()
    cursor = conn.cursor()
    cursor.execute("UPDATE notifications SET is_read = 1 WHERE id = ?", (notif_id,))
    conn.commit()
    conn.close()

def mark_all_notifications_read(department=None):
    conn = get_connection()
    cursor = conn.cursor()
    if department:
        cursor.execute("UPDATE notifications SET is_read = 1 WHERE department = ?", (department,))
    else:
        cursor.execute("UPDATE notifications SET is_read = 1")
    conn.commit()
    conn.close()

def get_analytics_summary():
    conn = get_connection()
    cursor = conn.cursor()

    cursor.execute("SELECT COUNT(*) FROM complaints")
    total = cursor.fetchone()[0]

    cursor.execute("SELECT COUNT(*) FROM complaints WHERE status = 'Open'")
    open_count = cursor.fetchone()[0]

    cursor.execute("SELECT COUNT(*) FROM complaints WHERE status = 'In Progress'")
```

```
    in_progress = cursor.fetchone()[0]

    cursor.execute("SELECT COUNT(*) FROM complaints WHERE status = 'Resolved'")
    resolved = cursor.fetchone()[0]

    cursor.execute("SELECT COUNT(*) FROM complaints WHERE priority IN ('High', 'Critical')")
    high_critical = cursor.fetchone()[0]

    cursor.execute("SELECT department, COUNT(*) as count FROM complaints GROUP BY department")
    dept_rows = cursor.fetchall()
    by_dept = {r["department"]: r["count"] for r in dept_rows}

    cursor.execute("SELECT category, COUNT(*) as count FROM complaints GROUP BY category")
    cat_rows = cursor.fetchall()
    by_cat = {r["category"]: r["count"] for r in cat_rows}

    cursor.execute("SELECT priority, COUNT(*) as count FROM complaints GROUP BY priority")
    prio_rows = cursor.fetchall()
    by_priority = {r["priority"]: r["count"] for r in prio_rows}

    conn.close()
    return {
        "total": total,
        "open": open_count,
        "in_progress": in_progress,
        "resolved": resolved,
        "high_critical": high_critical,
        "by_dept": by_dept,
        "by_cat": by_cat,
        "by_priority": by_priority,
    }

def get_department_analytics(dept_name):
    conn = get_connection()
    cursor = conn.cursor()

    cursor.execute("SELECT COUNT(*) FROM complaints WHERE department = ?", (dept_name,))
    total = cursor.fetchone()[0]

    cursor.execute("SELECT COUNT(*) FROM complaints WHERE department = ? AND status = 'Open'", (dept_name,))
```

```
0681 |     open_count = cursor.fetchone()[0]
0682 |
0683 |     cursor.execute("SELECT COUNT(*) FROM complaints WHERE department = ? AND status = 'In Progress'", (dept_name,))
0684 |     in_progress = cursor.fetchone()[0]
0685 |
0686 |     cursor.execute("SELECT COUNT(*) FROM complaints WHERE department = ? AND status = 'Resolved'", (dept_name,))
0687 |     resolved = cursor.fetchone()[0]
0688 |
0689 |     cursor.execute("SELECT COUNT(*) FROM complaints WHERE department = ? AND priority IN ('High', 'Critical')", (dept_name,))
0690 |     high_critical = cursor.fetchone()[0]
0691 |
0692 |     cursor.execute("SELECT priority, COUNT(*) as count FROM complaints WHERE department = ? GROUP BY priority", (dept_name,))
0693 |     prio_rows = cursor.fetchall()
0694 |     by_priority = {r["priority"]: r["count"] for r in prio_rows}
0695 |
0696 |     cursor.execute("SELECT AVG(student_rating) FROM complaints WHERE department = ? AND student_rating > 0", (dept_name,))
0697 |     avg_rating = cursor.fetchone()[0] or 0.0
0698 |
0699 |     conn.close()
0700 |     return {
0701 |         "total": total,
0702 |         "open": open_count,
0703 |         "in_progress": in_progress,
0704 |         "resolved": resolved,
0705 |         "high_critical": high_critical,
0706 |         "by_priority": by_priority,
0707 |         "avg_rating": round(float(avg_rating), 1)
0708 |     }
```

## FILE: test_system.py

**Type:** Python | **Lines:** 157

*Automated 9-Stage Multi-Agent System Verification Test Suite*

Path: smart_campus_ai/test_system.py

```
0001 | import sys
0002 | import os
0003 | from pathlib import Path
0004 |
0005 | # Ensure UTF-8 output on Windows consoles
0006 | if hasattr(sys.stdout, "reconfigure"):
0007 |     sys.stdout.reconfigure(encoding="utf-8")
0008 |
0009 | # Add project root to sys.path
0010 | sys.path.insert(0, str(Path(__file__).resolve().parent))
0011 |
0012 | import database as db
0013 | from agents.workflow import run_complaint_pipeline
0014 |
0015 | def test_full_system():
0016 |     print("================================================")
0017 |     print("SMART CAMPUS AI - END-TO-END VALIDATION TEST")
0018 |     print("================================================")
0019 |
0020 |     # 1. Database Init
0021 |     print("\n[Step 1] Initializing SQLite database...")
0022 |     db.init_db()
0023 |     print("[OK] Database initialized successfully.")
0024 |
0025 |     # 2. Authentication Checks for All Personas
0026 |     print("\n[Step 2] Testing User Authentication for All Personas...")
0027 |     admin = db.authenticate_user("admin", "Admin@123")
0028 |     assert admin is not None, "Admin authentication failed!"
0029 |     assert admin["role"] == "admin"
0030 |     print(f"[OK] Central Admin authenticated: {admin['full_name']} (Role: {admin['role']})")
0031 |
0032 |     student = db.authenticate_user("student1", "Student@123")
0033 |     assert student is not None, "Student authentication failed!"
0034 |     assert student["role"] == "student"
0035 |     print(f"[OK] Student authenticated: {student['full_name']} (Role: {student['role']})")
0036 |
0037 |     dept_officer = db.authenticate_user("dept_lab", "Lab@123")
0038 |     assert dept_officer is not None, "Department officer authentication failed!"
0039 |     print(f"[OK] Department Officer authenticated: {dept_officer['full_name']} (Dept: {dept_officer['department']})")
0040 |
```

```
0041 |     # 3. Test LangGraph 5-Agent Workflow with User's Demonstration Case
0042 |     print("\n[Step 3] Running 5-Agent LangGraph Workflow with Demonstration Case:")
0043 |     test_title = "CSE Laboratory Computers Down"
0044 |     test_desc = "The computers in the CSE laboratory are not working and we have a practical examination tomorrow."
0045 |     print(f"  Input: '{test_desc}'")
0046 |
0047 |     result = run_complaint_pipeline(
0048 |         student_id=student["id"],
0049 |         student_name=student["full_name"],
0050 |         student_email=student["email"],
0051 |         title=test_title,
0052 |         description=test_desc
0053 |     )
0054 |
0055 |     ticket_id = result["ticket_id"]
0056 |     category = result["category"]
0057 |     priority = result["priority"]
0058 |     dept = result["department"]
0059 |     reason = result["priority_reason"]
0060 |     status = result["status"]
0061 |     needs_rag = result["needs_rag"]
0062 |     retrieved_count = len(result.get("retrieved_docs", []))
0063 |
0064 |     print(f"[OK] Ticket Generated: {ticket_id}")
0065 |     print(f"[OK] Category: {category} (Expected: Laboratory)")
0066 |     print(f"[OK] Priority: {priority} (Expected: High)")
0067 |     print(f"[OK] Department: {dept} (Expected: Lab Maintenance)")
0068 |     print(f"[OK] Reason: {reason}")
0069 |     print(f"[OK] Status: {status} (Expected: Open)")
0070 |     print(f"[OK] Agentic RAG Triggered: {needs_rag} (Retrieved: {retrieved_count} docs)")
0071 |
0072 |     assert category == "Laboratory", f"Category mismatch: {category}"
0073 |     assert priority == "High", f"Priority mismatch: {priority}"
0074 |     assert dept == "Lab Maintenance", f"Department mismatch: {dept}"
0075 |     assert "practical examination" in reason.lower() or "examination" in reason.lower(), f"Reason unexpected: {reason}"
0076 |     assert status == "Open", f"Status mismatch: {status}"
0077 |     assert needs_rag is True, "Agentic RAG should have triggered for examination laboratory scenario"
0078 |
0079 |     # 4. Check In-App Department Notification Generation
0080 |     print("\n[Step 4] Verifying Department Alerts & Notifications...")
```

```
0081 |     lab_notifs = db.get_notifications(department="Lab Maintenance")
0082 |     assert len(lab_notifs) >= 1, "Department notification was not generated!"
0083 |     print(f"[OK] Lab Maintenance received {len(lab_notifs)} notification(s).")
0084 |     print(f"     Title: {lab_notifs[0]['title']}")
0085 |     print(f"     Message: {lab_notifs[0]['message']}")
0086 |
0087 |     # 5. Department Officer Assigning Technician
0088 |     print("\n[Step 5] Department Officer Assigning Field Technician...")
0089 |     ok, msg = db.assign_technician_to_ticket(
0090 |         ticket_id=ticket_id,
0091 |         technician_name="Robert Vance (Senior Systems Engineer)",
0092 |         assigned_by=dept_officer["full_name"],
0093 |         notes="Dispatched to CSE Lab 1 to diagnose workstation power supplies and switch."
0094 |     )
0095 |     assert ok, f"Failed to assign technician: {msg}"
0096 |     print(f"[OK] {msg}")
0097 |
0098 |     # 6. Status Transition: Open -> In Progress
0099 |     print("\n[Step 6] Updating Status to 'In Progress'...")
0100 |     ok, msg = db.update_complaint_status(
0101 |         ticket_id=ticket_id,
0102 |         new_status="In Progress",
0103 |         changed_by=dept_officer["full_name"],
0104 |         notes="Engineer on-site inspecting network switch switch-CSE-04 and power lines."
0105 |     )
0106 |     assert ok, f"Failed to update status: {msg}"
0107 |     print(f"[OK] {msg}")
0108 |
0109 |     # 7. Department Portal clicks 'Mark Work Completed' -> Admin Portal marked as RESOLVED
0110 |     print("\n[Step 7] Department Officer clicking 'Mark Work Completed'...")
0111 |     ok, msg = db.mark_complaint_completed(
0112 |         ticket_id=ticket_id,
0113 |         completed_by=dept_officer["full_name"],
0114 |         notes="Replaced faulty network switch switch-CSE-04 and rebooted 24 workstations. All systems verified and practical exam ready."
0115 |         replaced_parts="24-Port Gigabit Switch (Model GS-24), 4x Cat6 RJ45 Patch Cables"
0116 |     )
0117 |     assert ok, f"Failed to mark complaint completed: {msg}"
0118 |     print(f"[OK] {msg}")
0119 |
0120 |     # Verify Central Admin Portal sees it as RESOLVED
```

```
    print("\n[Step 7b] Verifying Central Admin sees status as RESOLVED...")
    admin_view_complaint = db.get_complaint_by_ticket_id(ticket_id)
    assert admin_view_complaint["status"] == "Resolved", f"Status in Central Admin is not Resolved! Got: {admin_view_complaint['status']}
    assert admin_view_complaint["completed_by_dept"] == 1, "completed_by_dept flag was not set to 1"
    assert "GS-24" in admin_view_complaint["replaced_parts"], "Replaced parts not recorded"
    print(f"[OK] Central Admin confirms ticket {ticket_id} status is: {admin_view_complaint['status']} [Completed by Dept]")

    # Check notification sent to Admin Portal
    admin_notifs = db.get_notifications(role="admin")
    completed_notif = [n for n in admin_notifs if f"Completed by {dept}" in n["title"] or f"Ticket {ticket_id}" in n["title"]]
    assert len(completed_notif) >= 1, "Admin alert for completed ticket was not dispatched!"
    print(f"[OK] Admin Portal received completion notification: '{completed_notif[0]['title']}'")

    # 8. Student Rating and Feedback Submission
    print("\n[Step 8] Student submitting 5-star rating and feedback...")
    ok, msg = db.submit_student_feedback(
        ticket_id=ticket_id,
        rating=5,
        feedback_text="Extremely fast response before our exam! All machines are operational."
    )
    assert ok, f"Failed to submit rating: {msg}"
    print(f"[OK] {msg}")

    # 9. Verify Department Analytics
    print("\n[Step 9] Verifying Department Analytics...")
    dept_stats = db.get_department_analytics("Lab Maintenance")
    print(f"[OK] Lab Maintenance Stats: Total={dept_stats['total']}, Resolved={dept_stats['resolved']}, Rating={dept_stats['avg_rating']}
    assert dept_stats["total"] >= 1
    assert dept_stats["resolved"] >= 1
    assert dept_stats["avg_rating"] == 5.0

    print("\n==================================================")
    print("ALL TESTS PASSED SUCCESSFULLY! NO EMOJIS, 100% FUNCTIONAL")
    print("==================================================")

if __name__ == "__main__":
    test_full_system()
```

## FILE: requirements.txt

**Type:** Config | **Lines:** 11

*Python Runtime Dependencies & Package Versions*

Path: smart_campus_ai/requirements.txt

```
0001 | streamlit>=1.35.0
0002 | langchain>=0.2.0
0003 | langgraph>=0.1.0
0004 | langchain-openai>=0.1.0
0005 | langchain-community>=0.2.0
0006 | langchain-core>=0.2.0
0007 | chromadb>=0.5.0
0008 | pandas>=2.0.0
0009 | pydantic>=2.0.0
0010 | python-dotenv>=1.0.0
0011 | altair>=5.0.0
```

## FILE: .env.example

**Type:** Config | **Lines:** 6

*Environment Variables Configuration Template*

Path: smart_campus_ai/.env.example

```
0001 | # OpenAI API Key (Optional: system automatically falls back to intelligent built-in rule & NLP engine if omitted)
0002 | OPENAI_API_KEY=your_openai_api_key_here
0003 |
0004 | # OpenAI Model configuration
0005 | OPENAI_MODEL=gpt-4o-mini
0006 | EMBEDDING_MODEL=text-embedding-3-small
```

## FILE: .gitignore

**Type:** Config | **Lines:** 36

*Git Exclusion Rules for Repository Cleanliness*

Path: smart_campus_ai/.gitignore

```
0001 | # Byte-compiled / optimized / DLL files
0002 | __pycache__/
0003 | *.py[cod]
0004 | *$py.class
0005 |
0006 | # Virtual Environments
0007 | venv/
0008 | env/
0009 | ENV/
0010 | .venv/
0011 |
0012 | # Environment Variables & Secrets
0013 | .env
0014 |
0015 | # SQLite Database & ChromaDB vector database
0016 | *.db
0017 | *.sqlite3
0018 | chroma_db/
0019 |
0020 | # Uploaded User Attachments (keep directory with .gitkeep)
0021 | uploads/*
0022 | !uploads/.gitkeep
0023 |
0024 | # IDEs & Editors
0025 | .vscode/
0026 | .idea/
0027 | *.swp
0028 | *.swo
0029 |
0030 | # OS Generated Files
0031 | .DS_Store
0032 | Thumbs.db
0033 |
0034 | # Generated CSV Exports
0035 | all_campus_complaints.csv
0036 | *_tickets.csv
```

# MODULE 2: LANGGRAPH 5-AGENT MULTI-AGENT SYSTEM

**Overview:** Autonomous cooperative agents orchestrating complaint understanding, taxonomy, priority scoring, department routing, and ticket generation.

## FILE: agents/state.py

**Type:** Python | **Lines:** 45

*TypedDict ComplaintState Schema for LangGraph Workflow*

Path: smart_campus_ai/agents/state.py

```
0001 | from typing import TypedDict, List, Dict, Any, Optional
0002 |
0003 | class ComplaintState(TypedDict):
0004 |     # Student and Input Details
0005 |     student_id: int
0006 |     student_name: str
0007 |     student_email: str
0008 |     title: str
0009 |     raw_text: str
0010 |     attachment_path: Optional[str]
0011 |
0012 |     # Agent 1: Complaint Understanding
0013 |     key_issue: str
0014 |     impacted_asset: str
0015 |     urgency_context: str
0016 |     understanding_summary: str
0017 |
0018 |     # Agent 2: Classification
0019 |     category: str
0020 |     category_confidence: float
0021 |
0022 |     # Agent 3: Priority Detection
0023 |     priority: str
0024 |     priority_reason: str
0025 |     sla_hours: int
0026 |
0027 |     # Agentic RAG Node
0028 |     needs_rag: bool
0029 |     rag_query: str
0030 |     retrieved_docs: List[Dict[str, Any]]
0031 |     rag_context_summary: str
0032 |
0033 |     # Agent 4: Department Routing
0034 |     department: str
0035 |     routing_confidence: float
0036 |     needs_review: bool
0037 |     routing_reason: str
0038 |
0039 |     # Agent 5: Ticket & Tracking
0040 |     ticket_id: str
```

```
0041 |     status: str
0042 |     created_at: str
0043 |
0044 |     # Workflow tracing
0045 |     logs: List[str]
```

## FILE: agents/understanding_agent.py

**Type:** Python | **Lines:** 87

*Agent 1: Natural Language Complaint Understanding & Intent Extraction*

Path: smart_campus_ai/agents/understanding_agent.py

```
0001 | import json
0002 | from config import OPENAI_API_KEY, OPENAI_MODEL
0003 | from agents.state import ComplaintState
0004 |
0005 | def complaint_understanding_agent(state: ComplaintState) -> ComplaintState:
0006 |     """
0007 |     Agent 1 – Complaint Understanding Agent
0008 |     Understands natural-language student complaints, extracts key problems,
0009 |     impacted assets, and temporal/urgency signals.
0010 |     """
0011 |     title = state.get("title", "")
0012 |     raw_text = state.get("raw_text", "")
0013 |     full_input = f"Title: {title}\nDescription: {raw_text}".strip()
0014 |
0015 |     key_issue = ""
0016 |     impacted_asset = ""
0017 |     urgency_context = ""
0018 |     summary = ""
0019 |
0020 |     # Attempt LLM processing if API key is present
0021 |     if OPENAI_API_KEY:
0022 |         try:
0023 |             from langchain_openai import ChatOpenAI
0024 |             from langchain_core.messages import SystemMessage, HumanMessage
0025 |
0026 |             llm = ChatOpenAI(model=OPENAI_MODEL, temperature=0.0, api_key=OPENAI_API_KEY)
0027 |             prompt = (
0028 |                 "You are Agent 1: Campus Complaint Understanding Agent.\n"
0029 |                 "Extract the following in strict JSON format with keys:\n"
0030 |                 "- 'key_issue': Concise 1-sentence statement of the core problem.\n"
0031 |                 "- 'impacted_asset': The physical room, equipment, lab, or campus service impacted.\n"
0032 |                 "- 'urgency_context': Any upcoming deadlines, examinations, hazards, or immediate time constraints.\n"
0033 |                 "- 'summary': Brief summary of student grievance.\n\n"
0034 |                 f"Complaint:\n{full_input}"
0035 |             )
0036 |             response = llm.invoke([HumanMessage(content=prompt)])
0037 |             content = response.content.strip()
0038 |             if "```json" in content:
0039 |                 content = content.split("```json")[1].split("```")[0].strip()
0040 |             elif "```" in content:
```

```
                content = content.split("```")[1].split("```")[0].strip()

            parsed = json.loads(content)
            key_issue = parsed.get("key_issue", "")
            impacted_asset = parsed.get("impacted_asset", "")
            urgency_context = parsed.get("urgency_context", "")
            summary = parsed.get("summary", "")
        except Exception:
            pass

    # Resilient Rule-Based NLP Fallback
    if not key_issue:
        text_lower = full_input.lower()

        # Check impacted asset
        if "cse lab" in text_lower or "cse laboratory" in text_lower:
            impacted_asset = "CSE Laboratory Computers"
        elif "lab" in text_lower or "laboratory" in text_lower:
            impacted_asset = "Campus Laboratory"
        elif "hostel" in text_lower or "room" in text_lower or "dorm" in text_lower:
            impacted_asset = "Hostel Living Facility"
        elif "class" in text_lower or "projector" in text_lower:
            impacted_asset = "Classroom Facility"
        elif "bus" in text_lower or "transport" in text_lower:
            impacted_asset = "Campus Transport Service"
        elif "library" in text_lower:
            impacted_asset = "Library Resources"
        else:
            impacted_asset = "General Campus Infrastructure"

        # Check urgency
        if "tomorrow" in text_lower or "today" in text_lower or "exam" in text_lower or "practical" in text_lower:
            urgency_context = "Practical examination or academic assessment scheduled tomorrow"
        elif "urgent" in text_lower or "spark" in text_lower or "leak" in text_lower:
            urgency_context = "Immediate safety or functional disruption"
        else:
            urgency_context = "Standard operational resolution timeline"

        key_issue = f"{impacted_asset} malfunction reported: {title or raw_text[:60]}"
        summary = f"Student reports issue concerning {impacted_asset.lower()} with urgency: {urgency_context}."

    state["key_issue"] = key_issue
    state["impacted_asset"] = impacted_asset
    state["urgency_context"] = urgency_context
    state["understanding_summary"] = summary
    state["logs"].append(f"[Agent 1 - Understanding]: Identified issue with '{impacted_asset}', urgency: '{urgency_context}'.")
    return state
```

## FILE: agents/classification_agent.py

**Type:** Python | **Lines:** 75

*Agent 2: Academic & Facility Taxonomy Classification*

Path: smart_campus_ai/agents/classification_agent.py

```
0001 | import json
0002 | from config import OPENAI_API_KEY, OPENAI_MODEL, CATEGORIES
0003 | from agents.state import ComplaintState
0004 |
0005 | def complaint_classification_agent(state: ComplaintState) -> ComplaintState:
0006 |     """
0007 |     Agent 2 – Complaint Classification Agent
0008 |     Classifies complaints into: Laboratory, Hostel, Classroom, Library, Transport, Infrastructure, Other.
0009 |     """
0010 |     title = state.get("title", "")
0011 |     raw_text = state.get("raw_text", "")
0012 |     key_issue = state.get("key_issue", "")
0013 |     impacted_asset = state.get("impacted_asset", "")
0014 |     full_text = f"{title} {raw_text} {key_issue} {impacted_asset}".strip()
0015 |
0016 |     category = ""
0017 |     confidence = 0.85
0018 |
0019 |     # Attempt LLM Classification
0020 |     if OPENAI_API_KEY:
0021 |         try:
0022 |             from langchain_openai import ChatOpenAI
0023 |             from langchain_core.messages import HumanMessage
0024 |
0025 |             llm = ChatOpenAI(model=OPENAI_MODEL, temperature=0.0, api_key=OPENAI_API_KEY)
0026 |             prompt = (
0027 |                 f"You are Agent 2: Campus Complaint Classification Agent.\n"
0028 |                 f"Classify the following complaint into EXACTLY one of these categories: {CATEGORIES}.\n"
0029 |                 f"Respond in strict JSON format: {{\"category\": \"<Category>\", \"confidence\": <float between 0 and 1>}}\n\n"
0030 |                 f"Complaint:\n{full_text}"
0031 |             )
0032 |             response = llm.invoke([HumanMessage(content=prompt)])
0033 |             content = response.content.strip()
0034 |             if "```json" in content:
0035 |                 content = content.split("```json")[1].split("```")[0].strip()
0036 |             elif "```" in content:
0037 |                 content = content.split("```")[1].split("```")[0].strip()
0038 |
0039 |             parsed = json.loads(content)
0040 |             cat = parsed.get("category", "")
```

```
0041 |             if cat in CATEGORIES:
0042 |                 category = cat
0043 |                 confidence = float(parsed.get("confidence", 0.90))
0044 |         except Exception:
0045 |             pass
0046 |
0047 |     # Resilient Rule-Based Classification Fallback
0048 |     if not category:
0049 |         text_lower = full_text.lower()
0050 |         if any(w in text_lower for w in ["lab", "laboratory", "computer", "pc", "monitor", "software", "hardware", "workstation", "cse la
0051 |             category = "Laboratory"
0052 |             confidence = 0.98
0053 |         elif any(w in text_lower for w in ["hostel", "room", "mess", "warden", "geyser", "bed", "dormitory", "corridor", "hot water"]):
0054 |             category = "Hostel"
0055 |             confidence = 0.95
0056 |         elif any(w in text_lower for w in ["classroom", "projector", "smartboard", "blackboard", "podium", "lecture hall", "desk", "bench
0057 |             category = "Classroom"
0058 |             confidence = 0.94
0059 |         elif any(w in text_lower for w in ["library", "book", "reading room", "journal", "rfid", "turnstile"]):
0060 |             category = "Library"
0061 |             confidence = 0.96
0062 |         elif any(w in text_lower for w in ["bus", "transport", "shuttle", "driver", "route", "van", "commute"]):
0063 |             category = "Transport"
0064 |             confidence = 0.95
0065 |         elif any(w in text_lower for w in ["road", "pavement", "streetlight", "elevator", "lift", "pipe", "drainage", "water cooler", "bu
0066 |             category = "Infrastructure"
0067 |             confidence = 0.92
0068 |         else:
0069 |             category = "Other"
0070 |             confidence = 0.70
0071 |
0072 |     state["category"] = category
0073 |     state["category_confidence"] = confidence
0074 |     state["logs"].append(f"[Agent 2 - Classification]: Assigned category '{category}' with confidence {confidence:.2f}.")
0075 |     return state
```

## FILE: agents/priority_agent.py

**Type:** Python | **Lines:** 84

*Agent 3: Multi-Factor Priority Detection (Low, Med, High, Critical)*

Path: smart_campus_ai/agents/priority_agent.py

```
0001 | import json
0002 | from config import OPENAI_API_KEY, OPENAI_MODEL, PRIORITY_LEVELS
0003 | from agents.state import ComplaintState
0004 |
0005 | def priority_detection_agent(state: ComplaintState) -> ComplaintState:
0006 |     """
0007 |     Agent 3 – Priority Detection Agent
0008 |     Determines Low, Medium, High, or Critical priority based on urgency, impact, and safety.
0009 |     Provides a clear justification.
0010 |     """
0011 |     raw_text = state.get("raw_text", "")
0012 |     title = state.get("title", "")
0013 |     urgency_context = state.get("urgency_context", "")
0014 |     category = state.get("category", "")
0015 |     full_text = f"{title} {raw_text} {urgency_context}".strip()
0016 |
0017 |     priority = ""
0018 |     reason = ""
0019 |     sla_hours = 24
0020 |
0021 |     # Attempt LLM Priority Detection
0022 |     if OPENAI_API_KEY:
0023 |         try:
0024 |             from langchain_openai import ChatOpenAI
0025 |             from langchain_core.messages import HumanMessage
0026 |
0027 |             llm = ChatOpenAI(model=OPENAI_MODEL, temperature=0.0, api_key=OPENAI_API_KEY)
0028 |             prompt = (
0029 |                 f"You are Agent 3: Campus Priority Detection Agent.\n"
0030 |                 f"Assess the urgency, academic impact, and safety hazard of this complaint.\n"
0031 |                 f"Assign one of these priorities: {PRIORITY_LEVELS}.\n"
0032 |                 f"- Critical: Safety hazards (electrical fires, sparks, main water line burst).\n"
0033 |                 f"- High: Direct impending academic disruption (e.g., practical examination tomorrow with unavailable computers, urgent t
0034 |                 f"- Medium: Disruption of comfort or ongoing classes without immediate exam emergency.\n"
0035 |                 f"- Low: Minor cosmetic, non-urgent issues.\n"
0036 |                 f"Respond in strict JSON format:\n"
0037 |                 f"{{\"priority\": \"<Priority>\", \"reason\": \"<Concise explanation>\", \"sla_hours\": <integer hours>}}\n\n"
0038 |                 f"Complaint Details:\n{full_text}"
0039 |             )
0040 |             response = llm.invoke([HumanMessage(content=prompt)])
```

```
            content = response.content.strip()
            if "```json" in content:
                content = content.split("```json")[1].split("```")[0].strip()
            elif "```" in content:
                content = content.split("```")[1].split("```")[0].strip()

            parsed = json.loads(content)
            p = parsed.get("priority", "")
            if p in PRIORITY_LEVELS:
                priority = p
                reason = parsed.get("reason", "")
                sla_hours = int(parsed.get("sla_hours", 24))
        except Exception:
            pass

    # Resilient Rule-Based Priority Fallback
    if not priority:
        text_lower = full_text.lower()
        if any(w in text_lower for w in ["fire", "spark", "electric shock", "smoke", "live wire", "flood", "pipe burst"]):
            priority = "Critical"
            reason = "Imminent safety or infrastructure hazard requires immediate emergency isolation."
            sla_hours = 4
        elif ("exam" in text_lower or "examination" in text_lower or "practical" in text_lower) and ("tomorrow" in text_lower or "today"
            priority = "High"
            reason = "Practical examination is tomorrow and laboratory computers are unavailable."
            sla_hours = 12
        elif any(w in text_lower for w in ["urgent", "exam", "test", "deadline", "tomorrow"]):
            priority = "High"
            reason = "Time-sensitive academic commitment directly affected."
            sla_hours = 24
        elif any(w in text_lower for w in ["not working", "broken", "malfunction", "damaged", "repair", "ac", "projector"]):
            priority = "Medium"
            reason = "Operational facility disruption affecting routine campus activities."
            sla_hours = 48
        else:
            priority = "Low"
            reason = "General administrative request or non-critical maintenance inquiry."
            sla_hours = 96

    state["priority"] = priority
```

```
    state["priority_reason"] = reason
    state["sla_hours"] = sla_hours
    state["logs"].append(f"[Agent 3 - Priority]: Assigned '{priority}' priority. Reason: {reason}")
    return state
```

## FILE: agents/routing_agent.py

**Type:** Python | **Lines:** 105

*Agent 4: Autonomous Department Routing & Ambiguity Detection*

Path: smart_campus_ai/agents/routing_agent.py

```
0001 | import json
0002 | from config import OPENAI_API_KEY, OPENAI_MODEL, DEPARTMENTS
0003 | from agents.state import ComplaintState
0004 |
0005 | def department_routing_agent(state: ComplaintState) -> ComplaintState:
0006 |     """
0007 |     Agent 4 – Department Routing Agent
0008 |     Identifies the responsible campus department, routes complaints automatically,
0009 |     and flags uncertain assignments for administrator review.
0010 |     """
0011 |     category = state.get("category", "")
0012 |     key_issue = state.get("key_issue", "")
0013 |     raw_text = state.get("raw_text", "")
0014 |     rag_context = state.get("rag_context_summary", "")
0015 |
0016 |     full_text = f"Category: {category}\nIssue: {key_issue}\nText: {raw_text}\nContext: {rag_context}".strip()
0017 |
0018 |     department = ""
0019 |     confidence = 0.85
0020 |     needs_review = False
0021 |     reason = ""
0022 |
0023 |     # Attempt LLM Routing
0024 |     if OPENAI_API_KEY:
0025 |         try:
0026 |             from langchain_openai import ChatOpenAI
0027 |             from langchain_core.messages import HumanMessage
0028 |
0029 |             llm = ChatOpenAI(model=OPENAI_MODEL, temperature=0.0, api_key=OPENAI_API_KEY)
0030 |             prompt = (
0031 |                 f"You are Agent 4: Campus Department Routing Agent.\n"
0032 |                 f"Identify the single most appropriate department from: {DEPARTMENTS}.\n"
0033 |                 f"If the assignment is uncertain or ambiguous, set 'needs_review' to true.\n"
0034 |                 f"Respond in strict JSON format:\n"
0035 |                 f"{{\"department\": \"<Department>\", \"confidence\": <float 0.0-1.0>, \"needs_review\": <boolean>, \"reason\": \"<Justif
0036 |                 f"Complaint Information:\n{full_text}"
0037 |             )
0038 |             response = llm.invoke([HumanMessage(content=prompt)])
0039 |             content = response.content.strip()
0040 |             if "```json" in content:
```

```
                content = content.split("```json")[1].split("```")[0].strip()
            elif "```" in content:
                content = content.split("```")[1].split("```")[0].strip()

            parsed = json.loads(content)
            dept = parsed.get("department", "")
            if dept in DEPARTMENTS:
                department = dept
                confidence = float(parsed.get("confidence", 0.90))
                needs_review = bool(parsed.get("needs_review", False))
                reason = parsed.get("reason", "")
        except Exception:
            pass

    # Resilient Rule-Based Department Routing Fallback
    if not department:
        text_lower = full_text.lower()
        if category == "Laboratory" or any(w in text_lower for w in ["lab", "computer", "pc", "cse lab", "ece lab", "workstation", "pract
            department = "Lab Maintenance"
            confidence = 0.98
            needs_review = False
            reason = "Computing laboratory hardware and practical exam readiness are managed by Lab Maintenance."
        elif category == "Hostel" or any(w in text_lower for w in ["hostel", "warden", "mess", "dormitory", "room geyser"]):
            department = "Hostel Administration"
            confidence = 0.96
            needs_review = False
            reason = "Hostel facilities and residential discipline are governed by Hostel Administration."
        elif category == "Classroom" or any(w in text_lower for w in ["classroom", "projector", "smartboard", "blackboard"]):
            department = "Estate & Facilities"
            confidence = 0.92
            needs_review = False
            reason = "Classroom fixtures and physical AV maintenance are handled by Estate & Facilities."
        elif category == "Library" or any(w in text_lower for w in ["library", "book", "reading room", "rfid"]):
            department = "Library Services"
            confidence = 0.95
            needs_review = False
            reason = "Central library facilities and access turnstiles are managed by Library Services."
        elif category == "Transport" or any(w in text_lower for w in ["bus", "transport", "shuttle", "commute"]):
            department = "Transport Office"
            confidence = 0.95
            needs_review = False
            reason = "Campus transit and bus routes are managed by the Transport Office."
        elif any(w in text_lower for w in ["wifi", "internet", "portal", "email", "server", "login"]):
            department = "IT Support"
            confidence = 0.93
            needs_review = False
            reason = "Digital accounts, campus Wi-Fi, and software access are supported by Central IT."
        elif category == "Infrastructure" or any(w in text_lower for w in ["road", "street", "power", "water pipe", "drain", "civil"]):
            department = "Estate & Facilities"
            confidence = 0.90
            needs_review = False
            reason = "Campus civil infrastructure and utility lines are routed to Estate & Facilities."
        else:
            department = "Unassigned / Review Needed"
            confidence = 0.50
            needs_review = True
            reason = "General or unclassified complaint; routed to Administrator for manual departmental assignment."

    state["department"] = department
    state["routing_confidence"] = confidence
    state["needs_review"] = needs_review
    state["routing_reason"] = reason
    flag_text = " (Flagged for Admin Review)" if needs_review else ""
    state["logs"].append(f"[Agent 4 - Department Routing]: Routed to '{department}' (Confidence: {confidence:.2f}){flag_text}.")
    return state
```

## FILE: agents/ticket_agent.py

**Type:** Python | **Lines:** 40

*Agent 5: Ticket Creation, Sequential ID Generator & Persistent Alert Dispatch*

Path: smart_campus_ai/agents/ticket_agent.py

```
0001 | from datetime import datetime
0002 | from agents.state import ComplaintState
0003 | import database as db
0004 |
0005 | def ticket_and_tracking_agent(state: ComplaintState) -> ComplaintState:
0006 |     """
0007 |     Agent 5 – Ticket and Resolution Tracking Agent
0008 |     Generates unique ticket ID, commits complaint to SQLite database,
0009 |     initializes status to 'Open', begins audit trail, and issues departmental notifications.
0010 |     """
0011 |     # Generate unique ticket ID if not already generated
0012 |     ticket_id = state.get("ticket_id")
0013 |     if not ticket_id:
0014 |         ticket_id = db.get_next_ticket_id()
0015 |
0016 |     sla_hours = state.get("sla_hours", 24)
0017 |
0018 |     # Insert into database
0019 |     complaint_id, final_ticket_id = db.insert_complaint(
0020 |         student_id=state.get("student_id", 1),
0021 |         student_name=state.get("student_name", "Student"),
0022 |         student_email=state.get("student_email", "student@campus.edu"),
0023 |         title=state.get("title", "Untitled Complaint"),
0024 |         description=state.get("raw_text", ""),
0025 |         category=state.get("category", "Other"),
0026 |         priority=state.get("priority", "Medium"),
0027 |         priority_reason=state.get("priority_reason", ""),
0028 |         department=state.get("department", "Unassigned / Review Needed"),
0029 |         attachment_path=state.get("attachment_path", "") or "",
0030 |         rag_context=state.get("rag_context_summary", "") or "",
0031 |         needs_review=1 if state.get("needs_review", False) else 0,
0032 |         ticket_id=ticket_id,
0033 |         sla_hours=sla_hours
0034 |     )
0035 |
0036 |     state["ticket_id"] = final_ticket_id
0037 |     state["status"] = "Open"
0038 |     state["created_at"] = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
0039 |     state["logs"].append(f"[Agent 5 - Ticket Tracking]: Created ticket #{final_ticket_id} with initial status 'Open' in database. Notific
0040 |     return state
```

## FILE: agents/workflow.py

**Type:** Python | **Lines:** 126

*LangGraph StateGraph Coordinator with Agentic RAG Dynamic Conditional Edges*

Path: smart_campus_ai/agents/workflow.py

```
0001 | from typing import Literal
0002 | from langgraph.graph import StateGraph, START, END
0003 |
0004 | from agents.state import ComplaintState
0005 | from agents.understanding_agent import complaint_understanding_agent
0006 | from agents.classification_agent import complaint_classification_agent
0007 | from agents.priority_agent import priority_detection_agent
0008 | from agents.routing_agent import department_routing_agent
0009 | from agents.ticket_agent import ticket_and_tracking_agent
0010 | from rag.rag_engine import evaluate_rag_necessity, execute_agentic_rag
0011 |
0012 | def understanding_node(state: ComplaintState) -> ComplaintState:
0013 |     return complaint_understanding_agent(state)
0014 |
0015 | def classification_node(state: ComplaintState) -> ComplaintState:
0016 |     return complaint_classification_agent(state)
0017 |
0018 | def priority_node(state: ComplaintState) -> ComplaintState:
0019 |     return priority_detection_agent(state)
0020 |
0021 | def rag_decision_node(state: ComplaintState) -> ComplaintState:
0022 |     """Agentic RAG decision node: decides if campus knowledge retrieval is necessary."""
0023 |     needs_rag, reason, query = evaluate_rag_necessity(
0024 |         key_issue=state.get("key_issue", ""),
0025 |         category=state.get("category", ""),
0026 |         raw_text=state.get("raw_text", "")
0027 |     )
0028 |     state["needs_rag"] = needs_rag
0029 |     state["rag_query"] = query
0030 |     if needs_rag:
0031 |         state["logs"].append(f"[Agentic RAG Router]: Decided campus document retrieval is REQUIRED. Reason: {reason}. Query: '{query}'.")
0032 |     else:
0033 |         state["logs"].append("[Agentic RAG Router]: Decided campus document retrieval is NOT required for this complaint.")
0034 |     return state
0035 |
0036 | def rag_retrieval_node(state: ComplaintState) -> ComplaintState:
0037 |     """Retrieves relevant knowledge base documents and attaches them to state."""
0038 |     query = state.get("rag_query") or state.get("key_issue", "")
0039 |     rag_result = execute_agentic_rag(query, top_k=2)
0040 |     state["retrieved_docs"] = rag_result.get("retrieved_docs", [])
```

```
    state["rag_context_summary"] = rag_result.get("summary", "")
    doc_titles = [d.get("title", "Doc") for d in state["retrieved_docs"]]
    state["logs"].append(f"[Agentic RAG Tool]: Retrieved {len(doc_titles)} documents ({', '.join(doc_titles)}).")
    return state

def routing_node(state: ComplaintState) -> ComplaintState:
    return department_routing_agent(state)

def ticket_node(state: ComplaintState) -> ComplaintState:
    return ticket_and_tracking_agent(state)

def rag_condition(state: ComplaintState) -> Literal["retrieve_knowledge", "route_department"]:
    if state.get("needs_rag", False):
        return "retrieve_knowledge"
    return "route_department"

def build_complaint_workflow():
    workflow = StateGraph(ComplaintState)

    # Register Nodes
    workflow.add_node("understand_complaint", understanding_node)
    workflow.add_node("classify_complaint", classification_node)
    workflow.add_node("detect_priority", priority_node)
    workflow.add_node("rag_decision", rag_decision_node)
    workflow.add_node("retrieve_knowledge", rag_retrieval_node)
    workflow.add_node("route_department", routing_node)
    workflow.add_node("create_ticket", ticket_node)

    # Register Edges
    workflow.add_edge(START, "understand_complaint")
    workflow.add_edge("understand_complaint", "classify_complaint")
    workflow.add_edge("classify_complaint", "detect_priority")
    workflow.add_edge("detect_priority", "rag_decision")

    # Conditional branching for Agentic RAG
    workflow.add_conditional_edges(
        "rag_decision",
        rag_condition,
        {
            "retrieve_knowledge": "retrieve_knowledge",
```

```
            "route_department": "route_department"
        }
    )
    workflow.add_edge("retrieve_knowledge", "route_department")
    workflow.add_edge("route_department", "create_ticket")
    workflow.add_edge("create_ticket", END)

    return workflow.compile()

# Global compiled workflow instance
complaint_pipeline = build_complaint_workflow()

def run_complaint_pipeline(student_id: int, student_name: str, student_email: str,
                           title: str, description: str, attachment_path: str = "") -> ComplaintState:
    """Execute the multi-agent LangGraph complaint pipeline."""
    initial_state: ComplaintState = {
        "student_id": student_id,
        "student_name": student_name,
        "student_email": student_email,
        "title": title,
        "raw_text": description,
        "attachment_path": attachment_path,
        "key_issue": "",
        "impacted_asset": "",
        "urgency_context": "",
        "understanding_summary": "",
        "category": "",
        "category_confidence": 0.0,
        "priority": "",
        "priority_reason": "",
        "sla_hours": 24,
        "needs_rag": False,
        "rag_query": "",
        "retrieved_docs": [],
        "rag_context_summary": "",
        "department": "",
        "routing_confidence": 0.0,
        "needs_review": False,
        "routing_reason": "",
        "ticket_id": "",
```

```
0121 |         "status": "",
0122 |         "created_at": "",
0123 |         "logs": []
0124 |     }
0125 |     final_state = complaint_pipeline.invoke(initial_state)
0126 |     return final_state
```

# MODULE 3: AGENTIC RAG & CAMPUS KNOWLEDGE BASE

**Overview:** ChromaDB vector store with feature embedding function, dynamic RAG router, and campus policy documents.

## FILE: rag/vector_store.py

**Type:** Python | **Lines:** 205

*ChromaDB Vector Store with Fast Local Vectorizer & Resilient Fallback*

Path: smart_campus_ai/rag/vector_store.py

```
0001 | import os
0002 | import glob
0003 | import math
0004 | import re
0005 | from pathlib import Path
0006 | from typing import List, Dict, Any
0007 | from config import KNOWLEDGE_BASE_DIR, CHROMA_PERSIST_DIR, OPENAI_API_KEY, EMBEDDING_MODEL
0008 |
0009 | class FastLocalEmbeddingFunction:
0010 |     """Lightweight 256-dim feature-hashed normalized embedding function.
0011 |     Runs instantaneously offline without downloading massive onnx models.
0012 |     """
0013 |     def __init__(self, dim: int = 256):
0014 |         self.dim = dim
0015 |
0016 |     def __call__(self, input: List[str]) -> List[List[float]]:
0017 |         embeddings = []
0018 |         for text in input:
0019 |             words = re.findall(r"\w+", text.lower())
0020 |             vec = [0.0] * self.dim
0021 |             if not words:
0022 |                 embeddings.append(vec)
0023 |                 continue
0024 |             for w in words:
0025 |                 h1 = abs(hash(w)) % self.dim
0026 |                 vec[h1] += 2.0
0027 |                 for i in range(len(w) - 1):
0028 |                     h2 = abs(hash(w[i:i+2])) % self.dim
0029 |                     vec[h2] += 0.5
0030 |             norm = math.sqrt(sum(x * x for x in vec))
0031 |             if norm > 0:
0032 |                 vec = [x / norm for x in vec]
0033 |             embeddings.append(vec)
0034 |         return embeddings
0035 |
0036 | class CampusVectorStore:
0037 |     def __init__(self):
0038 |         self.kb_dir = Path(KNOWLEDGE_BASE_DIR)
0039 |         self.persist_dir = Path(CHROMA_PERSIST_DIR)
0040 |         self.collection_name = "smart_campus_kb_v2"
```

```
        self.documents: List[Dict[str, Any]] = []
        self._load_local_documents()
        self._chroma_collection = None
        self._init_chroma()

    def _load_local_documents(self):
        """Load and cleanly chunk markdown files from knowledge_base directory."""
        self.documents = []
        md_files = glob.glob(str(self.kb_dir / "*.md"))
        for file_path in md_files:
            filename = Path(file_path).name
            with open(file_path, "r", encoding="utf-8") as f:
                content = f.read()

            # Extract clean single-line document title from first # header
            doc_title = filename.replace(".md", "").replace("_", " ").title()
            for line in content.splitlines():
                line_str = line.strip()
                if line_str.startswith("# "):
                    doc_title = line_str[2:].strip()
                    break

            # Split document by section headers (## )
            raw_sections = re.split(r"\n##\s+", content)

            # Process Section 0 (preamble/intro before first ##)
            intro_lines = [l for l in raw_sections[0].splitlines() if not l.strip().startswith("# ") and l.strip()]
            if intro_lines:
                intro_body = "\n".join(intro_lines).strip()
                self.documents.append({
                    "id": f"{filename}-intro",
                    "source": filename,
                    "title": doc_title,
                    "section": "Overview",
                    "text": intro_body
                })

            # Process each markdown section
            for idx, sec in enumerate(raw_sections[1:], start=1):
                sec_lines = sec.strip().splitlines()
```

```
                if not sec_lines:
                    continue
                sec_title = sec_lines[0].strip().rstrip(":")
                sec_body = "\n".join(sec_lines[1:]).strip()

                self.documents.append({
                    "id": f"{filename}-sec{idx}",
                    "source": filename,
                    "title": doc_title,
                    "section": sec_title,
                    "text": sec_body
                })

    def _init_chroma(self):
        """Initialize ChromaDB with custom fast embedding function or OpenAI."""
        try:
            import chromadb
            client = chromadb.PersistentClient(path=str(self.persist_dir))

            embedding_fn = None
            if OPENAI_API_KEY:
                try:
                    from chromadb.utils import embedding_functions
                    embedding_fn = embedding_functions.OpenAIEmbeddingFunction(
                        api_key=OPENAI_API_KEY,
                        model_name=EMBEDDING_MODEL
                    )
                except Exception:
                    embedding_fn = None

            if not embedding_fn:
                embedding_fn = FastLocalEmbeddingFunction(dim=256)

            # Ensure clean re-indexing if collection already exists with old schema
            try:
                self._chroma_collection = client.get_collection(
                    name=self.collection_name,
                    embedding_function=embedding_fn
                )
                if self._chroma_collection.count() == 0:
```

```
                    raise Exception("Empty collection, populate")
            except Exception:
                try:
                    client.delete_collection(self.collection_name)
                except Exception:
                    pass
                self._chroma_collection = client.create_collection(
                    name=self.collection_name,
                    embedding_function=embedding_fn
                )
                if self.documents:
                    ids = [doc["id"] for doc in self.documents]
                    # Index enriched text for semantic matching
                    docs = [f"{doc['title']} - {doc['section']}\n\n{doc['text']}" for doc in self.documents]
                    metadatas = [{"source": doc["source"], "title": doc["title"], "section": doc["section"], "body": doc["text"]} for doc
                    self._chroma_collection.add(ids=ids, documents=docs, metadatas=metadatas)

        except Exception as e:
            self._chroma_collection = None

    def search(self, query: str, top_k: int = 3) -> List[Dict[str, Any]]:
        """Search relevant campus documents using ChromaDB with resilient fallback."""
        if not query or not query.strip():
            return []

        # Attempt ChromaDB query
        if self._chroma_collection is not None:
            try:
                results = self._chroma_collection.query(
                    query_texts=[query],
                    n_results=min(top_k, len(self.documents))
                )
                formatted = []
                if results and "documents" in results and results["documents"]:
                    doc_list = results["documents"][0]
                    meta_list = results["metadatas"][0] if "metadatas" in results else [{}] * len(doc_list)
                    for i, doc_text in enumerate(doc_list):
                        meta = meta_list[i] if i < len(meta_list) else {}
                        clean_body = meta.get("body", "")
                        if not clean_body:
```

```
                            # Strip prefix if present
                            parts = doc_text.split("\n\n", 1)
                            clean_body = parts[1] if len(parts) > 1 else doc_text

                        formatted.append({
                            "source": meta.get("source", "Campus Policy"),
                            "title": meta.get("title", "Policy Document"),
                            "section": meta.get("section", "Section"),
                            "text": clean_body
                        })
                if formatted:
                    return formatted
            except Exception:
                pass

        # Resilient fallback: Word-matching relevance
        scored_docs = []
        query_words = set(query.lower().replace(",", " ").replace(".", " ").split())
        for doc in self.documents:
            text_lower = f"{doc['title']} {doc['section']} {doc['text']}".lower()
            score = 0
            for w in query_words:
                if len(w) > 2 and w in text_lower:
                    score += text_lower.count(w)
            if score > 0:
                scored_docs.append((score, doc))

        scored_docs.sort(key=lambda x: x[0], reverse=True)
        return [
            {
                "source": d["source"],
                "title": d["title"],
                "section": d["section"],
                "text": d["text"]
            }
            for _, d in scored_docs[:top_k]
        ]

_vector_store_instance = None

```

```
0201 | def get_vector_store() -> CampusVectorStore:
0202 |     global _vector_store_instance
0203 |     if _vector_store_instance is None:
0204 |         _vector_store_instance = CampusVectorStore()
0205 |     return _vector_store_instance
```

## FILE: rag/rag_engine.py

**Type:** Python | **Lines:** 74

*Agentic RAG Dynamic Router, Heuristic Trigger & Context Retriever*

Path: smart_campus_ai/rag/rag_engine.py

```
0001 | from typing import Tuple, Dict, Any, List
0002 | from rag.vector_store import get_vector_store
0003 |
0004 | def evaluate_rag_necessity(key_issue: str, category: str, raw_text: str) -> Tuple[bool, str, str]:
0005 |     """
0006 |     Agentic RAG decision engine:
0007 |     Decides whether knowledge base retrieval is required to ground the AI's analysis,
0008 |     department routing, or SLA assignment in official campus documentation.
0009 |
0010 |     Returns:
0011 |         (needs_rag: bool, reason: str, search_query: str)
0012 |     """
0013 |     text_lower = f"{key_issue} {raw_text}".lower()
0014 |
0015 |     # Check indicators that warrant document retrieval
0016 |     triggers = []
0017 |
0018 |     if any(k in text_lower for k in ["exam", "practical", "examination", "viva", "test tomorrow", "deadline"]):
0019 |         triggers.append("Examination / academic timeline contingency protocol required")
0020 |
0021 |     if any(k in text_lower for k in ["lab", "computer", "pc", "workstation", "cse", "ece", "software", "hardware"]):
0022 |         triggers.append("Laboratory maintenance protocol and technician emergency contact lookup")
0023 |
0024 |     if any(k in text_lower for k in ["hostel", "room", "mess", "warden", "curfew", "water", "geyser", "dorm"]):
0025 |         triggers.append("Hostel residential guidelines and escalation hierarchy lookup")
0026 |
0027 |     if any(k in text_lower for k in ["danger", "spark", "fire", "electric", "leak", "hazard", "burst", "urgent"]):
0028 |         triggers.append("Emergency safety and immediate isolation procedures lookup")
0029 |
0030 |     if any(k in text_lower for k in ["bus", "transport", "shuttle", "route", "driver"]):
0031 |         triggers.append("Campus transport schedules and office contact lookup")
0032 |
0033 |     if any(k in text_lower for k in ["library", "book", "fine", "turnstile", "id card"]):
0034 |         triggers.append("Library catalog and circulation policy lookup")
0035 |
0036 |     if triggers:
0037 |         reason = "; ".join(triggers)
0038 |         # Formulate an effective targeted semantic search query
0039 |         query_terms = [category]
0040 |         if "exam" in text_lower or "practical" in text_lower:
0041 |             query_terms.append("practical examination computer laboratory contingency")
0042 |         if "lab" in text_lower:
0043 |             query_terms.append("laboratory maintenance support technician")
0044 |         if "hostel" in text_lower:
0045 |             query_terms.append("hostel residential repair maintenance rules")
0046 |         if "electric" in text_lower or "spark" in text_lower or "hazard" in text_lower:
0047 |             query_terms.append("electrical hazard emergency procedures SLA")
0048 |         if "sla" in text_lower or "urgent" in text_lower:
0049 |             query_terms.append("maintenance procedures SLA timeline")
0050 |
0051 |         search_query = " ".join(query_terms) if query_terms else key_issue
0052 |         return True, reason, search_query
0053 |
0054 |     return False, "Standard operational inquiry; campus knowledge retrieval not required.", ""
0055 |
0056 | def execute_agentic_rag(query: str, top_k: int = 2) -> Dict[str, Any]:
0057 |     """Retrieve relevant campus documents and summarize findings without hallucinating."""
0058 |     vector_store = get_vector_store()
0059 |     retrieved = vector_store.search(query, top_k=top_k)
0060 |
0061 |     if not retrieved:
0062 |         return {
0063 |             "retrieved_docs": [],
0064 |             "summary": "No specific campus knowledge base documents found for this query."
0065 |         }
0066 |
0067 |     summaries = []
0068 |     for doc in retrieved:
0069 |         summaries.append(f"[{doc['title']} - {doc['section']}]:\n{doc['text'][:300]}...")
0070 |
0071 |     return {
0072 |         "retrieved_docs": retrieved,
0073 |         "summary": "\n\n".join(summaries)
0074 |     }
```

## FILE: knowledge_base/college_policies.md

**Type:** Markdown | **Lines:** 23

*Campus Code of Conduct, Student Responsibilities & Exam Contingencies*

Path: smart_campus_ai/knowledge_base/college_policies.md

```
0001 | # Smart Campus Institute of Technology - General College Policies
0002 |
0003 | ## 1. Academic and Campus Code of Conduct
0004 | - All students, faculty, and administrative staff are expected to uphold institutional integrity, academic honesty, and respect for campu
0005 | - Campus facilities, including classrooms, auditoriums, sports complexes, and study zones, are accessible to registered students carrying
0006 | - Willful damage or defacing of campus property, unauthorized tampering with network infrastructure, or misuse of campus resources will l
0007 |
0008 | ## 2. Emergency Response Protocol
0009 | - In case of critical infrastructure failure (such as electrical hazards, major water pipe bursts, fire alarms, or server room overheatin
0010 | - Emergency Contact Hotline: **+1 (555) 019-9111** (available 24/7).
0011 | - Campus Health Center: Block D, Ground Floor, ext. 401.
0012 |
0013 | ## 3. Examination Contingency Rules
0014 | - If any academic facility (laboratories, examination halls, audio-visual rooms) encounters equipment failure within 48 hours of schedule
0015 |   - The issue is automatically designated as **High or Critical Priority**.
0016 |   - Lab Maintenance and Academic Affairs must initiate immediate troubleshooting within 2 hours.
0017 |   - Backup laboratory halls (Block C Lab 304 and Block B Lab 102) are designated as emergency standby exam centers.
0018 |   - The Course Coordinator and Head of Department must be notified immediately.
0019 |
0020 | ## 4. Digital and Network Infrastructure Policies
0021 | - The Central IT Support division governs the campus-wide Wi-Fi network, student portal, and laboratory network switches.
0022 | - Scheduled network maintenance occurs Sundays between 02:00 AM and 06:00 AM.
0023 | - Unauthorized installation of peer-to-peer software or tampering with network switches in labs will result in immediate suspension of Wi
```

## FILE: knowledge_base/laboratory_guidelines.md

**Type:** Markdown | **Lines:** 23

*Computing Laboratories, Workstation Protocols & Hardware Guidelines*

Path: smart_campus_ai/knowledge_base/laboratory_guidelines.md

```
0001 | # Smart Campus Laboratory Operating Guidelines & Maintenance Protocols
0002 |
0003 | ## 1. Laboratory Governance & Oversight
0004 | - All computing laboratories (CSE Lab 1, CSE Lab 2, CSE Lab 3, AI & Robotics Lab, Data Science Lab) fall under the direct supervision of
0005 | - Operating Hours: Monday to Saturday, 08:00 AM - 08:00 PM. Special access during hackathons or exams requires written HOD approval.
0006 |
0007 | ## 2. Computer Hardware and Software Standards
0008 | - Workstations in CSE laboratories are equipped with Intel Core i7 / AMD Ryzen processors, 32GB RAM, and Gigabit Ethernet connectivity.
0009 | - Software configuration: Ubuntu Linux LTS dual-booted with Windows 11 Enterprise, configured with pre-installed GCC, Python 3.12, Java J
0010 | - Students are strictly prohibited from modifying BIOS settings, installing unapproved bootloaders, or unplugging peripherals (mice, keyb
0011 |
0012 | ## 3. Maintenance Procedures & Fault Reporting
0013 | - **Routine Maintenance**: Carried out bi-weekly by designated systems engineers.
0014 | - **Urgent Faults (Non-Exam Period)**: Reported through the Smart Campus AI complaint portal. Lab technicians must inspect within 24 hour
0015 | - **Exam Period Emergencies**:
0016 |   - If computers, switches, or UPS backups in any computing laboratory fail during practical examinations or within 24 to 48 hours preced
0017 |   - On-duty Lab Technicians (Mr. Robert Vance & Ms. Anita Sharma) must arrive on-site within 15 minutes.
0018 |   - If more than 5 machines are non-operational, standby systems from the buffer stock (Storage Room 204) must be deployed immediately.
0019 |   - Emergency Lab Maintenance Contact: **lab-maintenance@campus.edu** / Phone: ext. 505.
0020 |
0021 | ## 4. Electronics, Physics, and Mechanical Laboratories
0022 | - Chemical and physics equipment issues must be reported to Science Lab Support.
0023 | - Electrical hazards, sparking sockets, or fume hood malfunctions must be treated as **Critical Priority** with immediate isolation of th
```

## FILE: knowledge_base/hostel_rules.md

**Type:** Markdown | **Lines:** 21

*Residential Living, Maintenance Protocols & Curfew Regulations*

Path: smart_campus_ai/knowledge_base/hostel_rules.md

```
0001 | # Smart Campus Residential Life & Hostel Regulations
0002 |
0003 | ## 1. Residential Overview
0004 | - Smart Campus operates four residential blocks: Block A (Junior Boys), Block B (Senior Boys), Block C (Junior Girls), and Block D (Senio
0005 | - Daily quiet hours: 10:30 PM to 06:00 AM.
0006 | - Entry curfew: 09:30 PM for all residential students unless official library/lab late passes are presented.
0007 |
0008 | ## 2. Maintenance and Repair Requests
0009 | - **Plumbing & Water Supply**:
0010 |   - Issues involving total loss of drinking water or bathroom pipe bursts are classified as **Critical Priority** (SLA: Resolution within
0011 |   - Minor tap leaks or slow drainage are classified as **Low Priority** (SLA: 48-72 hours).
0012 | - **Electrical Fixtures & Air Conditioning**:
0013 |   - Sparking switchboards, exposed live wires, or main breaker trips are **Critical Priority**. Power must be switched off at the floor d
0014 |   - Air conditioning or ceiling fan failures in living quarters are classified as **Medium Priority** (SLA: 24 hours during summer season
0015 | - **Furniture & Carpentry**:
0016 |   - Broken study chairs, bed frames, or door lock malfunctions are assigned to Estate Carpentry (SLA: 3 business days).
0017 |
0018 | ## 3. Sanitation & Hygiene
0019 | - Daily housekeeping is managed between 09:00 AM and 01:00 PM.
0020 | - Pests, garbage accumulation, or sanitation issues must be escalated to the Hostel Warden and routed to **Hostel Administration & Housek
0021 | - Chief Warden Contact: **hostel-office@campus.edu** / Warden Helpline: ext. 310.
```

## FILE: knowledge_base/department_contacts.md

**Type:** Markdown | **Lines:** 22

*Official Campus Department Directory, Room Numbers & Helpdesks*

Path: smart_campus_ai/knowledge_base/department_contacts.md

```
0001 | # Smart Campus Official Department Directory & Responsibilities
0002 |
0003 | ## 1. Department Directory & Contact Information
0004 |
0005 | | Department Name | Responsible Head / In-Charge | Email Contact | Primary Scope of Responsibilities |
0006 | |---|---|---|---|
0007 | | **Lab Maintenance** | Eng. Robert Vance | `lab-maintenance@campus.edu` | Computing labs, hardware malfunctions, lab OS installations, l
0008 | | **Hostel Administration** | Dr. Sarah Jenkins (Chief Warden) | `hostel-office@campus.edu` | Residential life, room allocations, hostel
0009 | | **Estate & Facilities** | Mr. David Kumar (Director) | `estate-facilities@campus.edu` | Campus physical infrastructure, civil works, el
0010 | | **IT Support & Systems** | Ms. Priya Patel (Lead Admin) | `it-support@campus.edu` | Campus portal, Wi-Fi access credentials, university
0011 | | **Library Services** | Prof. Michael Chang | `library@campus.edu` | Central library resources, RFID turnstiles, quiet reading room faci
0012 | | **Transport Office** | Capt. Arvind Singh | `transport@campus.edu` | Campus shuttle buses, transport routes, driver schedules, parking
0013 | | **Academic Affairs** | Dean Dr. Eleanor Ross | `academic-affairs@campus.edu` | Course schedules, lecture hall allocation, examination c
0014 |
0015 | ## 2. Office Locations & Campus Helpdesk Centers
0016 | - **Lab Maintenance**: Technology Tower, Ground Floor, Room TT-012
0017 | - **Hostel Office**: Central Student Center, 1st Floor
0018 | - **Estate & Facilities**: Maintenance Yard, Operations Building
0019 | - **IT Support & Helpdesk**: Tech Building, 2nd Floor, Room TB-205
0020 | - **Library Services**: Central Library, Main Circulation Desk
0021 | - **Transport Office**: Gate 2 Administration Complex
0022 | - **Academic Affairs**: Academic Block A, Deanery Suite
```

## FILE: knowledge_base/campus_maintenance_procedures.md

**Type:** Markdown | **Lines:** 19

*Campus Maintenance SLA Turnaround Matrix & Escalation Levels*

Path: smart_campus_ai/knowledge_base/campus_maintenance_procedures.md

```
0001 | # Smart Campus Facility Maintenance Procedures & Service Level Agreements (SLAs)
0002 |
0003 | ## 1. Complaint Priority Matrix & Service Level Agreements (SLAs)
0004 |
0005 | The campus maintenance department enforces strict resolution turnaround times based on the AI-assigned or admin-verified priority:
0006 |
0007 | | Priority Level | Criteria & Examples | Initial Response SLA | Final Resolution Target | Responsible Actions |
0008 | |---|---|---|---|---|
0009 | | **Critical** | Safety hazards, electrical sparks, fire risks, total water outage in hostels, main server outage | **Within 15 minutes**
0010 | | **High** | Failures impacting scheduled exams (e.g. lab computer failures before practicals), Wi-Fi outage across entire block, library
0011 | | **Medium** | AC malfunction in a regular classroom, individual hostel room fan failure, broken bench in lecture hall, water dispenser f
0012 | | **Low** | Aesthetic touchups, non-urgent painting, suggestion for new software library, minor squeaky door hinge | **Within 24 hours**
0013 |
0014 | ## 2. Maintenance Work Order Lifecycle
0015 | 1. **Intake**: Complaint logged by student or staff via Smart Campus AI.
0016 | 2. **AI Triage**: LangGraph workflow categorizes, scores priority, and routes to appropriate department.
0017 | 3. **Acknowledgment**: Automated notification with unique ticket ID (`CAMP-XXXX`) and estimated SLA.
0018 | 4. **Investigation**: Department technician conducts physical or remote diagnosis.
0019 | 5. **Resolution & Sign-off**: Work completed, resolution notes recorded, status changed from `In Progress` to `Resolved`.
```

## FILE: knowledge_base/complaint_management_policies.md

**Type:** Markdown | **Lines:** 20

*Student Grievance Redressal Charter, SLA Guarantees & Appeals*

Path: smart_campus_ai/knowledge_base/complaint_management_policies.md

```
0001 | # Smart Campus Student Complaint Management & Grievance Redressal Policy
0002 |
0003 | ## 1. Principles of Grievance Redressal
0004 | - **Student Right to Prompt Resolution**: Every enrolled student has the right to report campus infrastructure, academic support, or resi
0005 | - **Fairness & Accountability**: Complaints must be logged transparently with timestamps, department ownership, and verifiable audit trai
0006 | - **Confidentiality & Data Protection**: Sensitive student contact details are masked from unauthorized staff; only authorized administra
0007 |
0008 | ## 2. Complaint Categories
0009 | Complaints are systematically indexed into seven canonical categories:
0010 | 1. **Laboratory**: Computer hardware, software licenses, lab instruments, chemical apparatus, practical exam readiness.
0011 | 2. **Hostel**: Dormitory rooms, mess food quality, residential washrooms, corridor lighting, hostel security.
0012 | 3. **Classroom**: Projectors, smart boards, audio microphones, podium computers, desks, air conditioning.
0013 | 4. **Library**: Reading room noise, book inventory, electronic catalog access, RFID turnstile gates.
0014 | 5. **Transport**: Shuttle bus timing, route coverage, bus condition, student transport passes.
0015 | 6. **Infrastructure**: Campus roads, pathways, streetlights, building elevators, centralized water & electric mains.
0016 | 7. **Other**: General student inquiries, lost & found, administrative service bottlenecks.
0017 |
0018 | ## 3. Resolution Verification & Appeals
0019 | - When a complaint status transitions to **Resolved**, the student receives full technician notes and an option to confirm resolution or
0020 | - No complaint may be marked **Resolved** by an administrator without providing descriptive resolution notes.
```

# MODULE 4: SAAS PRESENTATION LAYER & DEDICATED PORTALS

**Overview:** Modern SaaS responsive user interfaces for Students, 7 Department Operations Stations, Central Admin Console, Knowledge Base Browser, and Architecture Visualizer.

## FILE: ui/styles.py

**Type:** Python | **Lines:** 440

*Global SaaS CSS Styling, Metric Card Themes, Badges & Color Palette*

Path: smart_campus_ai/ui/styles.py

```
0001 | def get_custom_css():
0002 |     return """
0003 |     <style>
0004 |     /* Google Fonts */
0005 |     @import url('https://fonts.googleapis.com/css2?family=Plus+Jakarta+Sans:wght@400;500;600;700;800&family=JetBrains+Mono:wght@500;700&d
0006 |
0007 |     html, body, [class*="css"] {
0008 |         font-family: 'Plus Jakarta Sans', -apple-system, BlinkMacSystemFont, sans-serif;
0009 |     }
0010 |
0011 |     /* Main Container Padding & Clean SaaS Background */
0012 |     .main .block-container {
0013 |         padding-top: 1.2rem;
0014 |         padding-bottom: 3.5rem;
0015 |         max-width: 1280px;
0016 |     }
0017 |
0018 |     /* Global Vibrant Header Styling */
0019 |     .saas-header {
0020 |         background: linear-gradient(135deg, #0F172A 0%, #1E3A8A 45%, #2563EB 80%, #0284C7 100%);
0021 |         color: #FFFFFF;
0022 |         padding: 2.2rem 2.6rem;
0023 |         border-radius: 18px;
0024 |         margin-bottom: 2rem;
0025 |         box-shadow: 0 12px 35px -8px rgba(30, 58, 138, 0.45);
0026 |         border: 1px solid rgba(255, 255, 255, 0.15);
0027 |         position: relative;
0028 |         overflow: hidden;
0029 |     }
0030 |     .saas-header::after {
0031 |         content: '';
0032 |         position: absolute;
0033 |         bottom: 0;
0034 |         left: 0;
0035 |         right: 0;
0036 |         height: 5px;
0037 |         background: linear-gradient(90deg, #38BDF8, #818CF8, #34D399, #FBBF24, #F43F5E);
0038 |     }
0039 |     .saas-header h1 {
0040 |         color: #FFFFFF !important;
```

```
0041 |         font-weight: 800;
0042 |         font-size: 2.3rem;
0043 |         margin-bottom: 0.35rem;
0044 |         letter-spacing: -0.03em;
0045 |         text-shadow: 0 2px 4px rgba(0, 0, 0, 0.2);
0046 |     }
0047 |     .saas-header p {
0048 |         color: #E2E8F0;
0049 |         font-size: 1.05rem;
0050 |         margin-bottom: 0;
0051 |         opacity: 0.95;
0052 |     }
0053 |
0054 |     /* Department Sub-header with custom vibrant accent bar */
0055 |     .dept-banner {
0056 |         padding: 1.8rem 2.2rem;
0057 |         border-radius: 16px;
0058 |         margin-bottom: 1.8rem;
0059 |         color: #0F172A;
0060 |         border: 1.5px solid #CBD5E1;
0061 |         box-shadow: 0 6px 20px rgba(0, 0, 0, 0.05);
0062 |         position: relative;
0063 |     }
0064 |
0065 |     /* Colorful Metric Cards */
0066 |     .metric-card {
0067 |         background: #FFFFFF;
0068 |         border-radius: 16px;
0069 |         padding: 1.4rem 1.6rem;
0070 |         border: 1px solid #E2E8F0;
0071 |         box-shadow: 0 4px 16px rgba(0, 0, 0, 0.04);
0072 |         transition: transform 0.25s cubic-bezier(0.16, 1, 0.3, 1), box-shadow 0.25s ease;
0073 |         position: relative;
0074 |         overflow: hidden;
0075 |     }
0076 |     .metric-card::before {
0077 |         content: '';
0078 |         position: absolute;
0079 |         top: 0;
0080 |         left: 0;
```

```
0081 |         right: 0;
0082 |         height: 5px;
0083 |         background: #2563EB;
0084 |     }
0085 |     .metric-card-total::before { background: linear-gradient(90deg, #0284C7, #38BDF8); }
0086 |     .metric-card-open::before { background: linear-gradient(90deg, #2563EB, #60A5FA); }
0087 |     .metric-card-progress::before { background: linear-gradient(90deg, #D97706, #FBBF24); }
0088 |     .metric-card-resolved::before { background: linear-gradient(90deg, #059669, #34D399); }
0089 |     .metric-card-urgent::before { background: linear-gradient(90deg, #DC2626, #F87171); }
0090 |     .metric-card-rating::before { background: linear-gradient(90deg, #7C3AED, #A78BFA); }
0091 |
0092 |     .metric-card:hover {
0093 |         transform: translateY(-3px);
0094 |         box-shadow: 0 10px 28px rgba(0, 0, 0, 0.08);
0095 |     }
0096 |     .metric-title {
0097 |         color: #64748B;
0098 |         font-size: 0.8rem;
0099 |         font-weight: 700;
0100 |         text-transform: uppercase;
0101 |         letter-spacing: 0.07em;
0102 |         margin-bottom: 0.4rem;
0103 |     }
0104 |     .metric-value {
0105 |         font-size: 2.2rem;
0106 |         font-weight: 800;
0107 |         color: #0F2744;
0108 |         line-height: 1;
0109 |     }
0110 |     .metric-sub {
0111 |         font-size: 0.82rem;
0112 |         color: #94A3B8;
0113 |         margin-top: 0.5rem;
0114 |         font-weight: 500;
0115 |     }
0116 |
0117 |     /* Rounded Cards */
0118 |     .saas-card {
0119 |         background: #FFFFFF;
0120 |         border-radius: 16px;
```

```
0121 |         padding: 1.6rem 1.8rem;
0122 |         border: 1px solid #E2E8F0;
0123 |         box-shadow: 0 4px 16px rgba(0, 0, 0, 0.03);
0124 |         margin-bottom: 1.6rem;
0125 |     }
0126 |     .saas-card-title {
0127 |         font-size: 1.3rem;
0128 |         font-weight: 700;
0129 |         color: #0F2744;
0130 |         margin-bottom: 1rem;
0131 |         letter-spacing: -0.02em;
0132 |     }
0133 |
0134 |     /* Role Badges */
0135 |     .role-badge-dept {
0136 |         display: inline-block;
0137 |         padding: 5px 14px;
0138 |         border-radius: 9999px;
0139 |         font-size: 0.76rem;
0140 |         font-weight: 800;
0141 |         text-transform: uppercase;
0142 |         letter-spacing: 0.06em;
0143 |         background: linear-gradient(90deg, #0284C7, #0284c7);
0144 |         color: #FFFFFF;
0145 |         box-shadow: 0 2px 8px rgba(2, 132, 199, 0.35);
0146 |         margin-bottom: 6px;
0147 |     }
0148 |     .role-badge-admin {
0149 |         display: inline-block;
0150 |         padding: 5px 14px;
0151 |         border-radius: 9999px;
0152 |         font-size: 0.76rem;
0153 |         font-weight: 800;
0154 |         text-transform: uppercase;
0155 |         letter-spacing: 0.06em;
0156 |         background: linear-gradient(90deg, #0F172A, #1E293B);
0157 |         color: #93C5FD;
0158 |         border: 1px solid #334155;
0159 |         box-shadow: 0 2px 8px rgba(15, 23, 42, 0.35);
0160 |         margin-bottom: 6px;
```

```
0161 |     }
0162 |
0163 |     /* Portal Scope Comparison Box */
0164 |     .portal-diff-container {
0165 |         display: grid;
0166 |         grid-template-columns: 1fr 1fr;
0167 |         gap: 16px;
0168 |         margin-bottom: 1.5rem;
0169 |     }
0170 |     .portal-diff-col {
0171 |         background: #FFFFFF;
0172 |         border: 1.5px solid #E2E8F0;
0173 |         border-radius: 14px;
0174 |         padding: 1.4rem;
0175 |         box-shadow: 0 4px 12px rgba(0, 0, 0, 0.03);
0176 |     }
0177 |
0178 |     /* Completion Highlight Card */
0179 |     .completed-action-card {
0180 |         background: linear-gradient(135deg, #ECFDF5 0%, #D1FAE5 100%);
0181 |         border: 2px solid #059669;
0182 |         border-radius: 14px;
0183 |         padding: 1.4rem 1.6rem;
0184 |         margin-top: 1.2rem;
0185 |         margin-bottom: 1.4rem;
0186 |         box-shadow: 0 4px 14px rgba(5, 150, 105, 0.12);
0187 |     }
0188 |
0189 |     /* Modern Badges with Rich Colors */
0190 |     .badge {
0191 |         display: inline-block;
0192 |         padding: 0.4rem 0.85rem;
0193 |         border-radius: 9999px;
0194 |         font-size: 0.78rem;
0195 |         font-weight: 700;
0196 |         text-transform: uppercase;
0197 |         letter-spacing: 0.04em;
0198 |     }
0199 |     .badge-open {
0200 |         background: linear-gradient(135deg, #EFF6FF 0%, #DBEAFE 100%);
```

```
0201 |         color: #1D4ED8;
0202 |         border: 1px solid #93C5FD;
0203 |     }
0204 |     .badge-in-progress {
0205 |         background: linear-gradient(135deg, #FFFBEB 0%, #FEF3C7 100%);
0206 |         color: #B45309;
0207 |         border: 1px solid #FDE68A;
0208 |     }
0209 |     .badge-resolved {
0210 |         background: linear-gradient(135deg, #ECFDF5 0%, #D1FAE5 100%);
0211 |         color: #047857;
0212 |         border: 1px solid #6EE7B7;
0213 |     }
0214 |     .badge-low {
0215 |         background-color: #F1F5F9;
0216 |         color: #475569;
0217 |         border: 1px solid #CBD5E1;
0218 |     }
0219 |     .badge-medium {
0220 |         background: linear-gradient(135deg, #FFF7ED 0%, #FFEDD5 100%);
0221 |         color: #C2410C;
0222 |         border: 1px solid #FDBA74;
0223 |     }
0224 |     .badge-high {
0225 |         background: linear-gradient(135deg, #FEF2F2 0%, #FEE2E2 100%);
0226 |         color: #B91C1C;
0227 |         border: 1px solid #FCA5A5;
0228 |     }
0229 |     .badge-critical {
0230 |         background: linear-gradient(135deg, #7F1D1D 0%, #991B1B 100%);
0231 |         color: #FFFFFF;
0232 |         border: 1px solid #B91C1C;
0233 |         box-shadow: 0 2px 8px rgba(185, 28, 28, 0.4);
0234 |     }
0235 |
0236 |     /* Vibrant Department Tags */
0237 |     .dept-tag {
0238 |         display: inline-block;
0239 |         padding: 0.3rem 0.75rem;
0240 |         border-radius: 8px;
```

```
0241 |         font-size: 0.82rem;
0242 |         font-weight: 700;
0243 |         letter-spacing: 0.02em;
0244 |     }
0245 |     .dept-tag-lab { background: #E0F2FE; color: #0284C7; border: 1.5px solid #38BDF8; }
0246 |     .dept-tag-hostel { background: #EEF2FF; color: #4F46E5; border: 1.5px solid #818CF8; }
0247 |     .dept-tag-estate { background: #ECFDF5; color: #059669; border: 1.5px solid #34D399; }
0248 |     .dept-tag-it { background: #F5F3FF; color: #7C3AED; border: 1.5px solid #A78BFA; }
0249 |     .dept-tag-library { background: #FFFBEB; color: #D97706; border: 1.5px solid #FBBF24; }
0250 |     .dept-tag-transport { background: #FFF7ED; color: #EA580C; border: 1.5px solid #FB923C; }
0251 |     .dept-tag-academic { background: #FFF1F2; color: #E11D48; border: 1.5px solid #F43F5E; }
0252 |
0253 |     /* RAG Document Passage Card */
0254 |     .rag-result-card {
0255 |         background: #FFFFFF;
0256 |         border-radius: 14px;
0257 |         padding: 1.4rem 1.6rem;
0258 |         margin-bottom: 1.2rem;
0259 |         border: 1px solid #E2E8F0;
0260 |         border-left: 5px solid #2563EB;
0261 |         box-shadow: 0 4px 14px rgba(0, 0, 0, 0.04);
0262 |         transition: transform 0.2s ease, box-shadow 0.2s ease;
0263 |     }
0264 |     .rag-result-card:hover {
0265 |         transform: translateY(-2px);
0266 |         box-shadow: 0 8px 24px rgba(37, 99, 235, 0.1);
0267 |     }
0268 |     .rag-pill-source {
0269 |         font-size: 0.76rem;
0270 |         font-weight: 700;
0271 |         background: #EFF6FF;
0272 |         color: #1D4ED8;
0273 |         padding: 3px 10px;
0274 |         border-radius: 9999px;
0275 |         border: 1px solid #BFDBFE;
0276 |     }
0277 |     .rag-pill-section {
0278 |         font-size: 0.76rem;
0279 |         font-weight: 700;
0280 |         background: #F0FDF4;
```

```
0281 |         color: #059669;
0282 |         padding: 3px 10px;
0283 |         border-radius: 9999px;
0284 |         border: 1px solid #A7F3D0;
0285 |     }
0286 |
0287 |     /* Notification Box */
0288 |     .notif-box {
0289 |         background: #FFFFFF;
0290 |         border: 1px solid #E2E8F0;
0291 |         border-left: 5px solid #2563EB;
0292 |         padding: 1.1rem 1.3rem;
0293 |         border-radius: 10px;
0294 |         margin-bottom: 0.9rem;
0295 |         transition: all 0.2s ease;
0296 |     }
0297 |     .notif-box-critical {
0298 |         border-left-color: #DC2626;
0299 |         background: #FEF2F2;
0300 |     }
0301 |     .notif-box-unread {
0302 |         box-shadow: 0 4px 14px rgba(37, 99, 235, 0.1);
0303 |         border-color: #93C5FD;
0304 |     }
0305 |
0306 |     /* Agent Timeline Stepper */
0307 |     .agent-step {
0308 |         background: #F8FAFC;
0309 |         border-left: 4px solid #2563EB;
0310 |         padding: 1rem 1.3rem;
0311 |         margin-bottom: 0.9rem;
0312 |         border-radius: 0 12px 12px 0;
0313 |         box-shadow: 0 2px 6px rgba(0, 0, 0, 0.02);
0314 |     }
0315 |     .agent-step-title {
0316 |         font-weight: 700;
0317 |         font-size: 0.95rem;
0318 |         color: #0F2744;
0319 |     }
0320 |     .agent-step-desc {
```

```
0321 |         font-size: 0.88rem;
0322 |         color: #334155;
0323 |         margin-top: 0.2rem;
0324 |     }
0325 |
0326 |     /* Ticket Card */
0327 |     .ticket-card {
0328 |         border-radius: 14px;
0329 |         border: 1px solid #E2E8F0;
0330 |         padding: 1.3rem 1.5rem;
0331 |         background: #FFFFFF;
0332 |         margin-bottom: 1.1rem;
0333 |         transition: all 0.25s ease;
0334 |     }
0335 |     .ticket-card:hover {
0336 |         border-color: #2563EB;
0337 |         box-shadow: 0 8px 24px rgba(37, 99, 235, 0.12);
0338 |     }
0339 |     .ticket-id {
0340 |         font-family: 'JetBrains Mono', monospace;
0341 |         font-weight: 700;
0342 |         font-size: 1.15rem;
0343 |         color: #2563EB;
0344 |     }
0345 |
0346 |     /* SLA Indicator */
0347 |     .sla-pill {
0348 |         display: inline-block;
0349 |         padding: 0.25rem 0.7rem;
0350 |         border-radius: 8px;
0351 |         font-size: 0.78rem;
0352 |         font-weight: 700;
0353 |         background: #F1F5F9;
0354 |         color: #1E293B;
0355 |         border: 1px solid #CBD5E1;
0356 |     }
0357 |
0358 |     /* Modern SaaS Buttons */
0359 |     .stButton > button {
0360 |         background: linear-gradient(135deg, #2563EB 0%, #1D4ED8 100%);
```

```
0361 |         color: white;
0362 |         border-radius: 10px;
0363 |         border: none;
0364 |         padding: 0.6rem 1.3rem;
0365 |         font-weight: 700;
0366 |         font-size: 0.95rem;
0367 |         transition: all 0.2s cubic-bezier(0.16, 1, 0.3, 1);
0368 |         box-shadow: 0 4px 14px rgba(37, 99, 235, 0.25);
0369 |     }
0370 |     .stButton > button:hover {
0371 |         background: linear-gradient(135deg, #1D4ED8 0%, #1E40AF 100%);
0372 |         box-shadow: 0 6px 20px rgba(37, 99, 235, 0.4);
0373 |         transform: translateY(-1px);
0374 |     }
0375 |
0376 |     /* History Timeline */
0377 |     .history-item {
0378 |         position: relative;
0379 |         padding-left: 26px;
0380 |         border-left: 2px solid #CBD5E1;
0381 |         margin-bottom: 18px;
0382 |     }
0383 |     .history-dot {
0384 |         position: absolute;
0385 |         left: -6px;
0386 |         top: 2px;
0387 |         width: 10px;
0388 |         height: 10px;
0389 |         border-radius: 50%;
0390 |         background-color: #2563EB;
0391 |         box-shadow: 0 0 0 3px #DBEAFE;
0392 |     }
0393 |     .history-time {
0394 |         font-size: 0.78rem;
0395 |         color: #94A3B8;
0396 |         font-weight: 600;
0397 |     }
0398 |     .history-body {
0399 |         font-size: 0.92rem;
0400 |         color: #1E293B;
```

```
0401 |         margin-top: 3px;
0402 |     }
0403 |     </style>
0404 |     """
0405 |
0406 | def get_status_badge_html(status: str, completed_by_dept: bool = False) -> str:
0407 |     s = (status or "Open").strip()
0408 |     cls = "badge-open"
0409 |     if s == "In Progress":
0410 |         cls = "badge-in-progress"
0411 |     elif s == "Resolved":
0412 |         cls = "badge-resolved"
0413 |         if completed_by_dept:
0414 |             return f'<span class="badge {cls}" style="border: 2px solid #059669; font-weight: 800; background: linear-gradient(135deg, #E
0415 |     return f'<span class="badge {cls}">{s}</span>'
0416 |
0417 | def get_priority_badge_html(priority: str) -> str:
0418 |     p = (priority or "Low").strip()
0419 |     cls = "badge-low"
0420 |     if p == "Medium":
0421 |         cls = "badge-medium"
0422 |     elif p == "High":
0423 |         cls = "badge-high"
0424 |     elif p == "Critical":
0425 |         cls = "badge-critical"
0426 |     return f'<span class="badge {cls}">{p}</span>'
0427 |
0428 | def get_dept_tag_html(department: str) -> str:
0429 |     dept = (department or "Unassigned").strip()
0430 |     slug_map = {
0431 |         "Lab Maintenance": "dept-tag-lab",
0432 |         "Hostel Administration": "dept-tag-hostel",
0433 |         "Estate & Facilities": "dept-tag-estate",
0434 |         "IT Support": "dept-tag-it",
0435 |         "Library Services": "dept-tag-library",
0436 |         "Transport Office": "dept-tag-transport",
0437 |         "Academic Affairs": "dept-tag-academic",
0438 |     }
0439 |     cls = slug_map.get(dept, "dept-tag-lab")
0440 |     return f'<span class="dept-tag {cls}">{dept}</span>'
```

## FILE: ui/landing.py

**Type:** Python | **Lines:** 237

*Enterprise SaaS Landing Page, Feature Highlights, Department Carousel & Workflow Overview* Path: smart_campus_ai/ui/landing.py

```
0001 | import streamlit as st
0002 | from config import DEPARTMENT_DETAILS
0003 |
0004 | def render_landing_page(navigate_to):
0005 |     # Header Banner
0006 |     st.markdown("""
0007 |     <div class="saas-header">
0008 |         <div style="display: flex; align-items: center; justify-content: space-between; flex-wrap: wrap;">
0009 |             <div>
0010 |                 <span style="font-size: 0.82rem; font-weight: 700; text-transform: uppercase; letter-spacing: 0.08em; color: #93C5FD; dis
0011 |                     Autonomous Campus Operations Platform
0012 |                 </span>
0013 |                 <h1>Smart Campus AI</h1>
0014 |                 <p>Intelligent Multi-Agent Complaint Management & Resolution System</p>
0015 |             </div>
0016 |             <div style="margin-top: 10px;">
0017 |                 <span style="background: rgba(255, 255, 255, 0.15); border: 1px solid rgba(255, 255, 255, 0.3); padding: 6px 14px; border
0018 |                     LangGraph Multi-Agent + Agentic RAG
0019 |                 </span>
0020 |             </div>
0021 |         </div>
0022 |         <p style="margin-top: 12px; max-width: 880px; line-height: 1.6; color: #E2E8F0;">
0023 |             Empowering students, campus departments, and central administrators with an intelligent 5-agent LangGraph workflow.
0024 |             Automatically comprehends complaints in natural language, categorizes issues, evaluates urgency & safety hazards,
0025 |             dynamically queries the campus knowledge base, and alerts responsible departmental staff.
0026 |         </p>
0027 |     </div>
0028 |     """, unsafe_allow_html=True)
0029 |
0030 |     # Portal Access Quick Action Cards (3 Pillars)
0031 |     col1, col2, col3 = st.columns(3)
0032 |
0033 |     with col1:
0034 |         st.markdown("""
0035 |         <div class="saas-card" style="border-top: 4px solid #2563EB; min-height: 250px;">
0036 |             <div class="saas-card-title">Student Portal</div>
0037 |             <p style="color: #64748B; font-size: 0.92rem; min-height: 70px;">
0038 |                 Submit campus grievances in natural language, upload photographic evidence, observe step-by-step multi-agent AI triaging,
0039 |             </p>
0040 |             <div style="font-size: 0.8rem; color: #94A3B8; margin-top: 10px;">
```

```
                Role: Enrolled Students | Real-time Ticket Status
            </div>
        </div>
        """, unsafe_allow_html=True)
        if st.button("Enter Student Portal ->", key="btn_enter_student", use_container_width=True):
            navigate_to("student")

    with col2:
        st.markdown("""
        <div class="saas-card" style="border-top: 4px solid #0284C7; min-height: 250px;">
            <div class="saas-card-title">Department Portals</div>
            <p style="color: #64748B; font-size: 0.92rem; min-height: 70px;">
                Dedicated operations workspaces for Lab Maintenance, Hostel Admin, Estate, IT, Library, Transport, and Academic Affairs w
            </p>
            <div style="font-size: 0.8rem; color: #94A3B8; margin-top: 10px;">
                Role: Department Officers & Specialists
            </div>
        </div>
        """, unsafe_allow_html=True)
        if st.button("Open Department Portals ->", key="btn_enter_depts", use_container_width=True):
            navigate_to("dept_page")

    with col3:
        st.markdown("""
        <div class="saas-card" style="border-top: 4px solid #0F2744; min-height: 250px;">
            <div class="saas-card-title">Central Admin Console</div>
            <p style="color: #64748B; font-size: 0.92rem; min-height: 70px;">
                Campus-wide governance cockpit: monitor department metrics, review high-priority emergency alerts, reassign tickets, and
            </p>
            <div style="font-size: 0.8rem; color: #94A3B8; margin-top: 10px;">
                Role: Campus Administrator / Deanery
            </div>
        </div>
        """, unsafe_allow_html=True)
        if st.button("Enter Admin Dashboard ->", key="btn_enter_admin", use_container_width=True):
            navigate_to("admin")

    st.markdown("<hr style='margin: 2rem 0; border: none; border-top: 1px solid #E2E8F0;'>", unsafe_allow_html=True)

    # Portal Difference & Completion-to-Resolved Workflow Highlight
```

```
    st.markdown("""
    <div class="saas-card" style="border-left: 5px solid #059669; background: #F0FDF4;">
        <div style="display: flex; justify-content: space-between; align-items: center; flex-wrap: wrap;">
            <div>
                <strong style="color: #065F46; font-size: 1.1rem; display: block; margin-bottom: 4px;">
                    Department Portal vs Central Admin: Seamless Operational Sync
                </strong>
                <p style="color: #047857; font-size: 0.92rem; margin: 0; max-width: 850px;">
                    <strong>Department Portal:</strong> On-site field execution, technician dispatch, and hardware parts replacement. Whe
                </p>
                <p style="color: #047857; font-size: 0.92rem; margin: 4px 0 0 0; max-width: 850px;">
                    <strong>Central Admin Console:</strong> University-wide strategic oversight, cross-department ticket transfers, insti
                </p>
            </div>
            <div style="margin-top: 10px;">
                <span class="badge badge-resolved" style="padding: 6px 14px; font-size: 0.82rem;">
                    Completed in Dept -> Marked Resolved in Admin
                </span>
            </div>
        </div>
    </div>
    """, unsafe_allow_html=True)

    # Department Showcase Carousel / Cards
    st.markdown("<h3 style='color: #0F2744; font-weight: 700; margin-bottom: 0.8rem;'>Dedicated Department Operations Centers</h3>", unsa
    st.markdown("<p style='color: #64748B; font-size: 0.92rem; margin-bottom: 1.2rem;'>Every department has an independent operations das

    d_cols = st.columns(len(DEPARTMENT_DETAILS))
    for idx, (d_name, d_meta) in enumerate(DEPARTMENT_DETAILS.items()):
        with d_cols[idx]:
            st.markdown(f"""
            <div style="background: {d_meta['bg_color']}; border: 1px solid {d_meta['border_color']}; border-top: 3px solid {d_meta['colo
                <strong style="color: {d_meta['color']}; font-size: 0.85rem; display: block; margin-bottom: 4px;">{d_name}</strong>
                <span style="font-size: 0.75rem; color: #475569; display: block;">Lead: {d_meta['head'].split('(')[0].strip()}</span>
                <span style="font-size: 0.7rem; color: #64748B; display: block; margin-top: 4px;">Code: {d_meta['default_username']}</spa
            </div>
            """, unsafe_allow_html=True)
            if st.button(f"View {d_name.split()[0]}", key=f"quick_dept_{idx}", use_container_width=True):
                st.session_state["active_dept_name"] = d_name
                navigate_to("dept_page")
```

```
0121 |
0122 |     st.markdown("<hr style='margin: 2rem 0; border: none; border-top: 1px solid #E2E8F0;'>", unsafe_allow_html=True)
0123 |
0124 |     # Key Features Grid
0125 |     st.markdown("<h3 style='color: #0F2744; font-weight: 700; margin-bottom: 1.2rem;'>Key System Capabilities</h3>", unsafe_allow_html=Tr
0126 |
0127 |     f1, f2, f3, f4 = st.columns(4)
0128 |     with f1:
0129 |         st.markdown("""
0130 |         <div class="saas-card" style="text-align: center; padding: 1.4rem 1rem; border-top: 3px solid #2563EB;">
0131 |             <h4 style="color: #0F2744; font-size: 1.05rem; font-weight: 700; margin-bottom: 0.4rem;">5 LangGraph Agents</h4>
0132 |             <p style="color: #64748B; font-size: 0.85rem; margin: 0;">
0133 |                 Dedicated agents for understanding, classification, priority scoring, department routing, and ticket lifecycle.
0134 |             </p>
0135 |         </div>
0136 |         """, unsafe_allow_html=True)
0137 |
0138 |     with f2:
0139 |         st.markdown("""
0140 |         <div class="saas-card" style="text-align: center; padding: 1.4rem 1rem; border-top: 3px solid #0284C7;">
0141 |             <h4 style="color: #0F2744; font-size: 1.05rem; font-weight: 700; margin-bottom: 0.4rem;">Agentic RAG Engine</h4>
0142 |             <p style="color: #64748B; font-size: 0.85rem; margin: 0;">
0143 |                 Autonomous decision routing to query the ChromaDB campus policy knowledge base for rules, exam SLAs, and contacts.
0144 |             </p>
0145 |         </div>
0146 |         """, unsafe_allow_html=True)
0147 |
0148 |     with f3:
0149 |         st.markdown("""
0150 |         <div class="saas-card" style="text-align: center; padding: 1.4rem 1rem; border-top: 3px solid #059669;">
0151 |             <h4 style="color: #0F2744; font-size: 1.05rem; font-weight: 700; margin-bottom: 0.4rem;">Department Alerting</h4>
0152 |             <p style="color: #64748B; font-size: 0.85rem; margin: 0;">
0153 |                 In-app notification system alerts department administrators immediately when complaints are routed to their queue.
0154 |             </p>
0155 |         </div>
0156 |         """, unsafe_allow_html=True)
0157 |
0158 |     with f4:
0159 |         st.markdown("""
0160 |         <div class="saas-card" style="text-align: center; padding: 1.4rem 1rem; border-top: 3px solid #7C3AED;">
0161 |             <h4 style="color: #0F2744; font-size: 1.05rem; font-weight: 700; margin-bottom: 0.4rem;">Audited Resolution</h4>
0162 |             <p style="color: #64748B; font-size: 0.85rem; margin: 0;">
0163 |                 Strict status transition history, resolution report enforcement, technician logs, and student rating feedback.
0164 |             </p>
0165 |         </div>
0166 |         """, unsafe_allow_html=True)
0167 |
0168 |     # Workflow Visual Representation
0169 |     st.markdown("<h3 style='color: #0F2744; font-weight: 700; margin-top: 1.5rem; margin-bottom: 1rem;'>Autonomous Multi-Agent AI Workflo
0170 |
0171 |     st.markdown("""
0172 |     <div class="saas-card">
0173 |         <div style="display: flex; flex-wrap: wrap; align-items: center; justify-content: space-between; gap: 8px;">
0174 |             <div style="text-align: center; min-width: 120px;">
0175 |                 <div style="background: #EFF6FF; border: 1px solid #BFDBFE; color: #1D4ED8; padding: 10px; border-radius: 10px; font-weig
0176 |                     1. Student Submit
0177 |                 </div>
0178 |                 <div style="font-size: 0.75rem; color: #64748B; margin-top: 4px;">Natural Language</div>
0179 |             </div>
0180 |             <div style="color: #94A3B8; font-size: 1.2rem; font-weight: bold;">-></div>
0181 |             <div style="text-align: center; min-width: 120px;">
0182 |                 <div style="background: #F8FAFC; border: 1px solid #CBD5E1; color: #0F2744; padding: 10px; border-radius: 10px; font-weig
0183 |                     2. Understanding
0184 |                 </div>
0185 |                 <div style="font-size: 0.75rem; color: #64748B; margin-top: 4px;">Agent 1</div>
0186 |             </div>
0187 |             <div style="color: #94A3B8; font-size: 1.2rem; font-weight: bold;">-></div>
0188 |             <div style="text-align: center; min-width: 120px;">
0189 |                 <div style="background: #F8FAFC; border: 1px solid #CBD5E1; color: #0F2744; padding: 10px; border-radius: 10px; font-weig
0190 |                     3. Classification
0191 |                 </div>
0192 |                 <div style="font-size: 0.75rem; color: #64748B; margin-top: 4px;">Agent 2</div>
0193 |             </div>
0194 |             <div style="color: #94A3B8; font-size: 1.2rem; font-weight: bold;">-></div>
0195 |             <div style="text-align: center; min-width: 120px;">
0196 |                 <div style="background: #F8FAFC; border: 1px solid #CBD5E1; color: #0F2744; padding: 10px; border-radius: 10px; font-weig
0197 |                     4. Priority Detect
0198 |                 </div>
0199 |                 <div style="font-size: 0.75rem; color: #64748B; margin-top: 4px;">Agent 3</div>
0200 |             </div>
```

```
0201 |             <div style="color: #94A3B8; font-size: 1.2rem; font-weight: bold;">-></div>
0202 |             <div style="text-align: center; min-width: 120px;">
0203 |                 <div style="background: #FEF3C7; border: 1px solid #FDE68A; color: #B45309; padding: 10px; border-radius: 10px; font-weig
0204 |                     5. Agentic RAG
0205 |                 </div>
0206 |                 <div style="font-size: 0.75rem; color: #64748B; margin-top: 4px;">ChromaDB Query</div>
0207 |             </div>
0208 |             <div style="color: #94A3B8; font-size: 1.2rem; font-weight: bold;">-></div>
0209 |             <div style="text-align: center; min-width: 120px;">
0210 |                 <div style="background: #F8FAFC; border: 1px solid #CBD5E1; color: #0F2744; padding: 10px; border-radius: 10px; font-weig
0211 |                     6. Dept Routing
0212 |                 </div>
0213 |                 <div style="font-size: 0.75rem; color: #64748B; margin-top: 4px;">Agent 4</div>
0214 |             </div>
0215 |             <div style="color: #94A3B8; font-size: 1.2rem; font-weight: bold;">-></div>
0216 |             <div style="text-align: center; min-width: 120px;">
0217 |                 <div style="background: #ECFDF5; border: 1px solid #A7F3D0; color: #047857; padding: 10px; border-radius: 10px; font-weig
0218 |                     7. Ticket & Alerts
0219 |                 </div>
0220 |                 <div style="font-size: 0.75rem; color: #64748B; margin-top: 4px;">Agent 5 (CAMP-XXXX)</div>
0221 |             </div>
0222 |         </div>
0223 |     </div>
0224 |     """, unsafe_allow_html=True)
0225 |
0226 |     # Official Demonstration Scenario Box (No Emojis)
0227 |     st.markdown("""
0228 |     <div class="saas-card" style="background: #F8FAFC; border-left: 5px solid #2563EB;">
0229 |         <h4 style="color: #0F2744; font-weight: 700; margin-bottom: 0.4rem;">Official Demonstration Scenario</h4>
0230 |         <p style="color: #475569; font-size: 0.9rem; margin-bottom: 0.8rem;">
0231 |             Test the evaluation prompt: <em>"The computers in the CSE laboratory are not working and we have a practical examination tomo
0232 |         </p>
0233 |     </div>
0234 |     """, unsafe_allow_html=True)
0235 |     if st.button("Load Test Complaint in Student Portal ->", key="btn_load_demo_landing"):
0236 |         st.session_state["demo_prefill"] = True
0237 |         navigate_to("student")
```

## FILE: ui/login_pages.py

**Type:** Python | **Lines:** 213

*Dedicated Portal Sign-In Screens for Student, Department Staff & Central Admin*

Path: smart_campus_ai/ui/login_pages.py

```
0001 | import streamlit as st
0002 | import database as db
0003 | from config import DEPARTMENT_DETAILS
0004 |
0005 | def render_login_hub(navigate_to):
0006 |     st.markdown("""
0007 |     <div class="saas-header">
0008 |         <h1>Smart Campus Authentication Portal</h1>
0009 |         <p>Select your institutional role to access your dedicated campus interface.</p>
0010 |     </div>
0011 |     """, unsafe_allow_html=True)
0012 |
0013 |     col1, col2, col3 = st.columns(3)
0014 |
0015 |     with col1:
0016 |         st.markdown("""
0017 |         <div class="saas-card" style="border-top: 4px solid #2563EB; min-height: 240px;">
0018 |             <div class="saas-card-title">Student Portal</div>
0019 |             <p style="color: #64748B; font-size: 0.9rem;">
0020 |                 Submit natural-language complaints, track ticket progress in real-time, view technician notes, and rate resolutions.
0021 |             </p>
0022 |             <div style="font-size: 0.8rem; color: #94A3B8; margin-top: 15px;">
0023 |                 Credentials: <code>student1</code> / <code>Student@123</code>
0024 |             </div>
0025 |         </div>
0026 |         """, unsafe_allow_html=True)
0027 |         if st.button("Open Student Login Page", key="btn_goto_student_login", use_container_width=True):
0028 |             navigate_to("login_student")
0029 |
0030 |     with col2:
0031 |         st.markdown("""
0032 |         <div class="saas-card" style="border-top: 4px solid #0284C7; min-height: 240px;">
0033 |             <div class="saas-card-title">Department Officer Portal</div>
0034 |             <p style="color: #64748B; font-size: 0.9rem;">
0035 |                 Access dedicated department queues, receive real-time ticket alerts, assign field technicians, and update status.
0036 |             </p>
0037 |             <div style="font-size: 0.8rem; color: #94A3B8; margin-top: 15px;">
0038 |                 Credentials: <code>dept_lab</code>, <code>dept_hostel</code>, etc.
0039 |             </div>
0040 |         </div>
```

```
        """, unsafe_allow_html=True)
        if st.button("Open Department Login Page", key="btn_goto_dept_login", use_container_width=True):
            navigate_to("login_dept")

    with col3:
        st.markdown("""
        <div class="saas-card" style="border-top: 4px solid #0F2744; min-height: 240px;">
            <div class="saas-card-title">Central Admin Console</div>
            <p style="color: #64748B; font-size: 0.9rem;">
                Campus-wide governance cockpit: institutional analytics, cross-department triage, SLA compliance, and audit records.
            </p>
            <div style="font-size: 0.8rem; color: #94A3B8; margin-top: 15px;">
                Credentials: <code>admin</code> / <code>Admin@123</code>
            </div>
        </div>
        """, unsafe_allow_html=True)
        if st.button("Open Central Admin Login Page", key="btn_goto_admin_login", use_container_width=True):
            navigate_to("login_admin")

def render_student_login_page(navigate_to):
    st.markdown("""
    <div class="saas-header">
        <h1>Student Portal Sign In</h1>
        <p>Access your student grievance dashboard and track campus tickets.</p>
    </div>
    """, unsafe_allow_html=True)

    tab_sign_in, tab_register = st.tabs(["Student Sign In", "Register New Student Account"])

    with tab_sign_in:
        col_l, col_m, col_r = st.columns([1, 2, 1])
        with col_m:
            with st.form("dedicated_student_login_form"):
                st.markdown("<h3 style='color: #0F2744; margin-bottom: 0.8rem;'>Student Sign In</h3>", unsafe_allow_html=True)
                username = st.text_input("Username or University Email", value="student1")
                password = st.text_input("Password", type="password", value="Student@123")
                submitted = st.form_submit_button("Sign In to Student Portal", use_container_width=True)

                if submitted:
                    user = db.authenticate_user(username.strip(), password)
                    if user and user["role"] == "student":
                        st.session_state["student_user"] = user
                        st.success(f"Welcome back, {user['full_name']}!")
                        navigate_to("student")
                    else:
                        st.error("Invalid student credentials. Please check your username and password.")

            st.markdown("""
            <div style="background: #EFF6FF; border: 1px solid #BFDBFE; padding: 12px; border-radius: 8px; margin-top: 15px; font-size: 0
                <strong>Demo Student Account:</strong><br>
                Username: <code>student1</code><br>
                Password: <code>Student@123</code>
            </div>
            """, unsafe_allow_html=True)

    with tab_register:
        col_l, col_m, col_r = st.columns([1, 2, 1])
        with col_m:
            with st.form("dedicated_student_reg_form"):
                st.markdown("<h3 style='color: #0F2744; margin-bottom: 0.8rem;'>New Student Registration</h3>", unsafe_allow_html=True)
                full_name = st.text_input("Full Name *", placeholder="e.g. Maya Lin")
                username = st.text_input("Choose Username *", placeholder="e.g. mayalin25")
                email = st.text_input("University Email *", placeholder="e.g. mayalin@student.campus.edu")
                dept = st.selectbox("Academic Department", [
                    "Computer Science",
                    "Electronics & Communication",
                    "Mechanical Engineering",
                    "Civil Engineering",
                    "Information Technology",
                    "Biotechnology",
                    "Business Administration"
                ])
                phone = st.text_input("Phone Number", placeholder="e.g. +1-555-0144")
                pwd = st.text_input("Password *", type="password")
                pwd_confirm = st.text_input("Confirm Password *", type="password")
                reg_submitted = st.form_submit_button("Create Account", use_container_width=True)

                if reg_submitted:
                    if not full_name.strip() or not username.strip() or not email.strip() or not pwd.strip():
                        st.error("Please fill in all mandatory fields.")
```

```
                    elif pwd != pwd_confirm:
                        st.error("Passwords do not match.")
                    else:
                        ok, user_id, msg = db.create_user(
                            username=username.strip(),
                            password=pwd,
                            email=email.strip(),
                            full_name=full_name.strip(),
                            role="student",
                            department=dept,
                            phone=phone.strip()
                        )
                        if ok:
                            st.success(f"{msg} You can now log in using the 'Student Sign In' tab.")
                        else:
                            st.error(msg)

def render_dept_login_page(navigate_to):
    st.markdown("""
    <div class="saas-header" style="background: linear-gradient(135deg, #0284C7 0%, #0369A1 50%, #0F172A 100%);">
        <h1>Department Officer Sign In</h1>
        <p>Authorized portal access for department heads, engineers, and operational teams.</p>
    </div>
    """, unsafe_allow_html=True)

    col_l, col_m, col_r = st.columns([1, 2, 1])
    with col_m:
        with st.form("dedicated_dept_login_form"):
            st.markdown("<h3 style='color: #0F2744; margin-bottom: 0.8rem;'>Department Staff Login</h3>", unsafe_allow_html=True)

            selected_dept = st.selectbox("Select Your Department", list(DEPARTMENT_DETAILS.keys()))
            default_user = DEPARTMENT_DETAILS[selected_dept]["default_username"]
            default_pwd = f"{selected_dept.split()[0]}@123"

            username = st.text_input("Staff Username", value=default_user)
            password = st.text_input("Department Password", type="password", value=default_pwd)
            submitted = st.form_submit_button("Sign In to Department Dashboard", use_container_width=True)

            if submitted:
                user = db.authenticate_user(username.strip(), password)
                if user and (user["role"] in ["dept_admin", "admin"] or user["department"] == selected_dept):
                    st.session_state["dept_user"] = user
                    st.session_state["active_dept_name"] = selected_dept
                    st.success(f"Signed in as {user['full_name']} for {selected_dept}!")
                    navigate_to("dept_page")
                else:
                    st.error(f"Invalid department credentials for {selected_dept}.")

        st.markdown("""
        <div style="background: #F0FDF4; border: 1px solid #BBF7D0; padding: 12px; border-radius: 8px; margin-top: 15px; font-size: 0.82r
            <strong>Default Department Accounts:</strong><br>
            - Lab Maintenance: <code>dept_lab</code> / <code>Lab@123</code><br>
            - Hostel Admin: <code>dept_hostel</code> / <code>Hostel@123</code><br>
            - Estate & Facilities: <code>dept_estate</code> / <code>Estate@123</code><br>
            - IT Support: <code>dept_it</code> / <code>IT@123</code><br>
            - Library Services: <code>dept_library</code> / <code>Library@123</code><br>
            - Transport Office: <code>dept_transport</code> / <code>Transport@123</code><br>
            - Academic Affairs: <code>dept_academic</code> / <code>Academic@123</code>
        </div>
        """, unsafe_allow_html=True)

def render_admin_login_page(navigate_to):
    st.markdown("""
    <div class="saas-header" style="background: linear-gradient(135deg, #0F172A 0%, #1E293B 50%, #334155 100%);">
        <h1>Central Administrator Sign In</h1>
        <p>Institutional oversight, cross-department governance, and system-wide SLA monitoring.</p>
    </div>
    """, unsafe_allow_html=True)

    col_l, col_m, col_r = st.columns([1, 2, 1])
    with col_m:
        with st.form("dedicated_admin_login_form"):
            st.markdown("<h3 style='color: #0F2744; margin-bottom: 0.8rem;'>Admin Authentication</h3>", unsafe_allow_html=True)
            username = st.text_input("Administrator Username", value="admin")
            password = st.text_input("Master Password", type="password", value="Admin@123")
            submitted = st.form_submit_button("Sign In as Administrator", use_container_width=True)

            if submitted:
                user = db.authenticate_user(username.strip(), password)
                if user and user["role"] == "admin":
```

```
0201 |                     st.session_state["admin_user"] = user
0202 |                     st.success("Central Administrator Access Verified.")
0203 |                     navigate_to("admin")
0204 |                 else:
0205 |                     st.error("Access denied. Invalid administrator credentials.")
0206 |
0207 |         st.markdown("""
0208 |         <div style="background: #FEF2F2; border: 1px solid #FECACA; padding: 12px; border-radius: 8px; margin-top: 15px; font-size: 0.85r
0209 |             <strong>Default Central Admin:</strong><br>
0210 |             Username: <code>admin</code><br>
0211 |             Password: <code>Admin@123</code>
0212 |         </div>
0213 |         """, unsafe_allow_html=True)
```

## FILE: ui/student_portal.py

**Type:** Python | **Lines:** 374

*Student Portal: Complaint Submission, AI Execution Trace, Status Tracking & 5-Star Rating*

Path: smart_campus_ai/ui/student_portal.py

```
0001 | import os
0002 | import streamlit as st
0003 | from datetime import datetime
0004 | from config import UPLOAD_DIR, CATEGORIES, DEPARTMENTS
0005 | import database as db
0006 | from agents.workflow import run_complaint_pipeline
0007 | from ui.styles import get_status_badge_html, get_priority_badge_html, get_dept_tag_html
0008 |
0009 | def render_student_portal():
0010 |     st.markdown("""
0011 |     <div style="display: flex; align-items: center; justify-content: space-between; margin-bottom: 1.5rem;">
0012 |         <div>
0013 |             <h2 style="color: #0F2744; font-weight: 800; margin: 0;">Student Grievance Portal</h2>
0014 |             <p style="color: #64748B; margin: 0; font-size: 0.95rem;">Submit issues, follow automated AI triaging, and track resolution p
0015 |         </div>
0016 |     </div>
0017 |     """, unsafe_allow_html=True)
0018 |
0019 |     student_user = st.session_state.get("student_user")
0020 |
0021 |     if not student_user:
0022 |         render_student_auth()
0023 |         return
0024 |
0025 |     # Student Sub-navigation
0026 |     tab1, tab2, tab3, tab4 = st.tabs([
0027 |         "Submit New Complaint",
0028 |         "Track Ticket Status",
0029 |         "My Complaint History",
0030 |         "Student Profile"
0031 |     ])
0032 |
0033 |     with tab1:
0034 |         render_complaint_submission(student_user)
0035 |
0036 |     with tab2:
0037 |         render_ticket_tracking(student_user)
0038 |
0039 |     with tab3:
0040 |         render_complaint_history(student_user)
```

```
0041 |
0042 |     with tab4:
0043 |         render_student_profile(student_user)
0044 |
0045 | def render_student_auth():
0046 |     st.markdown("""
0047 |     <div class="saas-card" style="max-width: 520px; margin: 0 auto;">
0048 |         <h3 style="color: #0F2744; font-weight: 700; margin-bottom: 0.5rem; text-align: center;">Student Portal Access</h3>
0049 |         <p style="color: #64748B; font-size: 0.9rem; text-align: center; margin-bottom: 1.5rem;">Log in with your university student cred
0050 |     </div>
0051 |     """, unsafe_allow_html=True)
0052 |
0053 |     auth_mode = st.radio("Select Action", ["Student Login", "Register New Student Account"], horizontal=True, label_visibility="collapsed
0054 |
0055 |     col_l, col_m, col_r = st.columns([1, 2, 1])
0056 |     with col_m:
0057 |         if auth_mode == "Student Login":
0058 |             with st.form("student_login_form"):
0059 |                 st.markdown("<h4 style='color: #0F2744;'>Student Sign In</h4>", unsafe_allow_html=True)
0060 |                 username = st.text_input("Username or University Email", value="student1")
0061 |                 password = st.text_input("Password", type="password", value="Student@123")
0062 |                 submitted = st.form_submit_button("Sign In to Student Portal", use_container_width=True)
0063 |
0064 |                 if submitted:
0065 |                     user = db.authenticate_user(username.strip(), password)
0066 |                     if user and user["role"] == "student":
0067 |                         st.session_state["student_user"] = user
0068 |                         st.success(f"Welcome back, {user['full_name']}!")
0069 |                         st.rerun()
0070 |                     else:
0071 |                         st.error("Invalid student credentials. Please check username/password.")
0072 |
0073 |             st.info("Default Student Demo: Username: student1 | Password: Student@123")
0074 |
0075 |         else:
0076 |             with st.form("student_reg_form"):
0077 |                 st.markdown("<h4 style='color: #0F2744;'>Student Registration</h4>", unsafe_allow_html=True)
0078 |                 full_name = st.text_input("Full Name", placeholder="e.g. John Doe")
0079 |                 username = st.text_input("Choose Username", placeholder="e.g. jdoe24")
0080 |                 email = st.text_input("Student Email", placeholder="e.g. jdoe24@student.campus.edu")
```

```
0081 |                 dept = st.selectbox("Academic Department", [
0082 |                     "Computer Science",
0083 |                     "Electronics & Communication",
0084 |                     "Mechanical Engineering",
0085 |                     "Civil Engineering",
0086 |                     "Information Technology",
0087 |                     "Biotechnology",
0088 |                     "Business Administration"
0089 |                 ])
0090 |                 phone = st.text_input("Mobile Number", placeholder="e.g. +1 555-0199")
0091 |                 pwd = st.text_input("Create Password", type="password")
0092 |                 pwd_confirm = st.text_input("Confirm Password", type="password")
0093 |                 reg_submitted = st.form_submit_button("Create Student Account", use_container_width=True)
0094 |
0095 |                 if reg_submitted:
0096 |                     if not full_name or not username or not email or not pwd:
0097 |                         st.error("Please fill in all required fields.")
0098 |                     elif pwd != pwd_confirm:
0099 |                         st.error("Passwords do not match.")
0100 |                     else:
0101 |                         ok, user_id, msg = db.create_user(
0102 |                             username=username.strip(),
0103 |                             password=pwd,
0104 |                             email=email.strip(),
0105 |                             full_name=full_name.strip(),
0106 |                             role="student",
0107 |                             department=dept,
0108 |                             phone=phone.strip()
0109 |                         )
0110 |                         if ok:
0111 |                             st.success(msg)
0112 |                             st.info("You can now switch to 'Student Login' above to sign in.")
0113 |                         else:
0114 |                             st.error(msg)
0115 |
0116 | def render_complaint_submission(student_user):
0117 |     st.markdown("<div class='saas-card'>", unsafe_allow_html=True)
0118 |     st.markdown("<div class='saas-card-title'>Lodge a Campus Complaint</div>", unsafe_allow_html=True)
0119 |     st.markdown("""
0120 |         <p style='color: #64748B; font-size: 0.9rem;'>
```

```
            Describe the issue clearly. Our 5-agent LangGraph system will automatically analyze your complaint,
            classify the domain, assess urgency & safety, consult the campus policy knowledge base via Agentic RAG,
            assign the responsible department, and alert department administrators in real time.
        </p>
    """, unsafe_allow_html=True)

    default_title = ""
    default_desc = ""
    if st.session_state.get("demo_prefill", False):
        default_title = "CSE Laboratory Computers Out of Service"
        default_desc = "The computers in the CSE laboratory are not working and we have a practical examination tomorrow."
        st.session_state["demo_prefill"] = False
        st.toast("Pre-filled official demonstration test scenario!", icon=None)

    col_btn1, col_btn2 = st.columns([3, 1])
    with col_btn2:
        if st.button("Load Test Scenario", help="Pre-fill CSE Lab Computers exam scenario"):
            default_title = "CSE Laboratory Computers Out of Service"
            default_desc = "The computers in the CSE laboratory are not working and we have a practical examination tomorrow."

    with st.form("complaint_submission_form"):
        title = st.text_input("Complaint Title *", value=default_title, placeholder="e.g. Broken AC in Room 204 or Lab Computers Not Boot

        col_c1, col_c2 = st.columns(2)
        with col_c1:
            suggested_category = st.selectbox(
                "Complaint Category (AI will automatically refine this)",
                ["Auto-Detect via AI"] + CATEGORIES
            )
        with col_c2:
            st.text_input("Student Name & ID", value=f"{student_user['full_name']} ({student_user['username']})", disabled=True)

        description = st.text_area(
            "Detailed Complaint Description *",
            value=default_desc,
            placeholder="Please detail the location, problem specifics, and any time sensitivity or examination deadlines...",
            height=120
        )

        attachment = st.file_uploader(
```

```
            "Attachment Upload (Optional: Screenshot, Photo of damage, PDF proof)",
            type=["png", "jpg", "jpeg", "pdf", "txt"]
        )

        submitted = st.form_submit_button("Submit Complaint to AI Agent Workflow", use_container_width=True)

    if submitted:
        if not title.strip() or not description.strip():
            st.error("Please enter both a complaint title and a detailed description.")
            return

        attachment_path = ""
        if attachment is not None:
            safe_filename = f"{datetime.now().strftime('%Y%m%d_%H%M%S')}_{attachment.name.replace(' ', '_')}"
            save_path = UPLOAD_DIR / safe_filename
            with open(save_path, "wb") as f:
                f.write(attachment.getbuffer())
            attachment_path = str(save_path)

        with st.spinner("Autonomous LangGraph Agent Workflow processing complaint..."):
            pipeline_result = run_complaint_pipeline(
                student_id=student_user["id"],
                student_name=student_user["full_name"],
                student_email=student_user["email"],
                title=title.strip(),
                description=description.strip(),
                attachment_path=attachment_path
            )

        st.session_state["latest_submitted_ticket"] = pipeline_result["ticket_id"]

        st.success(f"Complaint successfully processed and ticket generated: {pipeline_result['ticket_id']}")

        st.markdown("<h4 style='color: #0F2744; margin-top: 1.5rem;'>Multi-Agent Processing Trace</h4>", unsafe_allow_html=True)
        for log_entry in pipeline_result.get("logs", []):
            st.markdown(f"""
            <div class="agent-step">
                <div class="agent-step-desc">{log_entry}</div>
            </div>
            """, unsafe_allow_html=True)
```

```
0201 |
0202 |         # Display Result Card
0203 |         st.markdown(f"""
0204 |         <div class="ticket-card" style="border: 2px solid #2563EB; background: #F8FAFC;">
0205 |             <div style="display: flex; justify-content: space-between; align-items: center; margin-bottom: 0.8rem;">
0206 |                 <span class="ticket-id">Ticket ID: {pipeline_result['ticket_id']}</span>
0207 |                 <div>
0208 |                     {get_status_badge_html(pipeline_result['status'])}
0209 |                     {get_priority_badge_html(pipeline_result['priority'])}
0210 |                 </div>
0211 |             </div>
0212 |             <div style="font-size: 1.15rem; font-weight: 700; color: #0F2744; margin-bottom: 0.5rem;">{title}</div>
0213 |             <div style="display: grid; grid-template-columns: repeat(auto-fit, minmax(200px, 1fr)); gap: 10px; margin-top: 10px;">
0214 |                 <div><strong>Category:</strong> {pipeline_result['category']}</div>
0215 |                 <div><strong>Responsible Dept:</strong> {pipeline_result['department']}</div>
0216 |                 <div><strong>Target SLA:</strong> {pipeline_result.get('sla_hours', 24)} Hours</div>
0217 |                 <div><strong>Priority Reason:</strong> {pipeline_result['priority_reason']}</div>
0218 |             </div>
0219 |         </div>
0220 |         """, unsafe_allow_html=True)
0221 |
0222 |         if pipeline_result.get("retrieved_docs"):
0223 |             with st.expander("View Campus Documents Retrieved by Agentic RAG"):
0224 |                 for doc in pipeline_result["retrieved_docs"]:
0225 |                     st.markdown(f"**Document**: `{doc.get('title')}` | **Section**: `{doc.get('section')}`")
0226 |                     st.caption(doc.get("text", "")[:400] + "...")
0227 |
0228 |     st.markdown("</div>", unsafe_allow_html=True)
0229 |
0230 | def render_ticket_tracking(student_user):
0231 |     st.markdown("<div class='saas-card'>", unsafe_allow_html=True)
0232 |     st.markdown("<div class='saas-card-title'>Track Complaint Status</div>", unsafe_allow_html=True)
0233 |
0234 |     col_t1, col_t2 = st.columns([3, 1])
0235 |     with col_t1:
0236 |         default_search = st.session_state.get("latest_submitted_ticket", "")
0237 |         search_ticket = st.text_input("Enter Ticket ID (e.g. CAMP-0001)", value=default_search)
0238 |     with col_t2:
0239 |         st.write("")
0240 |         st.write("")
```

```
0241 |         search_clicked = st.button("Search Ticket", use_container_width=True)
0242 |
0243 |     target_ticket = search_ticket.strip().upper()
0244 |     if target_ticket:
0245 |         complaint = db.get_complaint_by_ticket_id(target_ticket)
0246 |         if not complaint:
0247 |             st.warning(f"No complaint ticket found matching '{target_ticket}'.")
0248 |         else:
0249 |             render_single_ticket_detail(complaint)
0250 |
0251 |     st.markdown("</div>", unsafe_allow_html=True)
0252 |
0253 | def render_single_ticket_detail(complaint):
0254 |     ticket_id = complaint["ticket_id"]
0255 |     status_html = get_status_badge_html(complaint["status"])
0256 |     priority_html = get_priority_badge_html(complaint["priority"])
0257 |     dept_tag = get_dept_tag_html(complaint["department"])
0258 |
0259 |     st.markdown(f"""
0260 |     <div class="ticket-card">
0261 |         <div style="display: flex; justify-content: space-between; align-items: center; margin-bottom: 0.8rem;">
0262 |             <span class="ticket-id">Ticket: {ticket_id}</span>
0263 |             <div>
0264 |                 {status_html}
0265 |                 {priority_html}
0266 |             </div>
0267 |         </div>
0268 |         <h3 style="color: #0F2744; font-weight: 700; margin-bottom: 0.4rem;">{complaint['title']}</h3>
0269 |         <p style="color: #475569; font-size: 0.95rem;">{complaint['description']}</p>
0270 |         <hr style="margin: 0.8rem 0; border: none; border-top: 1px solid #E2E8F0;">
0271 |         <div style="display: grid; grid-template-columns: repeat(auto-fit, minmax(220px, 1fr)); gap: 12px; font-size: 0.9rem;">
0272 |             <div><span style="color: #64748B;">Category:</span> <strong>{complaint['category']}</strong></div>
0273 |             <div><span style="color: #64748B;">Assigned Department:</span> {dept_tag}</div>
0274 |             <div><span style="color: #64748B;">Assigned Specialist:</span> <strong>{complaint.get('assigned_technician') or 'In Queue'}</
0275 |             <div><span style="color: #64748B;">Target SLA:</span> <span class="sla-pill">{complaint.get('sla_hours', 24)} Hours</span></d
0276 |             <div><span style="color: #64748B;">Priority Assessment:</span> <strong>{complaint['priority_reason']}</strong></div>
0277 |             <div><span style="color: #64748B;">Submitted On:</span> <strong>{complaint['created_at']}</strong></div>
0278 |         </div>
0279 |     </div>
0280 |     """, unsafe_allow_html=True)
```

```
    # Resolution Updates & Administrator Notes
    st.markdown("<h4 style='color: #0F2744; margin-top: 1.5rem;'>Official Administrator Responses & Resolution Notes</h4>", unsafe_allow_
    notes = db.get_resolution_notes(ticket_id, include_internal=False)
    if not notes:
        st.info("No technician notes posted yet. The department is currently triaging the ticket.")
    else:
        for n in notes:
            st.markdown(f"""
            <div style="background: #F8FAFC; border-left: 4px solid #10B981; padding: 12px 16px; border-radius: 0 8px 8px 0; margin-botto
                <div style="display: flex; justify-content: space-between; margin-bottom: 4px;">
                    <strong style="color: #065F46;">{n['author_name']} ({n['author_role']})</strong>
                    <span style="font-size: 0.8rem; color: #94A3B8;">{n['created_at']}</span>
                </div>
                <div style="color: #1E293B; font-size: 0.92rem;">{n['note_text']}</div>
            </div>
            """, unsafe_allow_html=True)

    # Student Rating & Feedback Section (When Resolved)
    if complaint["status"] == "Resolved":
        st.markdown("<h4 style='color: #0F2744; margin-top: 1.5rem;'>Student Resolution Feedback & Rating</h4>", unsafe_allow_html=True)
        if complaint.get("student_rating", 0) > 0:
            st.success(f"You rated this resolution: {complaint['student_rating']} / 5 Stars. Feedback: '{complaint.get('student_feedback'
        else:
            with st.form(f"feedback_form_{ticket_id}"):
                star_rating = st.slider("Rate Resolution Quality (1 = Poor, 5 = Excellent)", 1, 5, 5)
                fb_text = st.text_input("Comments on Resolution", placeholder="e.g. Prompt service, machines tested and functioning perfe
                btn_feedback = st.form_submit_button("Submit Resolution Rating", use_container_width=True)

                if btn_feedback:
                    db.submit_student_feedback(ticket_id, star_rating, fb_text)
                    st.success("Thank you for your feedback! Rating recorded.")
                    st.rerun()

    # Audit Trail History
    st.markdown("<h4 style='color: #0F2744; margin-top: 1.5rem;'>Ticket Status Lifecycle Timeline</h4>", unsafe_allow_html=True)
    history = db.get_complaint_history(ticket_id)
    for h in history:
        st.markdown(f"""
        <div class="history-item">
            <div class="history-dot"></div>
            <div class="history-time">{h['timestamp']} | by {h['changed_by']}</div>
            <div class="history-body">Status transition: <strong>{h['previous_status']}</strong> -> <strong>{h['new_status']}</strong></d
            {f'<div style="font-size: 0.85rem; color: #64748B; margin-top: 2px;">{h["notes"]}</div>' if h['notes'] else ''}
        </div>
        """, unsafe_allow_html=True)

def render_complaint_history(student_user):
    st.markdown("<div class='saas-card'>", unsafe_allow_html=True)
    st.markdown("<div class='saas-card-title'>My Complaint History</div>", unsafe_allow_html=True)

    complaints = db.get_complaints_by_student(student_user["id"])
    if not complaints:
        st.info("You haven't submitted any complaints yet.")
        st.markdown("</div>", unsafe_allow_html=True)
        return

    st.write(f"Showing **{len(complaints)}** complaints submitted by {student_user['full_name']}:")

    for c in complaints:
        col_h1, col_h2 = st.columns([4, 1.2])
        with col_h1:
            st.markdown(f"""
            <div>
                <span class="ticket-id">{c['ticket_id']}</span> | <strong>{c['title']}</strong> |
                {get_dept_tag_html(c['department'])}
                <div style="font-size: 0.82rem; color: #94A3B8; margin-top: 2px;">Submitted: {c['created_at']}</div>
            </div>
            """, unsafe_allow_html=True)
        with col_h2:
            st.markdown(f"{get_status_badge_html(c['status'], completed_by_dept=bool(c.get('completed_by_dept', 0)))} {get_priority_badge
        st.markdown("<hr style='margin: 0.5rem 0; border: none; border-top: 1px solid #F1F5F9;'>", unsafe_allow_html=True)

    st.markdown("</div>", unsafe_allow_html=True)

def render_student_profile(student_user):
    st.markdown("<div class='saas-card'>", unsafe_allow_html=True)
    st.markdown("<div class='saas-card-title'>Student Profile</div>", unsafe_allow_html=True)

    col1, col2 = st.columns(2)
```

```
0361 |     with col1:
0362 |         st.text_input("Full Name", value=student_user["full_name"], disabled=True)
0363 |         st.text_input("Username / Roll No.", value=student_user["username"], disabled=True)
0364 |         st.text_input("Email Address", value=student_user["email"], disabled=True)
0365 |     with col2:
0366 |         st.text_input("Department", value=student_user.get("department", "Computer Science"), disabled=True)
0367 |         st.text_input("Phone Number", value=student_user.get("phone", "N/A"), disabled=True)
0368 |         st.text_input("Account Role", value="Enrolled Student", disabled=True)
0369 |
0370 |     if st.button("Log Out of Student Account"):
0371 |         st.session_state["student_user"] = None
0372 |         st.rerun()
0373 |
0374 |     st.markdown("</div>", unsafe_allow_html=True)
```

## FILE: ui/department_page.py

**Type:** Python | **Lines:** 392

*Department Operations Field Station: Alerts, Roster Dispatch, Hardware Logs & Work Completion Trigger*

Path: smart_campus_ai/ui/department_page.py

```
0001 | import os
0002 | import streamlit as st
0003 | import pandas as pd
0004 | from config import DEPARTMENT_DETAILS, STATUS_LEVELS, PRIORITY_LEVELS
0005 | import database as db
0006 | from ui.styles import get_status_badge_html, get_priority_badge_html, get_dept_tag_html
0007 |
0008 | def render_department_page():
0009 |     # Retrieve active department from session state or select one
0010 |     dept_user = st.session_state.get("dept_user")
0011 |     admin_user = st.session_state.get("admin_user")
0012 |
0013 |     # If neither logged in, prompt to log in
0014 |     if not dept_user and not admin_user:
0015 |         st.markdown("""
0016 |         <div class="saas-card" style="text-align: center; padding: 2rem;">
0017 |             <div class="saas-card-title">Department Staff Authentication Required</div>
0018 |             <p style="color: #64748B;">Please sign in with your department officer account to view this department workspace.</p>
0019 |         </div>
0020 |         """, unsafe_allow_html=True)
0021 |         col_l, col_m, col_r = st.columns([1, 1, 1])
0022 |         with col_m:
0023 |             if st.button("Go to Department Login", use_container_width=True):
0024 |                 st.session_state["current_page"] = "login_dept"
0025 |                 st.rerun()
0026 |         return
0027 |
0028 |     # Determine which department to show
0029 |     available_depts = list(DEPARTMENT_DETAILS.keys())
0030 |     if "active_dept_name" not in st.session_state:
0031 |         if dept_user and dept_user.get("department") in available_depts:
0032 |             st.session_state["active_dept_name"] = dept_user["department"]
0033 |         else:
0034 |             st.session_state["active_dept_name"] = "Lab Maintenance"
0035 |
0036 |     current_dept = st.session_state["active_dept_name"]
0037 |     dept_meta = DEPARTMENT_DETAILS.get(current_dept, DEPARTMENT_DETAILS["Lab Maintenance"])
0038 |
0039 |     # Header with distinct Role Badge & department theme color
0040 |     st.markdown(f"""
```

```
    <div style="margin-bottom: 0.8rem;">
        <span class="role-badge-dept">ROLE: OPERATIONAL FIELD STATION - {current_dept.upper()}</span>
    </div>
    <div class="dept-banner" style="background: {dept_meta['bg_color']}; border-left: 6px solid {dept_meta['color']}; border-color: {dept
        <div style="display: flex; justify-content: space-between; align-items: flex-start; flex-wrap: wrap;">
            <div>
                <span style="font-size: 0.8rem; font-weight: 700; text-transform: uppercase; color: {dept_meta['color']}; letter-spacing:
                <h1 style="color: #0F172A; font-size: 1.9rem; font-weight: 800; margin: 4px 0 6px 0;">{current_dept}</h1>
                <p style="color: #475569; font-size: 0.95rem; margin: 0; max-width: 750px;">{dept_meta['description']}</p>
            </div>
            <div style="text-align: right; font-size: 0.85rem; color: #475569; margin-top: 8px;">
                <div><strong>Lead Officer:</strong> {dept_meta['head']}</div>
                <div><strong>Contact:</strong> <code>{dept_meta['email']}</code></div>
                <div><strong>Location:</strong> {dept_meta['location']}</div>
            </div>
        </div>
    </div>
    """, unsafe_allow_html=True)

    # Department vs Central Admin Role Clarification Expander
    with st.expander("Compare Portal Roles: Department Portal vs Central Admin Portal"):
        st.markdown("""
        <div class="portal-diff-container">
            <div class="portal-diff-col" style="border-left: 4px solid #0284C7;">
                <strong style="color: #0284C7; font-size: 1rem; display: block; margin-bottom: 6px;">Department Operations Portal (Tactic
                <ul style="color: #475569; font-size: 0.88rem; padding-left: 18px; margin: 0;">
                    <li><strong>Scope:</strong> Restricted strictly to tickets assigned to this department.</li>
                    <li><strong>Technician Dispatch:</strong> Assign named field engineers from your department roster.</li>
                    <li><strong>Equipment & Parts:</strong> Document hardware parts swapped, tests run, and technical fixes.</li>
                    <li><strong>Resolution Action:</strong> Clicking <strong>Mark Work Completed</strong> automatically marks the complai
                </ul>
            </div>
            <div class="portal-diff-col" style="border-left: 4px solid #0F2744;">
                <strong style="color: #0F2744; font-size: 1rem; display: block; margin-bottom: 6px;">Central Admin Portal (Strategic Gove
                <ul style="color: #475569; font-size: 0.88rem; padding-left: 18px; margin: 0;">
                    <li><strong>Scope:</strong> Institutional oversight across all 7 campus departments.</li>
                    <li><strong>Cross-Department Triage:</strong> Reassign misrouted tickets and override priorities.</li>
                    <li><strong>SLA Audit:</strong> Monitor university-wide turnaround times and prevent SLA breaches.</li>
                    <li><strong>Resolution Supervision:</strong> Review completed department resolutions and enforce audit standards.</li
                </ul>
            </div>
        </div>
        """, unsafe_allow_html=True)

    # Department Selector Switcher
    col_sw1, col_sw2 = st.columns([3, 1])
    with col_sw1:
        selected_from_menu = st.selectbox(
            "Switch Department Operations View",
            available_depts,
            index=available_depts.index(current_dept) if current_dept in available_depts else 0
        )
        if selected_from_menu != current_dept:
            st.session_state["active_dept_name"] = selected_from_menu
            st.rerun()

    # SECTION 1: IN-APP DEPARTMENT NOTIFICATIONS
    render_department_notifications(current_dept)

    # SECTION 2: DEPARTMENT ANALYTICS
    render_department_kpis(current_dept, dept_meta)

    # SECTION 3: DEPARTMENT COMPLAINT QUEUE & ACTIONS
    render_department_complaint_queue(current_dept, dept_meta)

def render_department_notifications(dept_name):
    notifs = db.get_notifications(department=dept_name, unread_only=True)

    st.markdown("<div class='saas-card'>", unsafe_allow_html=True)
    header_col, btn_col = st.columns([4, 1])
    with header_col:
        unread_count = len(notifs)
        st.markdown(f"""
        <div style="display: flex; align-items: center; gap: 10px;">
            <div class="saas-card-title" style="margin: 0;">Department Complaint Alerts</div>
            <span style="background: {'#DC2626' if unread_count > 0 else '#64748B'}; color: white; padding: 2px 8px; border-radius: 9999p
                {unread_count} Unread Alerts
            </span>
        </div>
        """, unsafe_allow_html=True)
```

```
    with btn_col:
        if unread_count > 0:
            if st.button("Mark Alerts Read", key=f"btn_mark_read_{dept_name}", use_container_width=True):
                db.mark_all_notifications_read(department=dept_name)
                st.rerun()

    if not notifs:
        st.markdown("""
        <div style="padding: 10px; color: #64748B; font-size: 0.9rem;">
            No pending unread alerts for this department. All notifications acknowledged.
        </div>
        """, unsafe_allow_html=True)
    else:
        for n in notifs[:5]:
            is_critical = n["urgency"] in ["Critical", "High"]
            box_class = "notif-box-critical" if is_critical else "notif-box-unread"
            st.markdown(f"""
            <div class="notif-box {box_class}">
                <div style="display: flex; justify-content: space-between; align-items: center; margin-bottom: 4px;">
                    <strong style="color: {'#B91C1C' if is_critical else '#1D4ED8'}; font-size: 0.95rem;">
                        [{n['urgency'].upper()}] {n['title']}
                    </strong>
                    <span style="font-size: 0.78rem; color: #94A3B8;">{n['created_at']}</span>
                </div>
                <div style="color: #334155; font-size: 0.88rem;">{n['message']}</div>
            </div>
            """, unsafe_allow_html=True)

    st.markdown("</div>", unsafe_allow_html=True)

def render_department_kpis(dept_name, dept_meta):
    stats = db.get_department_analytics(dept_name)

    col1, col2, col3, col4, col5, col6 = st.columns(6)

    with col1:
        st.markdown(f"""
        <div class="metric-card metric-card-total">
            <div class="metric-title">Total Ingested</div>
            <div class="metric-value">{stats['total']}</div>
            <div class="metric-sub">All-time tickets</div>
        </div>
        """, unsafe_allow_html=True)

    with col2:
        st.markdown(f"""
        <div class="metric-card metric-card-open">
            <div class="metric-title">Open / Triage</div>
            <div class="metric-value" style="color: #2563EB;">{stats['open']}</div>
            <div class="metric-sub">Pending diagnosis</div>
        </div>
        """, unsafe_allow_html=True)

    with col3:
        st.markdown(f"""
        <div class="metric-card metric-card-progress">
            <div class="metric-title">In Progress</div>
            <div class="metric-value" style="color: #D97706;">{stats['in_progress']}</div>
            <div class="metric-sub">Under active repair</div>
        </div>
        """, unsafe_allow_html=True)

    with col4:
        st.markdown(f"""
        <div class="metric-card metric-card-resolved">
            <div class="metric-title">Resolved</div>
            <div class="metric-value" style="color: #059669;">{stats['resolved']}</div>
            <div class="metric-sub">Completed work</div>
        </div>
        """, unsafe_allow_html=True)

    with col5:
        st.markdown(f"""
        <div class="metric-card metric-card-urgent">
            <div class="metric-title">High & Critical</div>
            <div class="metric-value" style="color: #DC2626;">{stats['high_critical']}</div>
            <div class="metric-sub">SLA critical</div>
        </div>
        """, unsafe_allow_html=True)

```

```
    with col6:
        rating_str = f"{stats['avg_rating']} / 5.0" if stats['avg_rating'] > 0 else "N/A"
        st.markdown(f"""
        <div class="metric-card metric-card-rating">
            <div class="metric-title">Student Rating</div>
            <div class="metric-value" style="color: #7C3AED;">{rating_str}</div>
            <div class="metric-sub">Satisfaction score</div>
        </div>
        """, unsafe_allow_html=True)

def render_department_complaint_queue(dept_name, dept_meta):
    st.markdown("<div class='saas-card' style='margin-top: 1.5rem;'>", unsafe_allow_html=True)
    st.markdown(f"<div class='saas-card-title'>{dept_name} Work Queue & Triage</div>", unsafe_allow_html=True)

    # Filter row
    f1, f2, f3 = st.columns([2, 1, 1])
    with f1:
        search_query = st.text_input("Filter Tickets (ID, Title, Student Name)", key=f"search_{dept_name}")
    with f2:
        filter_status = st.selectbox("Status Filter", ["All"] + STATUS_LEVELS, key=f"fstat_{dept_name}")
    with f3:
        filter_prio = st.selectbox("Priority Filter", ["All"] + PRIORITY_LEVELS, key=f"fprio_{dept_name}")

    dept_complaints = db.get_complaints_by_department(
        dept_name=dept_name,
        search=search_query.strip(),
        priority=filter_prio,
        status=filter_status
    )

    st.write(f"Showing **{len(dept_complaints)}** complaints assigned to **{dept_name}**:")

    # Export to CSV Button
    if dept_complaints:
        df_export = pd.DataFrame(dept_complaints)
        csv_data = df_export.to_csv(index=False).encode("utf-8")
        st.download_button(
            label=f"Export {dept_name} Tickets to CSV",
            data=csv_data,
            file_name=f"{dept_meta['slug']}_tickets.csv",
            mime="text/csv",
            key=f"export_csv_{dept_name}"
        )

    if not dept_complaints:
        st.info(f"No complaints in queue matching the criteria for {dept_name}.")
        st.markdown("</div>", unsafe_allow_html=True)
        return

    # Select complaint for active department action
    options = [f"{c['ticket_id']} - {c['title']} [{c['status']}] ({c['priority']})" for c in dept_complaints]
    selected_option = st.selectbox("Select Ticket to Review, Assign Technician or Resolve:", options, key=f"sel_ticket_{dept_name}")
    selected_ticket = selected_option.split(" - ")[0].strip()

    c = db.get_complaint_by_ticket_id(selected_ticket)
    if c:
        render_department_ticket_action_card(c, dept_meta)

    st.markdown("</div>", unsafe_allow_html=True)

def render_department_ticket_action_card(c, dept_meta):
    ticket_id = c["ticket_id"]
    st.markdown("<hr style='margin: 1.5rem 0; border: none; border-top: 1px solid #E2E8F0;'>", unsafe_allow_html=True)

    completed_flag = bool(c.get("completed_by_dept", 0))

    st.markdown(f"""
    <div style="background: #F8FAFC; border: 1px solid #CBD5E1; border-left: 5px solid {dept_meta['color']}; border-radius: 12px; padding
        <div style="display: flex; justify-content: space-between; align-items: center; margin-bottom: 0.8rem;">
            <span class="ticket-id" style="font-size: 1.25rem;">Ticket: {ticket_id}</span>
            <div>
                {get_status_badge_html(c['status'], completed_by_dept=completed_flag)}
                {get_priority_badge_html(c['priority'])}
            </div>
        </div>
        <h3 style="color: #0F2744; font-weight: 700; margin-bottom: 0.5rem;">{c['title']}</h3>
        <p style="color: #334155; font-size: 0.95rem; line-height: 1.5;">{c['description']}</p>

        <div style="display: grid; grid-template-columns: repeat(auto-fit, minmax(200px, 1fr)); gap: 10px; font-size: 0.88rem; margin-top
            <div><span style="color: #64748B;">Student:</span> <strong>{c['student_name']}</strong></div>
```

```
            <div><span style="color: #64748B;">Email:</span> <strong>{c['student_email']}</strong></div>
            <div><span style="color: #64748B;">Target SLA:</span> <span class="sla-pill">{c.get('sla_hours', 24)} Hours</span></div>
            <div><span style="color: #64748B;">Assigned Specialist:</span> <strong>{c.get('assigned_technician') or 'Unassigned'}</strong
            <div><span style="color: #64748B;">AI Reason:</span> <strong>{c['priority_reason']}</strong></div>
            <div><span style="color: #64748B;">Replaced Hardware:</span> <strong>{c.get('replaced_parts') or 'None Logged'}</strong></div
        </div>
    </div>
    """, unsafe_allow_html=True)

    # Attachment viewer
    if c.get("attachment_path") and os.path.exists(c["attachment_path"]):
        with st.expander("View Student Evidence Attachment"):
            ext = os.path.splitext(c["attachment_path"])[1].lower()
            if ext in [".png", ".jpg", ".jpeg", ".webp"]:
                st.image(c["attachment_path"], caption=f"Attachment: {os.path.basename(c['attachment_path'])}")
            else:
                with open(c["attachment_path"], "rb") as f:
                    st.download_button("Download Attached File", f.read(), file_name=os.path.basename(c["attachment_path"]))

    # RAG Grounding Info
    if c.get("rag_context"):
        with st.expander("View Campus Policy Grounding & SLA Guidelines"):
            st.info(c["rag_context"])

    dept_user = st.session_state.get("dept_user")
    admin_user = st.session_state.get("admin_user")
    active_actor = dept_user or admin_user or {}
    officer_name = active_actor.get("full_name") or dept_meta.get("head", "Department Officer")

    # SECTION A: DEDICATED "MARK WORK COMPLETED" ACTION
    st.markdown("""
    <div class="completed-action-card">
        <strong style="color: #065F46; font-size: 1.05rem; display: block; margin-bottom: 4px;">
            Work Completion & Resolution Trigger
        </strong>
        <p style="color: #047857; font-size: 0.88rem; margin: 0;">
            When field repairs and diagnostics are completed on the ground, click <strong>Mark Work Completed</strong> below.
            This will immediately update the ticket status to <strong>RESOLVED</strong> across the Central Admin Portal and Student Porta
        </p>
    </div>
    """, unsafe_allow_html=True)

    if c["status"] == "Resolved":
        st.success(f"Work on Ticket {ticket_id} has been COMPLETED and is officially marked as RESOLVED across the Central Admin Portal."
        if c.get("replaced_parts"):
            st.info(f"Replaced Parts / Hardware Swapped: {c['replaced_parts']}")
    else:
        with st.form(f"mark_completed_form_{ticket_id}"):
            comp_notes = st.text_area(
                "Diagnostic Findings & Work Completed Report *",
                placeholder="Detail physical inspection results, technician troubleshooting steps, and ground testing verification...",
                height=90
            )
            comp_parts = st.text_input(
                "Replaced Parts / Hardware / Materials Used (Optional)",
                placeholder="e.g. Swapped Gigabit Switch (Model GS-24), 2x Ethernet Patch Cables"
            )
            btn_mark_completed = st.form_submit_button(
                "Mark Work Completed -> Resolve Ticket in Admin Portal",
                use_container_width=True
            )

            if btn_mark_completed:
                if not comp_notes.strip():
                    st.error("Please provide work completed notes before resolving the ticket.")
                else:
                    ok, msg = db.mark_complaint_completed(
                        ticket_id=ticket_id,
                        completed_by=officer_name,
                        notes=comp_notes.strip(),
                        replaced_parts=comp_parts.strip()
                    )
                    if ok:
                        st.success(msg)
                        st.rerun()
                    else:
                        st.error(msg)

    # SECTION B: FIELD TECHNICIAN DISPATCH & ROSTER ASSIGNMENT
    st.markdown("<h4 style='color: #0F2744; margin-top: 1.8rem;'>Field Technician & Staff Assignment</h4>", unsafe_allow_html=True)
```

```
    with st.form(f"tech_assign_form_{ticket_id}"):
        available_techs = dept_meta.get("technicians", [])
        current_tech = c.get("assigned_technician", "")
        tech_idx = available_techs.index(current_tech) if current_tech in available_techs else 0

        selected_tech = st.selectbox("Assign Dedicated Technician / Specialist", available_techs, index=tech_idx)
        assign_notes = st.text_input("Assignment Instruction / Work Order Notes", placeholder="e.g. Inspect switch in Lab 2; replace netw
        btn_tech_assign = st.form_submit_button("Confirm Technician Assignment", use_container_width=True)

        if btn_tech_assign:
            ok, msg = db.assign_technician_to_ticket(ticket_id, selected_tech, officer_name, assign_notes.strip())
            if ok:
                # Also move status to In Progress if it was Open
                if c["status"] == "Open":
                    db.update_complaint_status(ticket_id, "In Progress", officer_name, f"Assigned to {selected_tech}. Work initiated.")
                st.success(msg)
                st.rerun()
            else:
                st.error(msg)

    # SECTION C: AUDIT TRAIL HISTORY
    st.markdown("<h4 style='color: #0F2744; margin-top: 1.8rem;'>Ticket Audit Trail & Action History</h4>", unsafe_allow_html=True)
    history = db.get_complaint_history(ticket_id)
    for h in history:
        st.markdown(f"""
        <div class="history-item">
            <div class="history-dot"></div>
            <div class="history-time">{h['timestamp']} | by {h['changed_by']}</div>
            <div class="history-body">{h['previous_status']} -> <strong>{h['new_status']}</strong></div>
            {f'<div style="font-size: 0.85rem; color: #64748B; margin-top: 2px;">{h["notes"]}</div>' if h['notes'] else ''}
        </div>
        """, unsafe_allow_html=True)
```

## FILE: ui/admin_dashboard.py

**Type:** Python | **Lines:** 439

*Central Executive Governance Console: KPIs, Department Analytics, Ticket Triage & Audit Feed*

Path: smart_campus_ai/ui/admin_dashboard.py

```
0001 | import os
0002 | import streamlit as st
0003 | import pandas as pd
0004 | from config import CATEGORIES, PRIORITY_LEVELS, STATUS_LEVELS, DEPARTMENTS, DEPARTMENT_DETAILS
0005 | import database as db
0006 | from ui.styles import get_status_badge_html, get_priority_badge_html, get_dept_tag_html
0007 |
0008 | def render_admin_dashboard():
0009 |     st.markdown("""
0010 |     <div style="margin-bottom: 0.8rem;">
0011 |         <span class="role-badge-admin">ROLE: UNIVERSITY CENTRAL EXECUTIVE GOVERNANCE</span>
0012 |     </div>
0013 |     <div style="display: flex; align-items: center; justify-content: space-between; margin-bottom: 1.5rem;">
0014 |         <div>
0015 |             <h2 style="color: #0F2744; font-weight: 800; margin: 0;">Central Administrator Management Console</h2>
0016 |             <p style="color: #64748B; margin: 0; font-size: 0.95rem;">
0017 |                 Campus-wide governance: monitor all 7 departments, triage incoming AI classifications, reassign tickets, and track resolu
0018 |             </p>
0019 |         </div>
0020 |     </div>
0021 |     """, unsafe_allow_html=True)
0022 |
0023 |     admin_user = st.session_state.get("admin_user")
0024 |     if not admin_user:
0025 |         render_admin_auth()
0026 |         return
0027 |
0028 |     # Visual Scope Comparison Expander: Department vs Central Admin
0029 |     with st.expander("Compare Portal Responsibilities: Central Admin vs Department Portals"):
0030 |         st.markdown("""
0031 |         <div class="portal-diff-container">
0032 |             <div class="portal-diff-col" style="border-left: 4px solid #0F2744;">
0033 |                 <strong style="color: #0F2744; font-size: 1rem; display: block; margin-bottom: 6px;">Central Admin Console (Strategic Gov
0034 |                 <ul style="color: #475569; font-size: 0.88rem; padding-left: 18px; margin: 0;">
0035 |                     <li><strong>Institutional Oversight:</strong> Global view across all 7 campus operating departments.</li>
0036 |                     <li><strong>Cross-Department Triage:</strong> Reassign misrouted complaints and adjust priority levels.</li>
0037 |                     <li><strong>SLA Compliance Audit:</strong> Monitor university-wide response metrics and resolution SLAs.</li>
0038 |                     <li><strong>Resolution Acknowledgment:</strong> When a department marks work <em>Completed</em>, it is automatically
0039 |                 </ul>
0040 |             </div>
```

```
            <div class="portal-diff-col" style="border-left: 4px solid #0284C7;">
                <strong style="color: #0284C7; font-size: 1rem; display: block; margin-bottom: 6px;">Department Portals (Tactical Field R
                <ul style="color: #475569; font-size: 0.88rem; padding-left: 18px; margin: 0;">
                    <li><strong>Department Queue:</strong> Restricted strictly to tickets routed to that specific department.</li>
                    <li><strong>Field Specialist Dispatch:</strong> Assign named technicians and engineers from department roster.</li>
                    <li><strong>Parts & Diagnostic Log:</strong> Record hardware parts swapped, tests conducted, and repair notes.</li>
                    <li><strong>Completion Action:</strong> Clicking <strong>Mark Work Completed</strong> updates the complaint to <stron
                </ul>
            </div>
        </div>
        """, unsafe_allow_html=True)

    # Admin sub-tabs
    tab1, tab2, tab3, tab4 = st.tabs([
        "Overview & Analytics",
        "Complaint Triage & Resolutions",
        "Campus Notification Feed",
        "Admin Account"
    ])

    with tab1:
        render_admin_analytics()

    with tab2:
        render_complaint_management(admin_user)

    with tab3:
        render_global_notifications()

    with tab4:
        render_admin_profile(admin_user)

def render_admin_auth():
    st.markdown("""
    <div class="saas-card" style="max-width: 500px; margin: 0 auto;">
        <h3 style="color: #0F2744; font-weight: 700; margin-bottom: 0.5rem; text-align: center;">Administrator Sign In</h3>
        <p style="color: #64748B; font-size: 0.9rem; text-align: center; margin-bottom: 1.5rem;">Role-based authorized access for central
    </div>
    """, unsafe_allow_html=True)

    col_l, col_m, col_r = st.columns([1, 2, 1])
    with col_m:
        with st.form("admin_login_form"):
            username = st.text_input("Administrator Username", value="admin")
            password = st.text_input("Admin Password", type="password", value="Admin@123")
            submitted = st.form_submit_button("Sign In as Administrator", use_container_width=True)

            if submitted:
                user = db.authenticate_user(username.strip(), password)
                if user and user["role"] == "admin":
                    st.session_state["admin_user"] = user
                    st.success("Authorized Administrator Access Granted.")
                    st.rerun()
                else:
                    st.error("Access denied. Invalid administrator credentials.")

        st.info("Default Administrator Demo: Username: admin | Password: Admin@123")

def render_admin_analytics():
    summary = db.get_analytics_summary()

    # Metrics Row
    m1, m2, m3, m4, m5 = st.columns(5)
    with m1:
        st.markdown(f"""
        <div class="metric-card metric-card-total">
            <div class="metric-title">Total Complaints</div>
            <div class="metric-value">{summary['total']}</div>
            <div class="metric-sub">Logged across campus</div>
        </div>
        """, unsafe_allow_html=True)
    with m2:
        st.markdown(f"""
        <div class="metric-card metric-card-open">
            <div class="metric-title">Open Tickets</div>
            <div class="metric-value" style="color: #2563EB;">{summary['open']}</div>
            <div class="metric-sub">Pending diagnosis</div>
        </div>
        """, unsafe_allow_html=True)
    with m3:
```

```
        st.markdown(f"""
        <div class="metric-card metric-card-progress">
            <div class="metric-title">In Progress</div>
            <div class="metric-value" style="color: #D97706;">{summary['in_progress']}</div>
            <div class="metric-sub">Under active repair</div>
        </div>
        """, unsafe_allow_html=True)
    with m4:
        st.markdown(f"""
        <div class="metric-card metric-card-resolved">
            <div class="metric-title">Resolved</div>
            <div class="metric-value" style="color: #059669;">{summary['resolved']}</div>
            <div class="metric-sub">Completed & closed</div>
        </div>
        """, unsafe_allow_html=True)
    with m5:
        st.markdown(f"""
        <div class="metric-card metric-card-urgent">
            <div class="metric-title">High & Critical</div>
            <div class="metric-value" style="color: #DC2626;">{summary['high_critical']}</div>
            <div class="metric-sub">SLA urgent priorities</div>
        </div>
        """, unsafe_allow_html=True)

    st.markdown("<hr style='margin: 1.8rem 0; border: none; border-top: 1px solid #E2E8F0;'>", unsafe_allow_html=True)

    # Analytics Charts
    c1, c2 = st.columns(2)

    with c1:
        st.markdown("<div class='saas-card'>", unsafe_allow_html=True)
        st.markdown("<div class='saas-card-title'>Department-wise Ticket Allocation</div>", unsafe_allow_html=True)
        if summary["by_dept"]:
            df_dept = pd.DataFrame(list(summary["by_dept"].items()), columns=["Department", "Complaints"]).set_index("Department")
            st.bar_chart(df_dept)
        else:
            st.info("No complaint data available yet.")
        st.markdown("</div>", unsafe_allow_html=True)

    with c2:
```

```
        st.markdown("<div class='saas-card'>", unsafe_allow_html=True)
        st.markdown("<div class='saas-card-title'>Priority Distribution</div>", unsafe_allow_html=True)
        if summary["by_priority"]:
            df_prio = pd.DataFrame(list(summary["by_priority"].items()), columns=["Priority", "Count"]).set_index("Priority")
            st.bar_chart(df_prio, color="#F59E0B")
        else:
            st.info("No priority data available yet.")
        st.markdown("</div>", unsafe_allow_html=True)

    # Recent Complaints quick glance
    st.markdown("<div class='saas-card'>", unsafe_allow_html=True)
    st.markdown("<div class='saas-card-title'>Recently Logged Complaints</div>", unsafe_allow_html=True)
    recent = db.get_all_complaints()[:5]
    if recent:
        for r in recent:
            rc1, rc2, rc3 = st.columns([1.5, 4, 2.5])
            with rc1:
                st.markdown(f"<span class='ticket-id'>{r['ticket_id']}</span>", unsafe_allow_html=True)
            with rc2:
                st.markdown(f"<strong>{r['title']}</strong> | <span style='color: #64748B;'>{r['student_name']}</span> <span style='backg
            with rc3:
                st.markdown(f"{get_status_badge_html(r['status'], completed_by_dept=bool(r.get('completed_by_dept', 0)))} {get_priority_b
            st.markdown("<hr style='margin: 0.4rem 0; border: none; border-top: 1px solid #F1F5F9;'>", unsafe_allow_html=True)
    else:
        st.info("No complaints registered yet.")
    st.markdown("</div>", unsafe_allow_html=True)

def render_global_notifications():
    st.markdown("<div class='saas-card'>", unsafe_allow_html=True)
    st.markdown("<div class='saas-card-title'>Real-Time Campus Complaint Notifications Feed</div>", unsafe_allow_html=True)

    col1, col2 = st.columns([4, 1])
    with col2:
        if st.button("Mark All Notifications Read", use_container_width=True):
            db.mark_all_notifications_read()
            st.rerun()

    notifs = db.get_notifications()
    if not notifs:
        st.info("No notification records logged.")
```

```
    else:
        for n in notifs:
            is_critical = n["urgency"] in ["Critical", "High"]
            box_cls = "notif-box-critical" if is_critical else "notif-box"
            st.markdown(f"""
            <div class="{box_cls}">
                <div style="display: flex; justify-content: space-between; margin-bottom: 4px;">
                    <strong style="color: #0F2744;">[{n['urgency'].upper()}] {n['title']}</strong>
                    <span style="font-size: 0.8rem; color: #94A3B8;">{n['created_at']} | Dept: {n['department']}</span>
                </div>
                <div style="color: #475569; font-size: 0.9rem;">{n['message']}</div>
            </div>
            """, unsafe_allow_html=True)
    st.markdown("</div>", unsafe_allow_html=True)

def render_complaint_management(admin_user):
    st.markdown("<div class='saas-card'>", unsafe_allow_html=True)
    st.markdown("<div class='saas-card-title'>Search, Filter & Manage Complaints</div>", unsafe_allow_html=True)

    # Filter controls
    f_col1, f_col2, f_col3, f_col4, f_col5 = st.columns([2, 1, 1, 1, 1])
    with f_col1:
        search_query = st.text_input("Search (Ticket ID, Title, Student Name)", placeholder="Type to search...")
    with f_col2:
        filter_status = st.selectbox("Status", ["All"] + STATUS_LEVELS)
    with f_col3:
        filter_prio = st.selectbox("Priority", ["All"] + PRIORITY_LEVELS)
    with f_col4:
        filter_cat = st.selectbox("Category", ["All"] + CATEGORIES)
    with f_col5:
        filter_dept = st.selectbox("Department", ["All"] + DEPARTMENTS)

    complaints = db.get_all_complaints(
        search=search_query.strip(),
        category=filter_cat,
        priority=filter_prio,
        status=filter_status,
        department=filter_dept
    )

    st.write(f"Displaying **{len(complaints)}** matching complaints:")

    # CSV Download Button
    if complaints:
        df = pd.DataFrame(complaints)
        csv_bytes = df.to_csv(index=False).encode("utf-8")
        st.download_button("Export Complaints to CSV", data=csv_bytes, file_name="all_campus_complaints.csv", mime="text/csv")

    if not complaints:
        st.info("No complaints match the selected filter criteria.")
        st.markdown("</div>", unsafe_allow_html=True)
        return

    # Select complaint for active inspection
    ticket_options = [f"{c['ticket_id']} - {c['title']} ({c['status']})" for c in complaints]
    selected_option = st.selectbox("Select a Complaint Ticket to Inspect / Resolve:", ticket_options)
    selected_ticket = selected_option.split(" - ")[0].strip()

    active_complaint = db.get_complaint_by_ticket_id(selected_ticket)
    if active_complaint:
        render_active_complaint_management_card(active_complaint, admin_user)

    st.markdown("</div>", unsafe_allow_html=True)

def render_active_complaint_management_card(c, admin_user):
    ticket_id = c["ticket_id"]
    st.markdown("<hr style='margin: 1.5rem 0; border: none; border-top: 2px solid #E2E8F0;'>", unsafe_allow_html=True)

    dept_tag = get_dept_tag_html(c['department'])
    completed_by_dept = bool(c.get("completed_by_dept", 0))

    st.markdown(f"""
    <div style="background: #F8FAFC; border: 1px solid #CBD5E1; border-radius: 12px; padding: 1.4rem; margin-bottom: 1.5rem;">
        <div style="display: flex; justify-content: space-between; align-items: center; margin-bottom: 0.8rem;">
            <span class="ticket-id" style="font-size: 1.3rem;">Ticket: {ticket_id}</span>
            <div>
                {get_status_badge_html(c['status'], completed_by_dept=completed_by_dept)}
                {get_priority_badge_html(c['priority'])}
            </div>
        </div>
```

```
        <h3 style="color: #0F2744; font-weight: 700; margin-bottom: 0.5rem;">{c['title']}</h3>
        <p style="color: #334155; font-size: 1rem; line-height: 1.5;">{c['description']}</p>

        <div style="display: grid; grid-template-columns: repeat(auto-fit, minmax(200px, 1fr)); gap: 10px; font-size: 0.88rem; margin-top
            <div><span style="color: #64748B;">Student:</span> <strong>{c['student_name']}</strong></div>
            <div><span style="color: #64748B;">Student Email:</span> <strong>{c['student_email']}</strong></div>
            <div><span style="color: #64748B;">Department:</span> {dept_tag}</div>
            <div><span style="color: #64748B;">Assigned Specialist:</span> <strong>{c.get('assigned_technician') or 'Unassigned'}</strong
            <div><span style="color: #64748B;">Target SLA:</span> <span class="sla-pill">{c.get('sla_hours', 24)} Hours</span></div>
            <div><span style="color: #64748B;">AI Priority Reason:</span> <strong>{c['priority_reason']}</strong></div>
            <div><span style="color: #64748B;">Replaced Hardware:</span> <strong>{c.get('replaced_parts') or 'None Logged'}</strong></div
            <div><span style="color: #64748B;">Review Flag:</span> <strong>{'Admin Review Needed' if c['needs_review'] else 'Confident AI
        </div>
    </div>
    """, unsafe_allow_html=True)

    # Department Completion Banner if resolved via department portal
    if completed_by_dept or c["status"] == "Resolved":
        st.markdown(f"""
        <div style="background: #ECFDF5; border: 1.5px solid #059669; border-radius: 10px; padding: 14px 18px; margin-bottom: 1.2rem;">
            <div style="display: flex; justify-content: space-between; align-items: center;">
                <div>
                    <strong style="color: #065F46; font-size: 1rem;">Field Resolution Verified: Status RESOLVED</strong>
                    <p style="color: #047857; font-size: 0.88rem; margin: 3px 0 0 0;">
                        This ticket was marked completed by <strong>{c['department']}</strong>. The complaint is officially marked as <st
                    </p>
                    {f'<div style="font-size: 0.85rem; color: #065F46; margin-top: 4px;"><strong>Hardware / Materials Replaced:</strong>
                </div>
            </div>
        </div>
        """, unsafe_allow_html=True)

    # Display Attachment if any
    if c.get("attachment_path") and os.path.exists(c["attachment_path"]):
        with st.expander("View Uploaded Student Attachment"):
            ext = os.path.splitext(c["attachment_path"])[1].lower()
            if ext in [".png", ".jpg", ".jpeg", ".webp"]:
                st.image(c["attachment_path"], caption=f"Attachment: {os.path.basename(c['attachment_path'])}")
            else:
                st.write(f"File: `{os.path.basename(c['attachment_path'])}`")
```

```
                with open(c["attachment_path"], "rb") as f:
                    st.download_button("Download Attached Document", f.read(), file_name=os.path.basename(c["attachment_path"]))

    # AI Reasoning & RAG Retrieval Context
    if c.get("rag_context"):
        with st.expander("Inspect AI Agentic RAG Context & Policy Grounding"):
            st.info(c["rag_context"])

    # Action Section 1: Department Reassignment & Priority Adjustment (Central Admin Exclusive Power)
    st.markdown("<h4 style='color: #0F2744;'>Central Admin Authority: Reassign Department or Adjust Priority</h4>", unsafe_allow_html=Tru
    with st.form(f"assign_form_{ticket_id}"):
        ac1, ac2 = st.columns(2)
        with ac1:
            new_dept = st.selectbox(
                "Transfer to Department",
                DEPARTMENTS,
                index=DEPARTMENTS.index(c["department"]) if c["department"] in DEPARTMENTS else 0
            )
        with ac2:
            new_prio = st.selectbox(
                "Adjust Complaint Priority",
                PRIORITY_LEVELS,
                index=PRIORITY_LEVELS.index(c["priority"]) if c["priority"] in PRIORITY_LEVELS else 1
            )
        reassign_notes = st.text_input("Administrative Justification Notes (Optional)")
        submitted_reassign = st.form_submit_button("Execute Department Reassignment & Update Priority", use_container_width=True)

        if submitted_reassign:
            ok, msg = db.update_complaint_assignment(
                ticket_id=ticket_id,
                new_department=new_dept,
                new_priority=new_prio,
                changed_by=admin_user["full_name"],
                notes=reassign_notes.strip()
            )
            if ok:
                st.success(msg)
                st.rerun()
            else:
                st.error(msg)
```

```
0361 |
0362 |     # Action Section 2: Assign Technician (Central Admin Override)
0363 |     st.markdown("<h4 style='color: #0F2744; margin-top: 1.2rem;'>Assign Field Specialist / Technician</h4>", unsafe_allow_html=True)
0364 |     with st.form(f"admin_tech_form_{ticket_id}"):
0365 |         dept_meta = DEPARTMENT_DETAILS.get(c["department"], {})
0366 |         tech_roster = dept_meta.get("technicians", ["Senior Systems Engineer", "Operations Specialist"])
0367 |         cur_tech = c.get("assigned_technician", "")
0368 |         tech_idx = tech_roster.index(cur_tech) if cur_tech in tech_roster else 0
0369 |
0370 |         chosen_tech = st.selectbox("Select Technician to Dispatch", tech_roster, index=tech_idx)
0371 |         tech_notes = st.text_input("Dispatch Directive", placeholder="e.g. Carry backup 24-port switch and Ethernet cables")
0372 |         btn_tech = st.form_submit_button("Dispatch Technician", use_container_width=True)
0373 |
0374 |         if btn_tech:
0375 |             ok, msg = db.assign_technician_to_ticket(ticket_id, chosen_tech, admin_user["full_name"], tech_notes.strip())
0376 |             if ok:
0377 |                 st.success(msg)
0378 |                 st.rerun()
0379 |             else:
0380 |                 st.error(msg)
0381 |
0382 |     # Action Section 3: Status Transition & Resolution Updates
0383 |     st.markdown("<h4 style='color: #0F2744; margin-top: 1.5rem;'>Update Status Lifecycle & Post Resolution</h4>", unsafe_allow_html=True)
0384 |     with st.form(f"status_form_{ticket_id}"):
0385 |         current_status_index = STATUS_LEVELS.index(c["status"]) if c["status"] in STATUS_LEVELS else 0
0386 |         new_status = st.selectbox("Update Status Lifecycle", STATUS_LEVELS, index=current_status_index)
0387 |         resolution_notes = st.text_area(
0388 |             "Resolution Notes / Response to Student * (Required if marking Resolved)",
0389 |             placeholder="Document actions taken, engineer dispatch notes, equipment replaced, or inspection findings...",
0390 |             height=100
0391 |         )
0392 |         submitted_status = st.form_submit_button("Submit Status Update & Notify Student", use_container_width=True)
0393 |
0394 |         if submitted_status:
0395 |             if new_status == "Resolved" and not resolution_notes.strip():
0396 |                 st.error("Security Requirement: A complaint cannot be marked 'Resolved' without descriptive resolution notes.")
0397 |             else:
0398 |                 ok, msg = db.update_complaint_status(
0399 |                     ticket_id=ticket_id,
0400 |                     new_status=new_status,
0401 |                     changed_by=admin_user["full_name"],
0402 |                     notes=resolution_notes.strip()
0403 |                 )
0404 |                 if ok:
0405 |                     st.success(msg)
0406 |                     st.rerun()
0407 |                 else:
0408 |                     st.error(msg)
0409 |
0410 |     # Resolution History Log
0411 |     st.markdown("<h4 style='color: #0F2744; margin-top: 1.5rem;'>Audit Trail & History</h4>", unsafe_allow_html=True)
0412 |     hist = db.get_complaint_history(ticket_id)
0413 |     for h in hist:
0414 |         st.markdown(f"""
0415 |         <div class="history-item">
0416 |             <div class="history-dot"></div>
0417 |             <div class="history-time">{h['timestamp']} | by {h['changed_by']}</div>
0418 |             <div class="history-body">{h['previous_status']} -> <strong>{h['new_status']}</strong></div>
0419 |             {f'<div style="font-size: 0.85rem; color: #64748B; margin-top: 2px;">{h["notes"]}</div>' if h['notes'] else ''}
0420 |         </div>
0421 |         """, unsafe_allow_html=True)
0422 |
0423 | def render_admin_profile(admin_user):
0424 |     st.markdown("<div class='saas-card'>", unsafe_allow_html=True)
0425 |     st.markdown("<div class='saas-card-title'>Administrator Profile</div>", unsafe_allow_html=True)
0426 |
0427 |     col1, col2 = st.columns(2)
0428 |     with col1:
0429 |         st.text_input("Admin Name", value=admin_user["full_name"], disabled=True)
0430 |         st.text_input("Username", value=admin_user["username"], disabled=True)
0431 |     with col2:
0432 |         st.text_input("Role", value="Campus Administrator / Governance Officer", disabled=True)
0433 |         st.text_input("Official Email", value=admin_user["email"], disabled=True)
0434 |
0435 |     if st.button("Log Out of Administrator Console"):
0436 |         st.session_state["admin_user"] = None
0437 |         st.rerun()
0438 |
0439 |     st.markdown("</div>", unsafe_allow_html=True)
```

## FILE: ui/kb_browser.py

**Type:** Python | **Lines:** 105

*Campus Knowledge Base Browser & Live ChromaDB Semantic Search Engine*

Path: smart_campus_ai/ui/kb_browser.py

```
0001 | import glob
0002 | from pathlib import Path
0003 | import streamlit as st
0004 | from config import KNOWLEDGE_BASE_DIR
0005 | from rag.vector_store import get_vector_store
0006 |
0007 | def render_kb_browser():
0008 |     st.markdown("""
0009 |     <div style="margin-bottom: 1.5rem;">
0010 |         <h2 style="color: #0F2744; font-weight: 800; margin: 0;">Campus Knowledge Base & RAG Engine</h2>
0011 |         <p style="color: #64748B; margin: 0; font-size: 0.95rem;">
0012 |             Explore institutional guidelines, laboratory protocols, hostel regulations, and test ChromaDB semantic retrieval.
0013 |         </p>
0014 |     </div>
0015 |     """, unsafe_allow_html=True)
0016 |
0017 |     tab1, tab2 = st.tabs(["Semantic Search & RAG Test", "Browse Official Documents"])
0018 |
0019 |     with tab1:
0020 |         st.markdown("<div class='saas-card'>", unsafe_allow_html=True)
0021 |         st.markdown("<div class='saas-card-title'>Test ChromaDB Semantic Search</div>", unsafe_allow_html=True)
0022 |         st.write("Enter any query or complaint sentence to observe how the Agentic RAG system retrieves relevant campus policy passages:"
0023 |
0024 |         # Quick preset query buttons
0025 |         st.markdown("<div style='margin-bottom: 8px;'><span style='font-size: 0.8rem; font-weight: 700; color: #64748B; text-transform: u
0026 |         qb1, qb2, qb3, qb4 = st.columns(4)
0027 |
0028 |         default_q = "computers in CSE laboratory practical examination maintenance rules"
0029 |
0030 |         with qb1:
0031 |             if st.button("CSE Lab Exam Rules", use_container_width=True):
0032 |                 st.session_state["kb_query_input"] = "computers in CSE laboratory practical examination maintenance rules"
0033 |         with qb2:
0034 |             if st.button("Office Locations", use_container_width=True):
0035 |                 st.session_state["kb_query_input"] = "Office Locations department helpdesk rooms"
0036 |         with qb3:
0037 |             if st.button("Hostel Plumbing & Water", use_container_width=True):
0038 |                 st.session_state["kb_query_input"] = "hostel residential life plumbing water outage rules"
0039 |         with qb4:
0040 |             if st.button("Campus SLAs Matrix", use_container_width=True):
```

```
                st.session_state["kb_query_input"] = "complaint priority matrix maintenance SLA turnaround"

        current_val = st.session_state.get("kb_query_input", default_q)

        col1, col2 = st.columns([4, 1])
        with col1:
            query = st.text_input(
                "Search Query",
                value=current_val,
                key="rag_search_query_box",
                placeholder="e.g. practical exam computers lab, hostel curfew plumbing, transport bus routes, office locations"
            )
        with col2:
            st.write("")
            st.write("")
            search_btn = st.button("Query ChromaDB", use_container_width=True)

        if query:
            vs = get_vector_store()
            results = vs.search(query.strip(), top_k=3)

            st.write(f"Retrieved **{len(results)}** grounded passages from ChromaDB:")

            border_colors = ["#2563EB", "#0284C7", "#059669"]

            for idx, res in enumerate(results, start=1):
                b_color = border_colors[(idx - 1) % len(border_colors)]

                st.markdown(f"""
                <div class="rag-result-card" style="border-left: 5px solid {b_color};">
                    <div style="display: flex; justify-content: space-between; align-items: center; flex-wrap: wrap; gap: 8px; margin-bot
                        <span style="font-weight: 800; color: #0F2744; font-size: 1.05rem;">
                            Result #{idx}: {res.get('title', 'Policy Document')}
                        </span>
                        <div style="display: flex; gap: 6px;">
                            <span class="rag-pill-section">Section: {res.get('section', 'General')}</span>
                            <span class="rag-pill-source">Source: {res.get('source', 'Campus Doc')}</span>
                        </div>
                    </div>
                </div>
                """, unsafe_allow_html=True)

                # Render the clean markdown text directly
                st.markdown(res.get("text", ""))
                st.markdown("<hr style='margin: 1.2rem 0; border: none; border-top: 1px dashed #CBD5E1;'>", unsafe_allow_html=True)

        st.markdown("</div>", unsafe_allow_html=True)

    with tab2:
        kb_files = glob.glob(str(Path(KNOWLEDGE_BASE_DIR) / "*.md"))
        if not kb_files:
            st.info("No knowledge base documents found.")
            return

        file_dict = {Path(f).name: f for f in kb_files}
        selected_file = st.selectbox("Select a Campus Document to Read:", list(file_dict.keys()))

        if selected_file:
            file_path = file_dict[selected_file]
            with open(file_path, "r", encoding="utf-8") as f:
                content = f.read()

            st.markdown("<div class='saas-card'>", unsafe_allow_html=True)
            st.markdown(content)
            st.markdown("</div>", unsafe_allow_html=True)
```

## FILE: ui/workflow_view.py

**Type:** Python | **Lines:** 91

*Interactive Multi-Agent Architecture Viewer & LangGraph State Machine Diagram*

Path: smart_campus_ai/ui/workflow_view.py

```
0001 | import streamlit as st
0002 |
0003 | def render_workflow_view():
0004 |     st.markdown("""
0005 |     <div style="margin-bottom: 1.5rem;">
0006 |         <h2 style="color: #0F2744; font-weight: 800; margin: 0;">Agentic AI System Architecture</h2>
0007 |         <p style="color: #64748B; margin: 0; font-size: 0.95rem;">
0008 |             Deep-dive into the 5 specialized AI agents and the LangGraph state machine coordinating autonomous triage.
0009 |         </p>
0010 |     </div>
0011 |     """, unsafe_allow_html=True)
0012 |
0013 |     agents_info = [
0014 |         {
0015 |             "num": "Agent 1",
0016 |             "name": "Complaint Understanding Agent",
0017 |             "objective": "Understands natural-language student complaints, filters emotional noise, and extracts core problem statements,
0018 |             "inputs": "Raw student grievance title, text description, and student context.",
0019 |             "outputs": "key_issue, impacted_asset, urgency_context, understanding_summary."
0020 |         },
0021 |         {
0022 |             "num": "Agent 2",
0023 |             "name": "Complaint Classification Agent",
0024 |             "objective": "Categorizes complaints into standard institutional taxonomies: Laboratory, Hostel, Classroom, Library, Transpor
0025 |             "inputs": "Extracted key issue and problem context.",
0026 |             "outputs": "category, category_confidence (0.0 to 1.0)."
0027 |         },
0028 |         {
0029 |             "num": "Agent 3",
0030 |             "name": "Priority Detection Agent",
0031 |             "objective": "Evaluates urgency, upcoming academic impact (e.g. exams tomorrow), and safety risks (e.g. sparks, leaks) to sco
0032 |             "inputs": "Urgency context, problem text, and category.",
0033 |             "outputs": "priority (Low/Medium/High/Critical), priority_reason, target SLA in hours."
0034 |         },
0035 |         {
0036 |             "num": "Agentic Router",
0037 |             "name": "Agentic RAG Knowledge Engine",
0038 |             "objective": "Decides dynamically whether campus policy documents, laboratory exam standby protocols, or hostel escalation ch
0039 |             "inputs": "Category, issue keywords, temporal urgency signals.",
0040 |             "outputs": "needs_rag (bool), ChromaDB semantic query, retrieved grounded passages, rag_context_summary."
```

```
        },
        {
            "num": "Agent 4",
            "name": "Department Routing Agent",
            "objective": "Identifies responsible operating department (Lab Maintenance, Hostel Admin, Estate, IT, etc.) using category an
            "inputs": "Category, issue, RAG knowledge citations.",
            "outputs": "department, routing_confidence, needs_review (bool), routing_reason."
        },
        {
            "num": "Agent 5",
            "name": "Ticket & Resolution Tracking Agent",
            "objective": "Generates unique sequential ticket IDs (CAMP-XXXX), writes persistent records into SQLite, dispatches departmen
            "inputs": "Complete consolidated triaged state.",
            "outputs": "ticket_id, initial status 'Open', database record, audit timeline, department notification."
        }
    ]

    for a in agents_info:
        st.markdown(f"""
        <div class="saas-card" style="border-left: 4px solid #2563EB; margin-bottom: 1rem;">
            <div style="display: flex; align-items: center; justify-content: space-between; margin-bottom: 0.5rem;">
                <strong style="color: #0F2744; font-size: 1.1rem;">{a['num']} - {a['name']}</strong>
            </div>
            <p style="color: #334155; font-size: 0.92rem; margin-bottom: 0.8rem;">{a['objective']}</p>
            <div style="display: grid; grid-template-columns: 1fr 1fr; gap: 10px; font-size: 0.85rem; background: #F8FAFC; padding: 10px;
                <div><span style="color: #64748B;">Inputs:</span> <code style="color: #2563EB;">{a['inputs']}</code></div>
                <div><span style="color: #64748B;">Outputs:</span> <code style="color: #10B981;">{a['outputs']}</code></div>
            </div>
        </div>
        """, unsafe_allow_html=True)

    # LangGraph State Graph Workflow Diagram (Pure ASCII/Text, no emojis)
    st.markdown("<h3 style='color: #0F2744; font-weight: 700; margin-top: 1.5rem; margin-bottom: 0.8rem;'>LangGraph Coordination State Gr
    st.markdown("""
    ```mermaid
    flowchart TD
        Start([Start: Student Submits Complaint]) --> A1[Agent 1: Complaint Understanding]
        A1 --> A2[Agent 2: Complaint Classification]
        A2 --> A3[Agent 3: Priority Detection]
        A3 --> Router{Agentic RAG Router: Is Policy Retrieval Needed?}
```

```
        Router -- Yes: Exam / Hazard / Guidelines --> RAGNode[Retrieve ChromaDB Campus Documents]
        Router -- No: Standard Inquiry --> A4[Agent 4: Department Routing]
        RAGNode --> A4
        A4 --> A5[Agent 5: Ticket & Tracking Agent]
        A5 --> DB[(SQLite Database: Insert Ticket CAMP-XXXX)]
        A5 --> Notif[Alerts Dispatched: Department Inbound Queue]
        DB --> Admin[Department & Admin Dashboards]
        Admin --> Resolution[Authorized Status Transition & Resolution Notes]
        Resolution --> StudentTrack[Student Tracking & Redressal Rating]
    ```
    """)
```